

```

import pandas as pd
import matplotlib.pyplot as plt
import os
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV, KFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, roc_curve, auc, roc_auc_score
from sklearn.metrics import f1_score
from PIL import Image
from tensorflow.keras.preprocessing import image
import numpy as np
import time
import seaborn as sns
from sklearn.preprocessing import LabelBinarizer
from sklearn.svm import SVC
from lime import lime_image
from skimage.segmentation import mark_boundaries
from sklearn.utils import resample
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import layers, models
from tensorflow.keras.layers import Dense, Flatten
from scikeras.wrappers import KerasClassifier
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from keras.optimizers import Adam

```

```

photos_csv = r"C:\Users\Gaurav\Downloads\photos.csv"
photos_folder = r"D:\Dataset\photos"

```

```

# Load dataset

```

```

df = pd.read_csv(photos_csv)

```

```

df

```

	photo_id	label
0	zsvj7vloL4L5jhYyPIuVwg	inside
1	HCUdRJHHm_e00CTlZetGLg	outside
2	vkr8T0scuJmGVvN2HJeLEA	drink
3	pve7D6NUrafHW3EA0Rubyw	food
4	H52Er-uBg6rNrHcReWTD2w	food
...	...	...
200095	4Zia9NkAfQNjMfcIDhwJ-g	food
200096	KB96KRZRhRm8hUkI-OpGEA	inside
200097	Klmojvaf2_2dP1XKzTsFmQ	food
200098	FNEiq7Mogec7t3l0aU5juw	drink
200099	NHEtLh7APk7Yssjo0h45VA	food

```

[200100 rows x 2 columns]

```

```

def bar_plot(data):
    class_counts = pd.Series(data).value_counts()

    # Plot bar chart
    class_counts.plot(kind="bar", color=["green", "red", "blue",
    "orange", "yellow"], edgecolor="black", figsize=(8, 6))

    # Add values on top of each bar
    for i, v in enumerate(class_counts):
        plt.text(i, v + 0.2, str(v), ha="center", fontsize=12)

    print("class_count : ", class_counts)
    # Customize plot
    plt.xlabel("Sentiment Labels")
    plt.ylabel("Frequency")
    plt.title("Sentiment Distribution in Traget ")
    plt.xticks(rotation=0) # Keep labels readable
    plt.grid(axis="y", linestyle="--", alpha=0.7)

    # Show the plot
    plt.show()

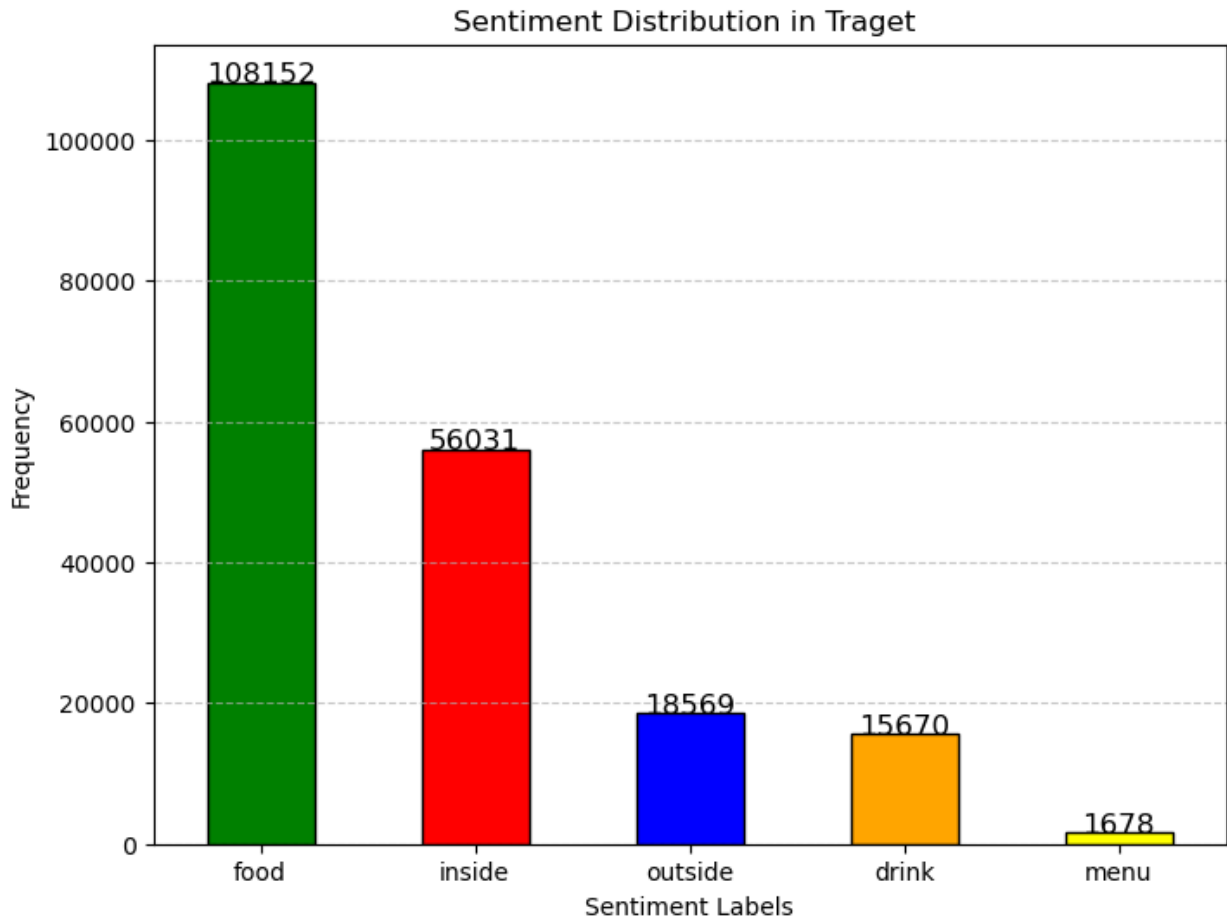
```

```
bar_plot(df['label'])
```

```

class_count : label
food         108152
inside        56031
outside       18569
drink         15670
menu          1678
Name: count, dtype: int64

```



```
# Separate majority and minority classes
df_majority_food = df[df['label'] == 'food']
df_majority_inside = df[df['label'] == 'inside']
df_majority_outside = df[df['label'] == 'outside']
df_majority_drink = df[df['label'] == 'drink']
df_minority_menu = df[df['label'] == 'menu']

# Undersample majority classes to 600 samples each
df_majority_food_downsampled = resample(df_majority_food,
replace=False, n_samples=300, random_state=42)
df_majority_inside_downsampled = resample(df_majority_inside,
replace=False, n_samples=300, random_state=42)
df_majority_outside_downsampled = resample(df_majority_outside,
replace=False, n_samples=300, random_state=42)
df_majority_drink_downsampled = resample(df_majority_drink,
replace=False, n_samples=300, random_state=42)

# Oversample minority class to 600 samples
df_minority_menu_upsampled = resample(df_minority_menu, replace=True,
n_samples=300, random_state=42)
```

```
# Combine all classes into a balanced DataFrame
df_balanced = pd.concat([df_majority_food_downsampled,
df_majority_inside_downsampled, df_majority_outside_downsampled,
df_majority_drink_downsampled, df_minority_menu_upsampled])
```

```
# Shuffle the DataFrame
df_balanced = df_balanced.sample(frac=1,
random_state=42).reset_index(drop=True)
```

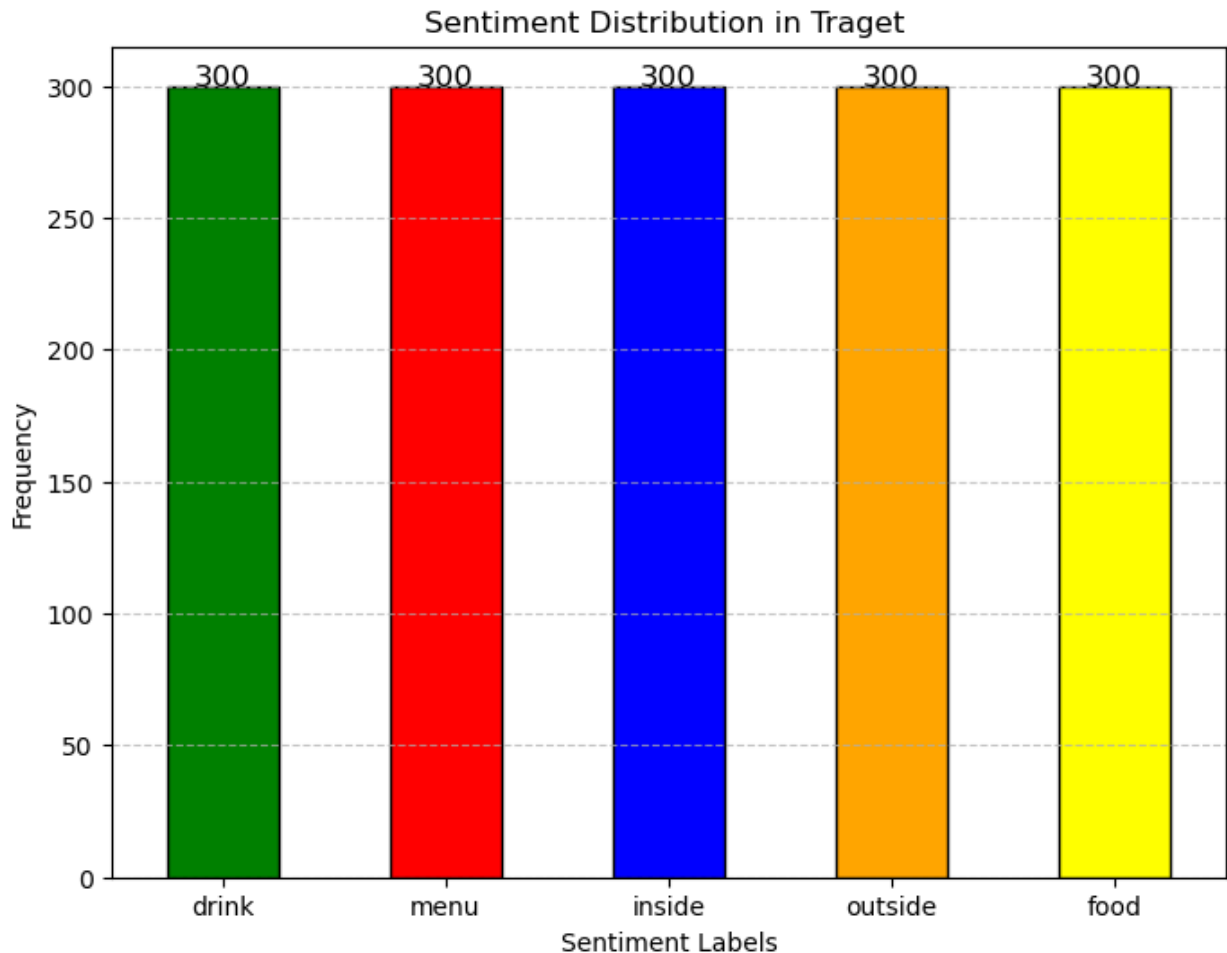
```
df_balanced
```

	photo_id	label
0	QcZzmuzz-eyQlekotAF50A	drink
1	5xIWbtcb4uIn_VW9pf8V3A	menu
2	Tmr4mNaDdf9uxM1YmaohCA	inside
3	TZgv-uQT_RiW2qNVlGE3JA	inside
4	6a7aYkFirlloeJycl1lvyIA	inside
...	...	...
1495	b2Z_mH6elUbFfZ__rFmxWg	drink
1496	Hja-ujyebg1J0xy0zeaY_w	menu
1497	JLe7-8Dzxieei0Hn-m1r5Q	outside
1498	FIw7ce992NJheBr9qEFHCA	menu
1499	lCcxfd5vG25sxzIf69qTXA	drink

```
[1500 rows x 2 columns]
```

```
bar_plot(df_balanced['label'])
```

```
class_count : label
drink        300
menu         300
inside       300
outside      300
food         300
Name: count, dtype: int64
```



```
df = df_balanced

# Build a dictionary to map photo_id to the image file paths
photo_id_to_path = {}
for file in os.listdir(photos_folder):
    # Assuming the photo_id is in the file name (e.g., photo_id.jpg)
    photo_id = file.split('.')[0] # Adjust based on your file naming
    # convention
    photo_id_to_path[photo_id] = os.path.join(photos_folder, file)

# Open the image
img = Image.open("D:\Dataset\photos\__02nEL2xViYvZihvV4_hw.jpg")

# Get the image size (width, height)
width, height = img.size
print(f"Width: {width}, Height: {height}")

Width: 400, Height: 400

<>:2: SyntaxWarning: invalid escape sequence '\D'
<>:2: SyntaxWarning: invalid escape sequence '\D'
```

```
C:\Users\Gaurav\AppData\Local\Temp\ipykernel_35808\3232345269.py:2:
SyntaxWarning: invalid escape sequence '\D'
img = Image.open("D:\Dataset\photos\__02nEL2xViYvZihvV4_hw.jpg")
```

```
# Function to preprocess images (resize to 224x224 and normalize)
```

```
def preprocess_image(img_path, target_size=(224, 224)):
```

```
    try:
```

```
        img = image.load_img(img_path, target_size=target_size)
```

```
        img_array = image.img_to_array(img)
```

```
        return img_array / 255.0 # Normalize the image
```

```
    except Exception as e:
```

```
        print(f"Error occurred: {e}")
```

```
        print(f"Image path: {img_path}")
```

```
        return None
```

```
# Preprocess images for each photo_id
```

```
x_images = []
```

```
y_labels = []
```

```
for _, row in df.iterrows():
```

```
    photo_id = row['photo_id']
```

```
    label = row['label']
```

```
# Get the image path based on the photo_id
```

```
img_path = photo_id_to_path.get(photo_id)
```

```
if img_path:
```

```
    # Preprocess the image
```

```
    img = preprocess_image(img_path)
```

```
    if (img is not None) :
```

```
        x_images.append(img)
```

```
        y_labels.append(label)
```

```
# Convert lists to numpy arrays for further processing
```

```
print(len(x_images))
```

```
x_images = np.array(x_images)
```

```
print(x_images)
```

```
print(y_labels)
```

```
print("\nShape of the image array: ", x_images.shape)
```

```
print("length of target: ", len(y_labels))
```

```
Error occurred: cannot identify image file <_io.BytesIO object at
0x000001A509791CB0>
```

```
Image path: D:\Dataset\photos\l_rMdwgrvjm2PyHyXBcBTw.jpg
```

```
1393
```

```
[[[0.9529412  0.9529412  0.9607843 ]
```

```
[0.9607843 0.9607843 0.96862745]
[0.9647059 0.9647059 0.972549 ]
...
[0.69411767 0.8117647 0.9137255 ]
[0.7607843 0.8627451 0.9529412 ]
[0.78431374 0.88235295 0.96862745]]
```

```
[[0.95686275 0.95686275 0.9647059 ]
 [0.9607843 0.9607843 0.96862745]
 [0.9647059 0.9647059 0.972549 ]
 ...
 [0.8509804 0.9372549 1.          ]
 [0.8745098 0.95686275 1.          ]
 [0.8862745 0.9607843 1.          ]]
```

```
[[0.9607843 0.9607843 0.96862745]
 [0.9607843 0.9607843 0.96862745]
 [0.9647059 0.9647059 0.972549 ]
 ...
 [0.89411765 0.95686275 1.          ]
 [0.8980392 0.9529412 0.99607843]
 [0.89411765 0.9411765 0.9882353 ]]
```

...

```
[[0.5137255 0.41960785 0.37254903]
 [0.49803922 0.39607844 0.3372549 ]
 [0.40784314 0.30980393 0.22352941]
 ...
 [0.42745098 0.36078432 0.32941177]
 [0.4117647 0.34901962 0.29803923]
 [0.38431373 0.32156864 0.2627451 ]]
```

```
[[0.5647059 0.47058824 0.43137255]
 [0.5529412 0.4509804 0.39215687]
 [0.5686275 0.47058824 0.38431373]
 ...
 [0.43529412 0.36078432 0.33333334]
 [0.41960785 0.34901962 0.3019608 ]
 [0.43137255 0.36078432 0.30588236]]
```

```
[[0.5647059 0.47058824 0.43137255]
 [0.54901963 0.44705883 0.3882353 ]
 [0.5294118 0.43137255 0.34509805]
 ...
 [0.45490196 0.38039216 0.3529412 ]
 [0.43137255 0.36078432 0.3137255 ]
 [0.4         0.32941177 0.27450982]]]
```

```
[[[0.67058825 0.60784316 0.54901963]
 [0.7529412 0.68235296 0.627451 ]
 [0.7254902 0.654902 0.6 ]
 ...
 [1. 1. 1. ]
 [1. 1. 1. ]
 [1. 1. 1. ]]]
```

```
[[0.6784314 0.6156863 0.5568628 ]
 [0.7411765 0.67058825 0.6156863 ]
 [0.7490196 0.6784314 0.62352943]
 ...
 [1. 1. 1. ]
 [1. 1. 1. ]
 [1. 1. 1. ]]
```

```
[[0.5411765 0.47058824 0.41568628]
 [0.53333336 0.4627451 0.40784314]
 [0.5294118 0.45882353 0.40392157]
 ...
 [1. 1. 1. ]
 [1. 1. 1. ]
 [1. 1. 1. ]]
```

...

```
[[0.4392157 0.3529412 0.29803923]
 [0.49019608 0.40392157 0.34901962]
 [0.5294118 0.44313726 0.3882353 ]
 ...
 [0.92156863 0.7764706 0.6745098 ]
 [0.92156863 0.7764706 0.6745098 ]
 [0.90588236 0.75686276 0.6666667 ]]
```

```
[[0.5019608 0.41568628 0.3647059 ]
 [0.48235294 0.39607844 0.34509805]
 [0.5176471 0.43137255 0.38039216]
 ...
 [0.90588236 0.74509805 0.6666667 ]
 [0.92156863 0.7607843 0.68235296]
 [0.92941177 0.76862746 0.6901961 ]]
```

```
[[0.5058824 0.41960785 0.36862746]
 [0.52156866 0.43529412 0.38431373]
 [0.5176471 0.43137255 0.38039216]
 ...
 [0.92156863 0.7607843 0.68235296]
 [0.93333334 0.77254903 0.69411767]
 [0.9254902 0.7647059 0.6862745 ]]]
```



```
[[[0.48235294 0.20784314 0.1764706 ]
  [0.49019608 0.21568628 0.18431373]
  [0.49411765 0.22745098 0.19215687]
  ...
  [0.90588236 0.18039216 0.2         ]
  [0.9098039  0.18431373 0.20392157]
  [0.9137255  0.1882353  0.20784314]]]
```

```
[[[0.45490196 0.16470589 0.12941177]
  [0.44705883 0.15686275 0.12156863]
  [0.47058824 0.1764706  0.15294118]
  ...
  [0.9098039  0.18431373 0.20392157]
  [0.9098039  0.18431373 0.20392157]
  [0.9137255  0.1882353  0.20784314]]]
```

```
[[[0.44705883 0.13333334 0.10588235]
  [0.4392157  0.1254902  0.09803922]
  [0.45882353 0.14509805 0.11764706]
  ...
  [0.9098039  0.18431373 0.20392157]
  [0.9137255  0.1882353  0.20784314]
  [0.91764706 0.19215687 0.21176471]]]
```

...

```
[[[0.13333334 0.1254902  0.14509805]
  [0.13333334 0.1254902  0.13725491]
  [0.13333334 0.1254902  0.13725491]
  ...
  [0.08235294 0.08235294 0.11372549]
  [0.08627451 0.08627451 0.11764706]
  [0.09019608 0.09019608 0.12941177]]]
```

```
[[[0.13333334 0.1254902  0.14509805]
  [0.13333334 0.1254902  0.13725491]
  [0.13333334 0.1254902  0.13725491]
  ...
  [0.08235294 0.08627451 0.10588235]
  [0.08627451 0.09019608 0.10980392]
  [0.09019608 0.09019608 0.12156863]]]
```

```
[[[0.13333334 0.1254902  0.14509805]
  [0.13333334 0.1254902  0.14509805]
  [0.13333334 0.1254902  0.13725491]
  ...
  [0.08235294 0.08627451 0.10588235]
  [0.08627451 0.09019608 0.10980392]
  [0.09019608 0.09411765 0.11372549]]]
```

...

```
[[[0.7254902 0.72156864 0.7137255 ]
  [0.29803923 0.28627452 0.3137255 ]
  [0.25490198 0.24313726 0.28627452]
  ...
  [0.7411765 0.6862745 0.63529414]
  [0.7254902 0.67058825 0.61960787]
  [0.7529412 0.69803923 0.64705884]]]
```

```
[[0.7490196 0.7411765 0.74509805]
 [0.44705883 0.43529412 0.47058824]
 [0.3137255 0.3019608 0.34509805]
  ...
 [0.74509805 0.6901961 0.6392157 ]
 [0.7372549 0.68235296 0.6313726 ]
 [0.75686276 0.7019608 0.6509804 ]]
```

```
[[0.7607843 0.7529412 0.75686276]
 [0.44313726 0.43137255 0.46666667]
 [0.38431373 0.37254903 0.41568628]
  ...
 [0.7607843 0.7058824 0.654902 ]
 [0.7607843 0.7058824 0.654902 ]
 [0.7764706 0.72156864 0.67058825]]]
```

...

```
[[0.19607843 0.1882353 0.19215687]
 [0.1882353 0.18039216 0.18431373]
 [0.16470589 0.16470589 0.16470589]
  ...
 [0.23921569 0.24705882 0.30588236]
 [0.22745098 0.23529412 0.28627452]
 [0.21176471 0.21176471 0.25882354]]]
```

```
[[0.19215687 0.18431373 0.1882353 ]
 [0.18431373 0.1764706 0.18039216]
 [0.16078432 0.16078432 0.16078432]
  ...
 [0.23921569 0.24705882 0.30588236]
 [0.22745098 0.23529412 0.28627452]
 [0.21568628 0.21568628 0.2627451 ]]
```

```
[[0.1882353 0.18039216 0.18431373]
 [0.1764706 0.16862746 0.17254902]
 [0.14901961 0.14901961 0.14901961]]]
```

```
...
[0.23529412 0.24313726 0.29411766]
[0.23529412 0.24313726 0.2901961 ]
[0.21960784 0.21960784 0.25882354]]]
```

```
[[[0.          0.01176471 0.          ]
  [0.          0.00784314 0.00392157]
  [0.          0.00392157 0.          ]
  ...
  [0.09019608 0.04313726 0.04313726]
  [0.10980392 0.05490196 0.05098039]
  [0.12156863 0.0627451  0.05098039]]]
```

```
[ [0.00392157 0.00392157 0.          ]
  [0.01568628 0.          0.          ]
  [0.01568628 0.          0.          ]
  ...
  [0.08627451 0.04313726 0.03529412]
  [0.09411765 0.03921569 0.03529412]
  [0.12941177 0.07058824 0.05098039]]]
```

```
[ [0.04705882 0.00784314 0.          ]
  [0.07058824 0.00784314 0.00784314]
  [0.07843138 0.00784314 0.00784314]
  ...
  [0.08627451 0.04313726 0.02745098]
  [0.08627451 0.03529412 0.01176471]
  [0.13725491 0.07843138 0.05098039]]]
```

```
...
[ [0.05098039 0.00784314 0.          ]
  [0.04313726 0.04705882 0.02352941]
  [0.04705882 0.06666667 0.03921569]
  ...
  [0.23137255 0.14901961 0.06666667]
  [0.22352941 0.13725491 0.05490196]
  [0.21176471 0.10588235 0.02352941]]]
```

```
[ [0.2          0.05490196 0.          ]
  [0.09803922 0.          0.          ]
  [0.08235294 0.00392157 0.          ]
  ...
  [0.24313726 0.17254902 0.08627451]
  [0.40784314 0.3254902  0.24313726]
  [0.19607843 0.09019608 0.00784314]]]
```

```
[ [0.5921569  0.33333334 0.12941177]
  [0.7607843  0.5568628  0.33333334]
```

```
[0.7882353 0.5882353 0.3647059 ]
...
[0.07058824 0.          0.          ]
[0.13725491 0.03921569 0.          ]
[0.45490196 0.3372549 0.23529412]]]
```

```
[[[0.2          0.19215687 0.20392157]
 [0.18039216 0.17254902 0.18431373]
 [0.17254902 0.16470589 0.1764706 ]
 ...
 [0.28627452 0.26666668 0.25490198]
 [0.28627452 0.26666668 0.25490198]
 [0.28627452 0.26666668 0.25490198]]]
```

```
[[[0.1882353 0.18039216 0.19215687]
 [0.1764706 0.16862746 0.18039216]
 [0.16862746 0.16078432 0.17254902]
 ...
 [0.28627452 0.26666668 0.25490198]
 [0.28627452 0.26666668 0.25490198]
 [0.28627452 0.26666668 0.25490198]]]
```

```
[[[0.1764706 0.16862746 0.18039216]
 [0.17254902 0.16470589 0.1764706 ]
 [0.16862746 0.16078432 0.17254902]
 ...
 [0.2901961 0.27058825 0.25882354]
 [0.2901961 0.27058825 0.25882354]
 [0.2901961 0.27058825 0.25882354]]]
```

...

```
[[[0.07450981 0.05098039 0.10588235]
 [0.07450981 0.0627451 0.09803922]
 [0.07450981 0.07058824 0.09019608]
 ...
 [0.16078432 0.19607843 0.34901962]
 [0.19215687 0.22352941 0.40392157]
 [0.1764706 0.1882353 0.40392157]]]
```

```
[[[0.07450981 0.05098039 0.10588235]
 [0.07450981 0.0627451 0.09803922]
 [0.07450981 0.07058824 0.09411765]
 ...
 [0.13333334 0.1764706 0.28627452]
 [0.16078432 0.2          0.34117648]
 [0.21960784 0.24313726 0.4392157 ]]]]
```

```
[[[0.07450981 0.05098039 0.10588235]
```

```
[0.07450981 0.0627451 0.09803922]
[0.07450981 0.07058824 0.09411765]
...
[0.12156863 0.16862746 0.2627451 ]
[0.16862746 0.21176471 0.3372549 ]
[0.34509805 0.37254903 0.5529412 ]]]
['drink', 'menu', 'inside', 'inside', 'inside', 'outside', 'drink',
'outside', 'menu', 'outside', 'menu', 'outside', 'menu', 'food',
'menu', 'menu', 'outside', 'inside', 'food', 'drink', 'drink',
'drink', 'menu', 'menu', 'inside', 'menu', 'menu', 'menu', 'drink',
'outside', 'menu', 'drink', 'food', 'inside', 'inside', 'inside',
'drink', 'menu', 'inside', 'menu', 'outside', 'drink', 'inside',
'inside', 'outside', 'drink', 'outside', 'inside', 'drink', 'menu',
'outside', 'food', 'inside', 'inside', 'drink', 'outside', 'inside',
'food', 'outside', 'inside', 'inside', 'menu', 'food', 'inside',
'outside', 'outside', 'food', 'outside', 'menu', 'outside', 'inside',
'inside', 'drink', 'drink', 'food', 'food', 'outside', 'outside',
'food', 'outside', 'drink', 'outside', 'drink', 'drink', 'food',
'food', 'menu', 'drink', 'inside', 'inside', 'food', 'drink',
'inside', 'drink', 'outside', 'drink', 'food', 'food', 'drink',
'food', 'outside', 'inside', 'menu', 'outside', 'food', 'food',
'inside', 'outside', 'food', 'food', 'drink', 'drink', 'menu', 'menu',
'inside', 'food', 'menu', 'menu', 'menu', 'inside', 'menu', 'food',
'menu', 'inside', 'food', 'inside', 'inside', 'drink', 'food',
'drink', 'food', 'menu', 'inside', 'menu', 'outside', 'drink',
'inside', 'outside', 'menu', 'menu', 'drink', 'menu', 'inside',
'menu', 'drink', 'food', 'inside', 'food', 'inside', 'inside',
'drink', 'menu', 'food', 'food', 'menu', 'drink', 'drink', 'inside',
'drink', 'inside', 'outside', 'food', 'food', 'drink', 'outside',
'inside', 'menu', 'food', 'inside', 'food', 'food', 'menu', 'drink',
'drink', 'menu', 'inside', 'outside', 'menu', 'menu', 'drink', 'food',
'drink', 'food', 'outside', 'inside', 'drink', 'outside', 'inside',
'inside', 'drink', 'drink', 'food', 'outside', 'inside', 'outside',
'food', 'drink', 'outside', 'menu', 'food', 'inside', 'menu', 'menu',
'food', 'menu', 'drink', 'outside', 'inside', 'food', 'inside',
'food', 'food', 'menu', 'menu', 'drink', 'inside', 'outside', 'drink',
'drink', 'menu', 'menu', 'menu', 'menu', 'outside', 'drink',
'outside', 'food', 'inside', 'inside', 'menu', 'drink', 'outside',
'drink', 'food', 'menu', 'outside', 'drink', 'drink', 'inside',
'inside', 'drink', 'drink', 'drink', 'food', 'inside', 'drink',
'menu', 'drink', 'inside', 'food', 'drink', 'inside', 'food',
'inside', 'menu', 'outside', 'menu', 'menu', 'menu', 'menu', 'menu',
'outside', 'food', 'outside', 'food', 'menu', 'food', 'inside',
'drink', 'food', 'menu', 'outside', 'drink', 'outside', 'menu',
'drink', 'outside', 'inside', 'inside', 'menu', 'drink', 'inside',
'inside', 'menu', 'drink', 'outside', 'inside', 'inside', 'menu',
'inside', 'outside', 'drink', 'outside', 'outside', 'outside',
'drink', 'drink', 'drink', 'outside', 'outside', 'outside', 'menu',
'food', 'menu', 'food', 'food', 'menu', 'outside', 'menu', 'drink',
```

'inside', 'drink', 'food', 'food', 'inside', 'food', 'inside',
'drink', 'drink', 'drink', 'drink', 'food', 'outside', 'inside',
'drink', 'food', 'inside', 'food', 'drink', 'outside', 'drink',
'outside', 'food', 'outside', 'drink', 'drink', 'menu', 'drink',
'drink', 'food', 'outside', 'food', 'menu', 'food', 'drink', 'drink',
'outside', 'menu', 'menu', 'menu', 'food', 'menu', 'menu', 'drink',
'drink', 'outside', 'inside', 'drink', 'drink', 'inside', 'drink',
'outside', 'drink', 'menu', 'inside', 'menu', 'outside', 'food',
'menu', 'menu', 'outside', 'drink', 'inside', 'menu', 'menu',
'inside', 'outside', 'inside', 'inside', 'food', 'menu', 'drink',
'inside', 'drink', 'food', 'outside', 'food', 'drink', 'outside',
'menu', 'food', 'food', 'inside', 'drink', 'drink', 'inside', 'food',
'menu', 'outside', 'inside', 'food', 'inside', 'outside', 'drink',
'inside', 'food', 'drink', 'drink', 'food', 'menu', 'food', 'drink',
'drink', 'outside', 'drink', 'menu', 'inside', 'inside', 'food',
'outside', 'outside', 'drink', 'menu', 'drink', 'food', 'food',
'outside', 'food', 'food', 'outside', 'food', 'outside', 'inside',
'drink', 'inside', 'menu', 'food', 'drink', 'inside', 'inside',
'outside', 'inside', 'inside', 'food', 'inside', 'inside', 'menu',
'inside', 'outside', 'menu', 'outside', 'food', 'menu', 'inside',
'inside', 'drink', 'menu', 'menu', 'inside', 'outside', 'food',
'menu', 'drink', 'inside', 'menu', 'outside', 'food', 'inside',
'food', 'inside', 'inside', 'menu', 'inside', 'inside', 'menu',
'menu', 'drink', 'inside', 'inside', 'menu', 'menu', 'menu', 'drink',
'drink', 'outside', 'outside', 'outside', 'outside', 'inside', 'menu',
'food', 'drink', 'menu', 'inside', 'drink', 'menu', 'drink', 'drink',
'food', 'menu', 'drink', 'inside', 'inside', 'menu', 'drink',
'inside', 'inside', 'inside', 'inside', 'outside', 'menu', 'outside',
'menu', 'inside', 'inside', 'menu', 'drink', 'food', 'food', 'food',
'outside', 'menu', 'outside', 'food', 'inside', 'outside', 'outside',
'outside', 'inside', 'food', 'menu', 'drink', 'food', 'drink',
'inside', 'food', 'drink', 'drink', 'outside', 'menu', 'drink',
'menu', 'inside', 'inside', 'drink', 'food', 'menu', 'inside',
'inside', 'food', 'drink', 'inside', 'menu', 'food', 'inside', 'food',
'food', 'outside', 'outside', 'outside', 'drink', 'menu', 'inside',
'menu', 'outside', 'menu', 'inside', 'inside', 'inside', 'inside',
'menu', 'outside', 'drink', 'inside', 'inside', 'inside', 'outside',
'menu', 'menu', 'drink', 'food', 'outside', 'outside', 'drink',
'drink', 'food', 'inside', 'inside', 'drink', 'food', 'drink',
'outside', 'inside', 'outside', 'outside', 'food', 'menu', 'menu',
'drink', 'outside', 'menu', 'outside', 'drink', 'menu', 'food',
'inside', 'outside', 'food', 'food', 'inside', 'drink', 'drink',
'menu', 'food', 'menu', 'food', 'menu', 'drink', 'outside', 'outside',
'outside', 'outside', 'outside', 'food', 'inside', 'inside', 'food',
'food', 'inside', 'food', 'outside', 'food', 'food', 'outside',
'outside', 'food', 'menu', 'outside', 'food', 'menu', 'inside',
'outside', 'drink', 'menu', 'outside', 'inside', 'drink', 'outside',
'inside', 'menu', 'menu', 'menu', 'drink', 'food', 'outside', 'food',
'outside', 'outside', 'food', 'outside', 'inside', 'inside', 'food',

'menu', 'food', 'food', 'drink', 'food', 'outside', 'inside', 'drink',
'drink', 'menu', 'drink', 'food', 'food', 'drink', 'food', 'menu',
'food', 'menu', 'inside', 'food', 'food', 'inside', 'inside',
'outside', 'inside', 'food', 'outside', 'outside', 'outside', 'drink',
'food', 'inside', 'food', 'menu', 'drink', 'menu', 'inside', 'menu',
'menu', 'outside', 'inside', 'food', 'menu', 'food', 'inside', 'food',
'outside', 'food', 'drink', 'food', 'food', 'outside', 'food',
'drink', 'inside', 'inside', 'outside', 'outside', 'menu', 'drink',
'drink', 'food', 'food', 'outside', 'food', 'inside', 'drink',
'outside', 'drink', 'menu', 'inside', 'outside', 'outside', 'drink',
'drink', 'drink', 'inside', 'menu', 'drink', 'menu', 'food',
'outside', 'menu', 'drink', 'drink', 'food', 'food', 'outside',
'menu', 'food', 'food', 'menu', 'inside', 'menu', 'food', 'drink',
'food', 'menu', 'drink', 'food', 'inside', 'outside', 'outside',
'food', 'inside', 'inside', 'drink', 'menu', 'inside', 'menu',
'outside', 'inside', 'outside', 'menu', 'food', 'inside', 'drink',
'outside', 'menu', 'outside', 'inside', 'food', 'menu', 'outside',
'drink', 'drink', 'drink', 'outside', 'menu', 'outside', 'outside',
'drink', 'inside', 'outside', 'outside', 'inside', 'drink', 'drink',
'food', 'drink', 'drink', 'drink', 'food', 'inside', 'drink', 'drink',
'drink', 'menu', 'inside', 'inside', 'menu', 'food', 'drink', 'food',
'menu', 'menu', 'food', 'food', 'food', 'outside', 'drink', 'outside',
'drink', 'drink', 'food', 'drink', 'outside', 'inside', 'inside',
'drink', 'inside', 'menu', 'inside', 'food', 'inside', 'outside',
'outside', 'menu', 'inside', 'menu', 'food', 'outside', 'menu',
'drink', 'menu', 'food', 'inside', 'food', 'drink', 'inside',
'outside', 'outside', 'food', 'menu', 'food', 'outside', 'drink',
'outside', 'inside', 'food', 'food', 'outside', 'food', 'inside',
'menu', 'inside', 'menu', 'drink', 'inside', 'menu', 'outside',
'inside', 'inside', 'menu', 'inside', 'food', 'food', 'inside',
'menu', 'menu', 'drink', 'menu', 'food', 'food', 'outside', 'inside',
'menu', 'menu', 'drink', 'inside', 'drink', 'food', 'menu', 'inside',
'outside', 'outside', 'outside', 'outside', 'food', 'food', 'food',
'menu', 'food', 'menu', 'drink', 'food', 'food', 'outside', 'food',
'food', 'inside', 'drink', 'menu', 'inside', 'inside', 'outside',
'drink', 'food', 'drink', 'drink', 'food', 'outside', 'menu',
'inside', 'inside', 'outside', 'menu', 'inside', 'outside', 'outside',
'menu', 'inside', 'food', 'drink', 'food', 'inside', 'inside',
'outside', 'food', 'food', 'menu', 'menu', 'inside', 'menu',
'outside', 'inside', 'food', 'menu', 'food', 'menu', 'outside',
'drink', 'menu', 'food', 'food', 'outside', 'menu', 'drink', 'drink',
'outside', 'food', 'drink', 'inside', 'outside', 'outside', 'menu',
'outside', 'inside', 'outside', 'outside', 'food', 'inside', 'menu',
'drink', 'outside', 'menu', 'food', 'outside', 'drink', 'menu',
'outside', 'inside', 'food', 'inside', 'food', 'outside', 'menu',
'menu', 'outside', 'outside', 'food', 'outside', 'drink', 'food',
'inside', 'menu', 'drink', 'menu', 'outside', 'drink', 'inside',
'food', 'inside', 'menu', 'inside', 'inside', 'inside', 'food',
'drink', 'menu', 'drink', 'food', 'drink', 'drink', 'outside',

'drink', 'food', 'menu', 'outside', 'inside', 'outside', 'menu',
'inside', 'inside', 'outside', 'inside', 'drink', 'menu', 'outside',
'food', 'outside', 'drink', 'outside', 'food', 'outside', 'outside',
'inside', 'food', 'outside', 'drink', 'outside', 'outside', 'food',
'menu', 'drink', 'outside', 'outside', 'food', 'drink', 'menu',
'food', 'menu', 'menu', 'inside', 'drink', 'drink', 'outside', 'menu',
'inside', 'drink', 'inside', 'food', 'inside', 'food', 'menu',
'outside', 'outside', 'drink', 'inside', 'drink', 'inside', 'inside',
'inside', 'food', 'outside', 'food', 'drink', 'outside', 'menu',
'inside', 'drink', 'inside', 'outside', 'drink', 'food', 'drink',
'outside', 'inside', 'food', 'drink', 'outside', 'outside', 'inside',
'menu', 'food', 'food', 'drink', 'drink', 'menu', 'food', 'drink',
'drink', 'menu', 'menu', 'outside', 'food', 'inside', 'inside',
'menu', 'menu', 'menu', 'inside', 'inside', 'food', 'drink', 'food',
'drink', 'inside', 'outside', 'menu', 'drink', 'menu', 'drink',
'menu', 'food', 'outside', 'food', 'food', 'inside', 'menu', 'food',
'outside', 'inside', 'menu', 'menu', 'outside', 'menu', 'outside',
'menu', 'menu', 'drink', 'menu', 'outside', 'food', 'drink', 'drink',
'outside', 'menu', 'outside', 'drink', 'food', 'menu', 'drink',
'menu', 'food', 'inside', 'menu', 'drink', 'food', 'food', 'inside',
'menu', 'food', 'menu', 'drink', 'food', 'drink', 'menu', 'drink',
'inside', 'food', 'outside', 'menu', 'outside', 'menu', 'outside',
'inside', 'menu', 'food', 'food', 'inside', 'outside', 'menu', 'menu',
'inside', 'drink', 'inside', 'menu', 'drink', 'food', 'inside',
'drink', 'outside', 'inside', 'food', 'drink', 'menu', 'outside',
'food', 'outside', 'drink', 'food', 'outside', 'food', 'food', 'menu',
'inside', 'food', 'menu', 'food', 'outside', 'food', 'menu', 'drink',
'inside', 'menu', 'outside', 'outside', 'food', 'food', 'outside',
'menu', 'drink', 'menu', 'menu', 'menu', 'inside', 'drink', 'menu',
'outside', 'menu', 'menu', 'menu', 'inside', 'outside', 'outside',
'menu', 'drink', 'outside', 'outside', 'menu', 'menu', 'menu', 'menu',
'menu', 'outside', 'outside', 'drink', 'inside', 'outside', 'inside',
'drink', 'menu', 'drink', 'inside', 'drink', 'drink', 'outside',
'menu', 'inside', 'food', 'inside', 'menu', 'outside', 'outside',
'outside', 'drink', 'food', 'inside', 'outside', 'inside', 'menu',
'menu', 'drink', 'outside', 'outside', 'drink', 'outside', 'outside',
'food', 'outside', 'inside', 'inside', 'drink', 'drink', 'outside',
'food', 'food', 'food', 'food', 'inside', 'food', 'inside', 'inside',
'inside', 'drink', 'drink', 'drink', 'menu', 'inside', 'food', 'food',
'drink', 'drink', 'inside', 'inside', 'outside', 'food', 'outside',
'inside', 'outside', 'drink', 'menu', 'inside', 'menu', 'outside',
'drink', 'food', 'food', 'outside', 'inside', 'outside', 'drink',
'inside', 'menu', 'drink', 'drink', 'menu', 'drink', 'food',
'outside', 'inside', 'menu', 'menu', 'food', 'menu', 'outside',
'inside', 'outside', 'food', 'food', 'inside', 'outside', 'inside',
'menu', 'food', 'outside', 'food', 'drink', 'outside', 'menu',
'inside', 'menu', 'outside', 'drink', 'food', 'drink', 'outside',
'inside', 'drink', 'inside', 'outside', 'outside', 'food', 'menu',
'drink', 'drink', 'menu', 'inside', 'outside', 'menu', 'inside',
'outside', 'menu', 'food', 'outside', 'drink', 'menu', 'food', 'menu',


```
'menu', 'inside', 'food', 'drink', 'drink', 'drink', 'menu',  
'outside', 'menu', 'drink']
```

```
Shape of the image array: (1393, 224, 224, 3)  
length of target: 1393
```

```
label_mapping = {'food' : 0, 'inside' : 1, 'outside' : 2, 'drink' : 3,  
'menu' : 4}  
reverse_label_mapping = {0 : 'food', 1 : 'inside', 2 : 'outside', 3 :  
'drink', 4 : 'menu'}
```

```
# Map the labels using the custom dictionary  
y = np.array([label_mapping[label] for label in y_labels])
```

```
# Flatten the images  
x = x_images.reshape(x_images.shape[0], -1)
```

```
print("Shape of the image array: ", x.shape)  
print("Shape of the target array: ", y.shape)
```

```
print(y[:100])
```

```
Shape of the image array: (1393, 150528)  
Shape of the target array: (1393,)  
[3 4 1 1 1 2 3 2 4 2 4 2 4 0 4 4 2 1 0 3 3 3 4 4 1 4 4 4 3 2 4 3 0 1 1  
1 3  
4 1 4 2 3 1 1 2 3 2 1 3 4 2 0 1 1 3 2 1 0 2 1 1 4 0 1 2 2 0 2 4 2 1 1  
3 3  
0 0 2 2 0 2 3 2 3 3 0 0 4 3 1 1 0 3 1 3 2 3 0 0 3 0]
```

```
from tensorflow.keras.applications.efficientnet import  
preprocess_input as effnet_preprocess
```

```
# Apply EfficientNet preprocessing to the entire batch of images  
preprocessed_images = effnet_preprocess(x_images)
```

```
x_train, x_test, y_train, y_test =  
train_test_split(preprocessed_images, y, test_size=0.2,  
random_state=42)  
print("Shape of x_train: ", x_train.shape) # Should print  
(num_samples, 224, 224, 3)  
print("Shape of y_train: ", y_train.shape)  
print("Shape of x_test: ", x_test.shape)  
print("Shape of y_test: ", y_test.shape)
```

```
Shape of x_train: (1114, 224, 224, 3)  
Shape of y_train: (1114,)  
Shape of x_test: (279, 224, 224, 3)  
Shape of y_test: (279,)
```

```

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from skimage.segmentation import mark_boundaries
from lime import lime_image
from tensorflow.keras.applications import VGG16, EfficientNetB0
from tensorflow.keras.applications.vgg16 import preprocess_input as
vgg_preprocess
from tensorflow.keras.applications.efficientnet import
preprocess_input as effnet_preprocess

def explain_with_lime(model, image_array, model_type="vgg",
top_labels=5, num_samples=1000):
    """
    Generates LIME explanations for a given image (numpy array).

    Parameters:
    - model: Trained TensorFlow model (VGG16 or EfficientNet)
    - image_array: Single image as a numpy array (shape: (224, 224,
3))
    - model_type: "vgg" for VGG16, "efficientnet" for EfficientNet
    - top_labels: Number of labels to explain
    - num_samples: Number of perturbations for LIME

    Returns:
    - Displays LIME explanation heatmap
    """

    # Expand dims to match the model input shape (1, 224, 224, 3)
    img_array = np.expand_dims(image_array, axis=0)

    # Apply the correct preprocessing
    if model_type == "vgg":
        img_array = vgg_preprocess(img_array) # Preprocess for VGG16
    elif model_type == "efficientnet":
        img_array = effnet_preprocess(img_array) # Preprocess for
EfficientNet

    # Define prediction function for LIME
    def model_predict(images):
        images = np.array([
            effnet_preprocess(img) if model_type == "efficientnet"
else vgg_preprocess(img)
            for img in images
        ])
        return model.predict(images)

    # Initialize LIME Explainer
    explainer = lime_image.LimeImageExplainer()

```

```

# Explain the prediction
explanation = explainer.explain_instance(
    image_array.astype('double'), # Convert to double for LIME
    model_predict,
    top_labels=top_labels,
    hide_color=0,
    num_samples=num_samples
)

# Get the top predicted label
top_label = explanation.top_labels[0]

# Get the explanation mask (increase num_features for better
visibility)
temp, mask = explanation.get_image_and_mask(
    top_label, positive_only=True, num_features=20,
hide_rest=False # Increased num_features for better visibility
)

# Display the result
plt.figure(figsize=(8, 8))
plt.imshow(mark_boundaries(temp, mask))
plt.title(f"LIME Explanation for {model_type.upper()} - Class
{top_label}")
plt.axis("off")
plt.show()

import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout, Flatten,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
import os
from sklearn.utils import resample
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt

# Load EfficientNetB0 without top layers
base_model = EfficientNetB0(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = False # Keep it frozen for now

image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)

```

```

x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Fine-tune the model
start_time = time.time()
model.fit(x_train, y_train, epochs=10, batch_size=32,
validation_data=(x_test, y_test))
end_time = time.time()

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

Epoch 1/10
35/35 _____ 31s 552ms/step - accuracy: 0.1898 - loss:
1.6449 - val_accuracy: 0.1864 - val_loss: 1.6132
Epoch 2/10
35/35 _____ 13s 356ms/step - accuracy: 0.1975 - loss:
1.6295 - val_accuracy: 0.1864 - val_loss: 1.6113
Epoch 3/10
35/35 _____ 14s 395ms/step - accuracy: 0.1904 - loss:
1.6255 - val_accuracy: 0.2043 - val_loss: 1.6094
Epoch 4/10
35/35 _____ 20s 383ms/step - accuracy: 0.2128 - loss:
1.6149 - val_accuracy: 0.1935 - val_loss: 1.6103
Epoch 5/10
35/35 _____ 23s 460ms/step - accuracy: 0.2077 - loss:
1.6200 - val_accuracy: 0.1864 - val_loss: 1.6102
Epoch 6/10
35/35 _____ 18s 389ms/step - accuracy: 0.2011 - loss:
1.6134 - val_accuracy: 0.1864 - val_loss: 1.6102
Epoch 7/10
35/35 _____ 23s 461ms/step - accuracy: 0.1823 - loss:
1.6160 - val_accuracy: 0.1864 - val_loss: 1.6107
Epoch 8/10
35/35 _____ 21s 469ms/step - accuracy: 0.2085 - loss:
1.6112 - val_accuracy: 0.1864 - val_loss: 1.6102
Epoch 9/10
35/35 _____ 21s 480ms/step - accuracy: 0.2230 - loss:
1.6080 - val_accuracy: 0.1864 - val_loss: 1.6099
Epoch 10/10
35/35 _____ 16s 470ms/step - accuracy: 0.2122 - loss:
1.6124 - val_accuracy: 0.1864 - val_loss: 1.6113
Training Time: 205.47 seconds

```

```

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

35/35 ————— 15s 361ms/step

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

9/9 ————— 3s 296ms/step

# Reverse the label mapping for confusion matrix
reverse_label_mapping = {v: k for k, v in label_mapping.items()}

def plot_confusion_matrix(y_true, y_pred, title="Confusion Matrix"):
    # Step 1: Compute the confusion matrix
    cm = confusion_matrix(y_true, y_pred)

    # Step 2: Convert the confusion matrix to a DataFrame for better
    # visualization
    cm_df = pd.DataFrame(cm, index=[reverse_label_mapping[i] for i in
    range(len(cm))],
                           columns=[reverse_label_mapping[i] for i in
    range(len(cm))])

    # Print confusion matrix dataframe
    print("Confusion Matrix:\n", cm_df)

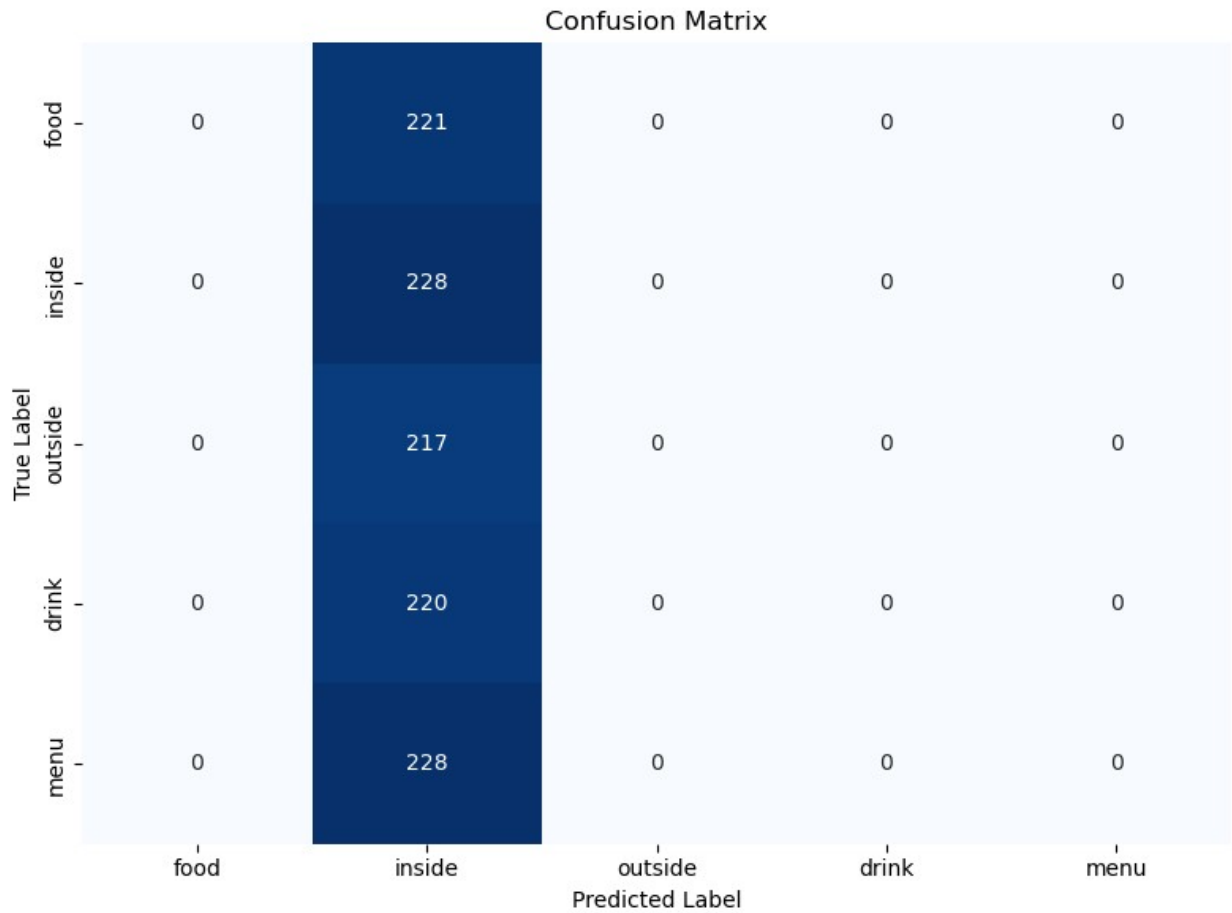
    # Step 3: Plot the confusion matrix as a heatmap
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm_df, annot=True, fmt='d', cmap="Blues",
    xticklabels=cm_df.columns, yticklabels=cm_df.columns, cbar=False)
    plt.title(title)
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.tight_layout()
    plt.show()

plot_confusion_matrix(y_train, y_train_pred_labels )

```

Confusion Matrix:

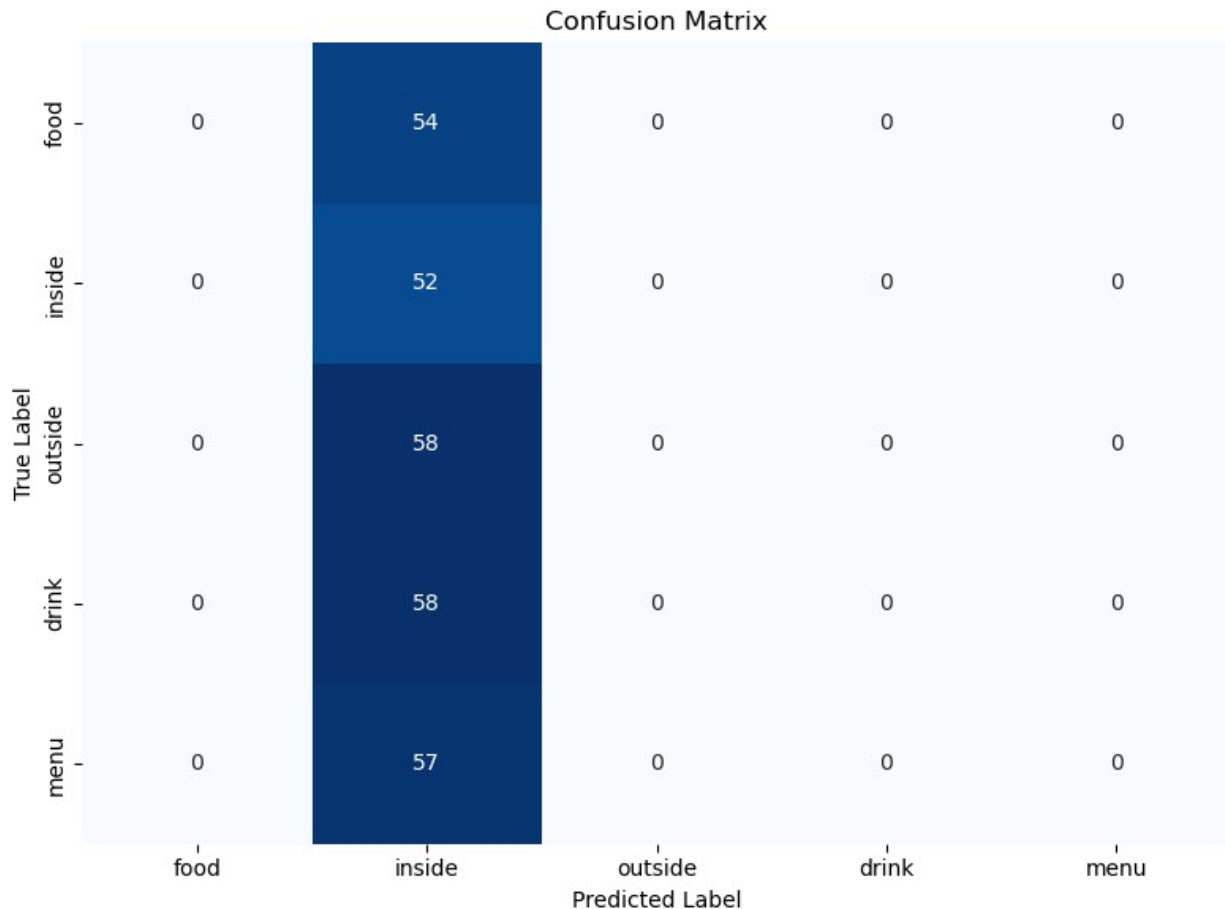
	food	inside	outside	drink	menu
food	0	221	0	0	0
inside	0	228	0	0	0
outside	0	217	0	0	0
drink	0	220	0	0	0
menu	0	228	0	0	0



```
plot_confusion_matrix(y_test, y_test_pred_labels)
```

Confusion Matrix:

	food	inside	outside	drink	menu
food	0	54	0	0	0
inside	0	52	0	0	0
outside	0	58	0	0	0
drink	0	58	0	0	0
menu	0	57	0	0	0



```
def predict_image(image_path):
    img = preprocess_image(image_path)
    if img is None:
        return "Error loading image"
    img = np.expand_dims(img, axis=0)
    prediction = model.predict(img)
    predicted_class = np.argmax(prediction)
    return label_mapping[predicted_class]

def plot_roc_curve(y_true, y_pred_probs):
    # Initialize LabelBinarizer for multi-class handling
    lb = LabelBinarizer()

    # Binarize the true labels (convert to one-hot encoding)
    y_true_bin = lb.fit_transform(y_true)

    # Compute ROC curve and AUC for each class
    fpr = dict()
    tpr = dict()
    roc_auc = dict()

    # Iterate through each class (works for multi-class)
```

```

    for i in range(y_true_bin.shape[1]):
        fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i],
y_pred_probs[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Plot all ROC curves in one plot
    plt.figure(figsize=(10, 8))

    # Plot ROC for each class
    for i in range(y_true_bin.shape[1]):
        label = lb.classes_[i]
        plt.plot(fpr[i], tpr[i],
label=f'{reverse_label_mapping[label]} (AUC = {roc_auc[i]:.2f})')

    # Plot diagonal line (random classifier)
    plt.plot([0, 1], [0, 1], color='gray', linestyle='--',
label='Random Classifier')

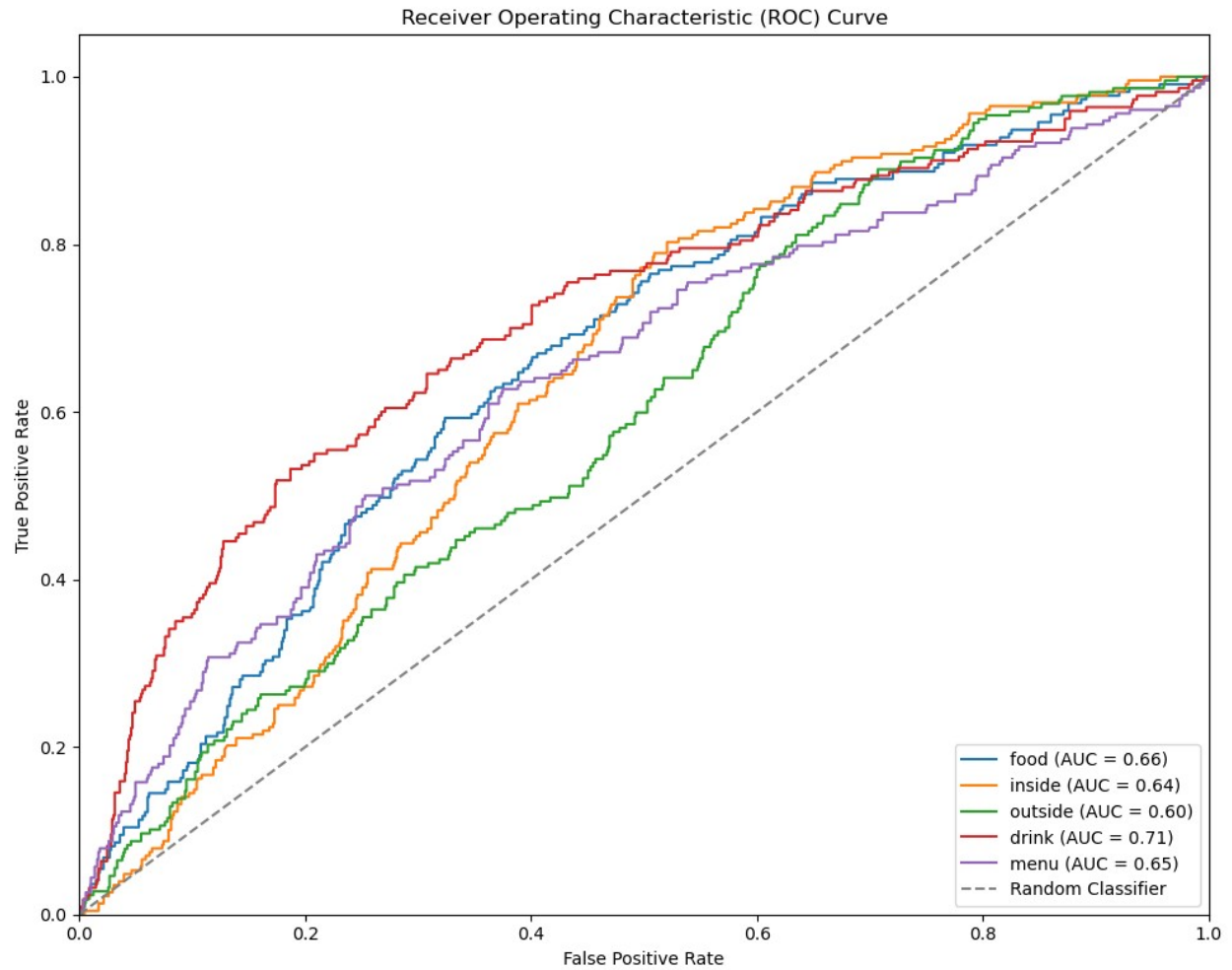
    # Labeling and Title
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Receiver Operating Characteristic (ROC) Curve')
    plt.legend(loc='lower right')

    # Show the plot
    plt.tight_layout()
    plt.show()

    # Print AUC values for each class
    for i in range(y_true_bin.shape[1]):
        print(f'AUC for class {reverse_label_mapping[lb.classes_[i]]}:
{roc_auc[i]:.2f}')

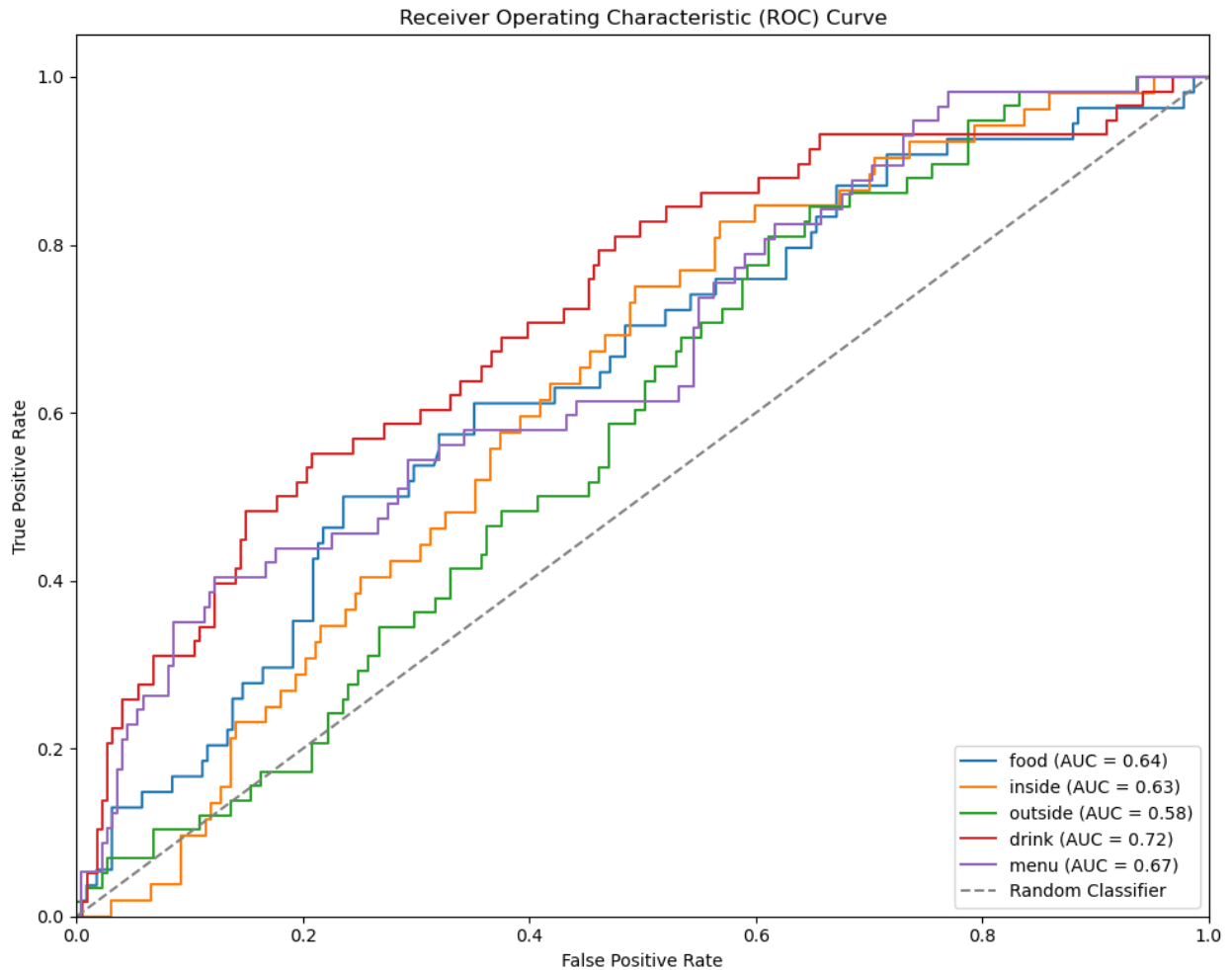
plot_roc_curve(y_train, y_train_pred )

```

```
AUC for class food: 0.66  
AUC for class inside: 0.64  
AUC for class outside: 0.60  
AUC for class drink: 0.71  
AUC for class menu: 0.65
```

```
plot_roc_curve(y_test, y_test_pred)
```



AUC for class food: 0.64
AUC for class inside: 0.63
AUC for class outside: 0.58
AUC for class drink: 0.72
AUC for class menu: 0.67

```
# Calculate and print the classification report for training data
print("Classification Report for Training Data:")
print(classification_report(y_train, y_train_pred_labels,
target_names=[reverse_label_mapping[i] for i in range(5)]))
```

Classification Report for Training Data:

	precision	recall	f1-score	support
food	0.00	0.00	0.00	221
inside	0.20	1.00	0.34	228
outside	0.00	0.00	0.00	217
drink	0.00	0.00	0.00	220
menu	0.00	0.00	0.00	228

accuracy			0.20	1114
macro avg	0.04	0.20	0.07	1114
weighted avg	0.04	0.20	0.07	1114

```
C:\Users\Gaurav\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1531: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

```
C:\Users\Gaurav\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1531: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

```
C:\Users\Gaurav\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1531: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

Calculate and print the classification report for test data

```
print("Classification Report for Test Data:")
print(classification_report(y_test, y_test_pred_labels,
target_names=[reverse_label_mapping[i] for i in range(5)]))
```

Classification Report for Test Data:

	precision	recall	f1-score	support
food	0.00	0.00	0.00	54
inside	0.19	1.00	0.31	52
outside	0.00	0.00	0.00	58
drink	0.00	0.00	0.00	58
menu	0.00	0.00	0.00	57
accuracy			0.19	279
macro avg	0.04	0.20	0.06	279
weighted avg	0.03	0.19	0.06	279

```
C:\Users\Gaurav\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1531: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
```

```
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

```
C:\Users\Gaurav\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1531: UndefinedMetricWarning: Precision is ill-
```

```

defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
C:\Users\Gaurav\anaconda3\Lib\site-packages\sklearn\metrics\
_classification.py:1531: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

```

lime

```

explain_with_lime(model, x_test[0], model_type="efficientnet")

{"model_id": "ce2257f56631479eaae26f179ba351b1", "version_major": 2, "version_minor": 0}

1/1 _____ 0s 329ms/step
1/1 _____ 0s 280ms/step
1/1 _____ 0s 351ms/step
1/1 _____ 0s 263ms/step
1/1 _____ 0s 370ms/step
1/1 _____ 0s 353ms/step
1/1 _____ 0s 353ms/step
1/1 _____ 0s 320ms/step
1/1 _____ 0s 350ms/step
1/1 _____ 0s 340ms/step
1/1 _____ 0s 275ms/step
1/1 _____ 0s 269ms/step
1/1 _____ 0s 282ms/step
1/1 _____ 0s 313ms/step
1/1 _____ 0s 224ms/step
1/1 _____ 0s 360ms/step
1/1 _____ 0s 265ms/step
1/1 _____ 0s 382ms/step
1/1 _____ 0s 300ms/step
1/1 _____ 0s 295ms/step
1/1 _____ 0s 416ms/step
1/1 _____ 0s 251ms/step
1/1 _____ 0s 313ms/step
1/1 _____ 0s 263ms/step
1/1 _____ 0s 393ms/step
1/1 _____ 0s 384ms/step
1/1 _____ 0s 287ms/step
1/1 _____ 0s 299ms/step
1/1 _____ 0s 408ms/step
1/1 _____ 0s 429ms/step
1/1 _____ 0s 397ms/step

```

1/1	_____	0s	253ms/step
1/1	_____	0s	349ms/step
1/1	_____	0s	272ms/step
1/1	_____	0s	324ms/step
1/1	_____	0s	303ms/step
1/1	_____	0s	276ms/step
1/1	_____	0s	265ms/step
1/1	_____	0s	229ms/step
1/1	_____	0s	317ms/step
1/1	_____	0s	252ms/step
1/1	_____	0s	210ms/step
1/1	_____	0s	289ms/step
1/1	_____	0s	306ms/step
1/1	_____	0s	297ms/step
1/1	_____	0s	217ms/step
1/1	_____	0s	255ms/step
1/1	_____	0s	283ms/step
1/1	_____	0s	404ms/step
1/1	_____	0s	329ms/step
1/1	_____	0s	248ms/step
1/1	_____	0s	283ms/step
1/1	_____	0s	275ms/step
1/1	_____	0s	305ms/step
1/1	_____	0s	263ms/step
1/1	_____	0s	279ms/step
1/1	_____	0s	288ms/step
1/1	_____	0s	251ms/step
1/1	_____	0s	213ms/step
1/1	_____	0s	279ms/step
1/1	_____	0s	239ms/step
1/1	_____	0s	292ms/step
1/1	_____	0s	296ms/step
1/1	_____	0s	281ms/step
1/1	_____	0s	372ms/step
1/1	_____	0s	213ms/step
1/1	_____	0s	268ms/step
1/1	_____	0s	353ms/step
1/1	_____	0s	367ms/step
1/1	_____	0s	271ms/step
1/1	_____	0s	275ms/step
1/1	_____	0s	254ms/step
1/1	_____	0s	451ms/step
1/1	_____	0s	289ms/step
1/1	_____	0s	307ms/step
1/1	_____	0s	325ms/step
1/1	_____	0s	287ms/step
1/1	_____	0s	224ms/step
1/1	_____	0s	232ms/step
1/1	_____	0s	281ms/step

1/1	_____	0s 350ms/step
1/1	_____	0s 292ms/step
1/1	_____	0s 453ms/step
1/1	_____	0s 269ms/step
1/1	_____	0s 284ms/step
1/1	_____	0s 307ms/step
1/1	_____	0s 292ms/step
1/1	_____	0s 246ms/step
1/1	_____	0s 275ms/step
1/1	_____	0s 263ms/step
1/1	_____	0s 247ms/step
1/1	_____	0s 281ms/step
1/1	_____	0s 254ms/step
1/1	_____	0s 290ms/step
1/1	_____	0s 275ms/step
1/1	_____	0s 263ms/step
1/1	_____	0s 233ms/step
1/1	_____	0s 257ms/step
1/1	_____	0s 267ms/step
1/1	_____	0s 367ms/step

LIME Explanation for EFFICIENTNET - Class 1



Unfreezing the last layers

```
tf.keras.backend.clear_session()
```

```
WARNING:tensorflow:From C:\Users\Gaurav\anaconda3\Lib\site-packages\
keras\src\backend\common\global_state.py:82: The name
tf.reset_default_graph is deprecated. Please use
tf.compat.v1.reset_default_graph instead.
```

```

# One-hot encode the target labels for training and testing data
y_train_onehot = to_categorical(y_train, num_classes=5)
y_test_onehot = to_categorical(y_test, num_classes=5)

for layer in model.layers[-10:]: # Unfreeze the last 10 layers (you
    can adjust this number)
    layer.trainable = True

model.compile(optimizer=Adam(learning_rate=0.00001), # Lower learning
    rate for fine-tuning
    loss='categorical_crossentropy',
    metrics=['accuracy'])

start_time = time.time()
# Retrain the model with the unfreezed layers
history_finetuned = model.fit(x_train, y_train_onehot,
    epochs=10, # Adjust the number of
    epochs for fine-tuning
    batch_size=32,
    validation_data=(x_test, y_test_onehot))

end_time = time.time()

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

Epoch 1/10
35/35 _____ 96s 2s/step - accuracy: 0.1731 - loss:
1.7361 - val_accuracy: 0.2043 - val_loss: 1.6077
Epoch 2/10
35/35 _____ 83s 2s/step - accuracy: 0.2358 - loss:
1.6749 - val_accuracy: 0.1900 - val_loss: 1.6109
Epoch 3/10
35/35 _____ 78s 2s/step - accuracy: 0.2552 - loss:
1.6263 - val_accuracy: 0.2473 - val_loss: 1.6071
Epoch 4/10
35/35 _____ 83s 2s/step - accuracy: 0.3084 - loss:
1.5580 - val_accuracy: 0.1756 - val_loss: 1.6134
Epoch 5/10
35/35 _____ 59s 2s/step - accuracy: 0.3288 - loss:
1.5406 - val_accuracy: 0.2043 - val_loss: 1.6114
Epoch 6/10
35/35 _____ 82s 2s/step - accuracy: 0.3900 - loss:
1.4632 - val_accuracy: 0.1971 - val_loss: 1.6090
Epoch 7/10
35/35 _____ 76s 2s/step - accuracy: 0.3973 - loss:
1.4239 - val_accuracy: 0.2115 - val_loss: 1.6071
Epoch 8/10
35/35 _____ 53s 2s/step - accuracy: 0.4491 - loss:
1.4005 - val_accuracy: 0.2581 - val_loss: 1.6057

```



```
Epoch 9/10
35/35 ————— 80s 1s/step - accuracy: 0.4748 - loss:
1.3387 - val_accuracy: 0.2616 - val_loss: 1.5990
Epoch 10/10
35/35 ————— 53s 2s/step - accuracy: 0.4858 - loss:
1.2992 - val_accuracy: 0.2903 - val_loss: 1.5881
Training Time: 743.64 seconds
```

```
y_train_pred_probs = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred_probs, axis=1)
```

```
35/35 ————— 16s 372ms/step
```

```
y_test_pred_probs = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred_probs, axis=1)
```

```
9/9 ————— 3s 297ms/step
```

```
# Classification Report for Training Data
print("\nClassification Report for Training Data:")
print(classification_report(np.argmax(y_train_onehot, axis=1),
y_train_pred_labels))
```

Classification Report for Training Data:

	precision	recall	f1-score	support
0	0.26	0.78	0.39	221
1	0.17	0.07	0.09	228
2	0.20	0.10	0.14	217
3	0.27	0.08	0.13	220
4	0.41	0.34	0.37	228
accuracy			0.27	1114
macro avg	0.26	0.27	0.22	1114
weighted avg	0.26	0.27	0.22	1114

```
# Classification Report for Test Data
print("\nClassification Report for Test Data:")
print(classification_report(np.argmax(y_test_onehot, axis=1),
y_test_pred_labels))
```

Classification Report for Test Data:

	precision	recall	f1-score	support
0	0.24	0.76	0.36	54
1	0.27	0.12	0.16	52
2	0.26	0.10	0.15	58
3	0.25	0.07	0.11	58

	4	0.53	0.42	0.47	57
accuracy				0.29	279
macro avg		0.31	0.29	0.25	279
weighted avg		0.31	0.29	0.25	279


```

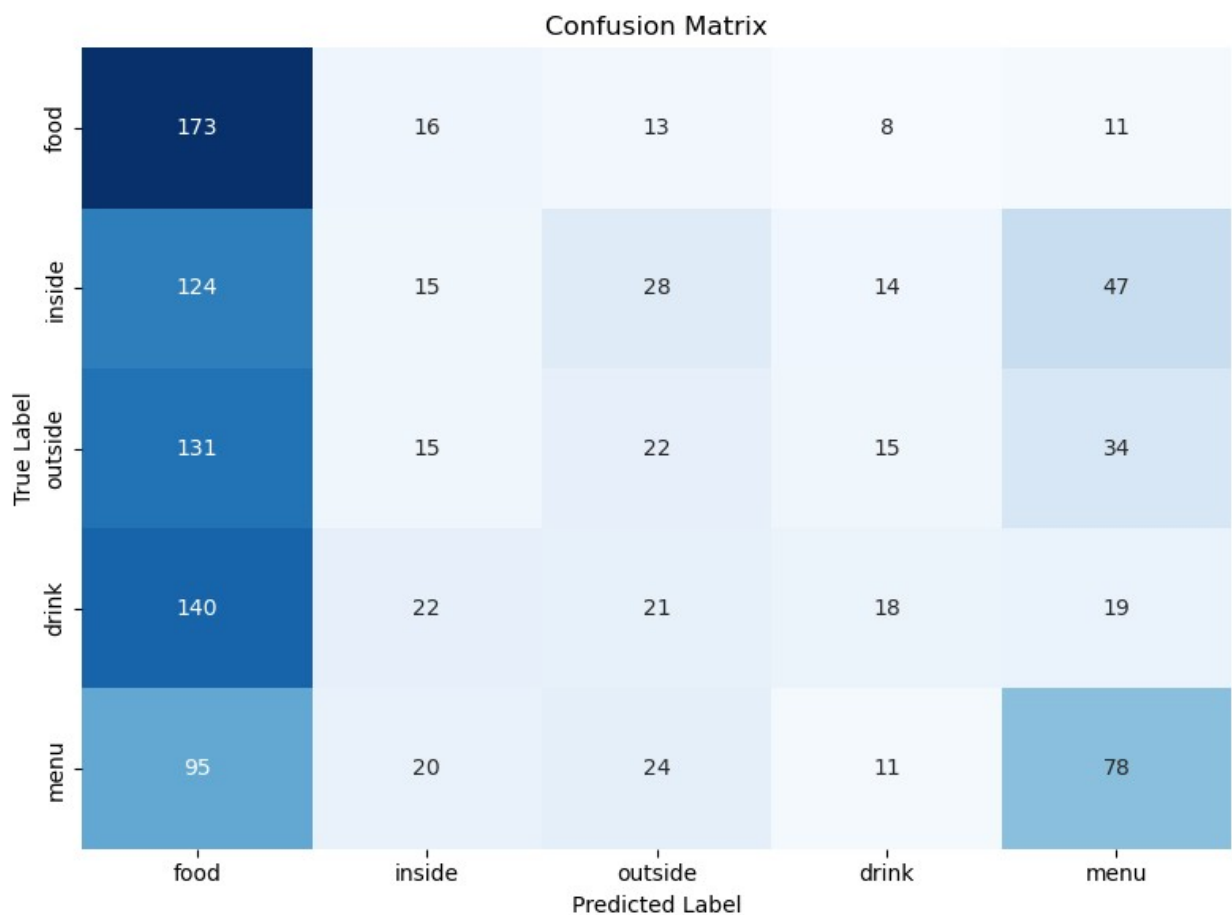
# Plot Confusion Matrix for Training Data
print("\nConfusion Matrix for Training Data:")
plot_confusion_matrix(np.argmax(y_train_onehot, axis=1),
y_train_pred_labels)

```

Confusion Matrix for Training Data:

Confusion Matrix:

	food	inside	outside	drink	menu
food	173	16	13	8	11
inside	124	15	28	14	47
outside	131	15	22	15	34
drink	140	22	21	18	19
menu	95	20	24	11	78

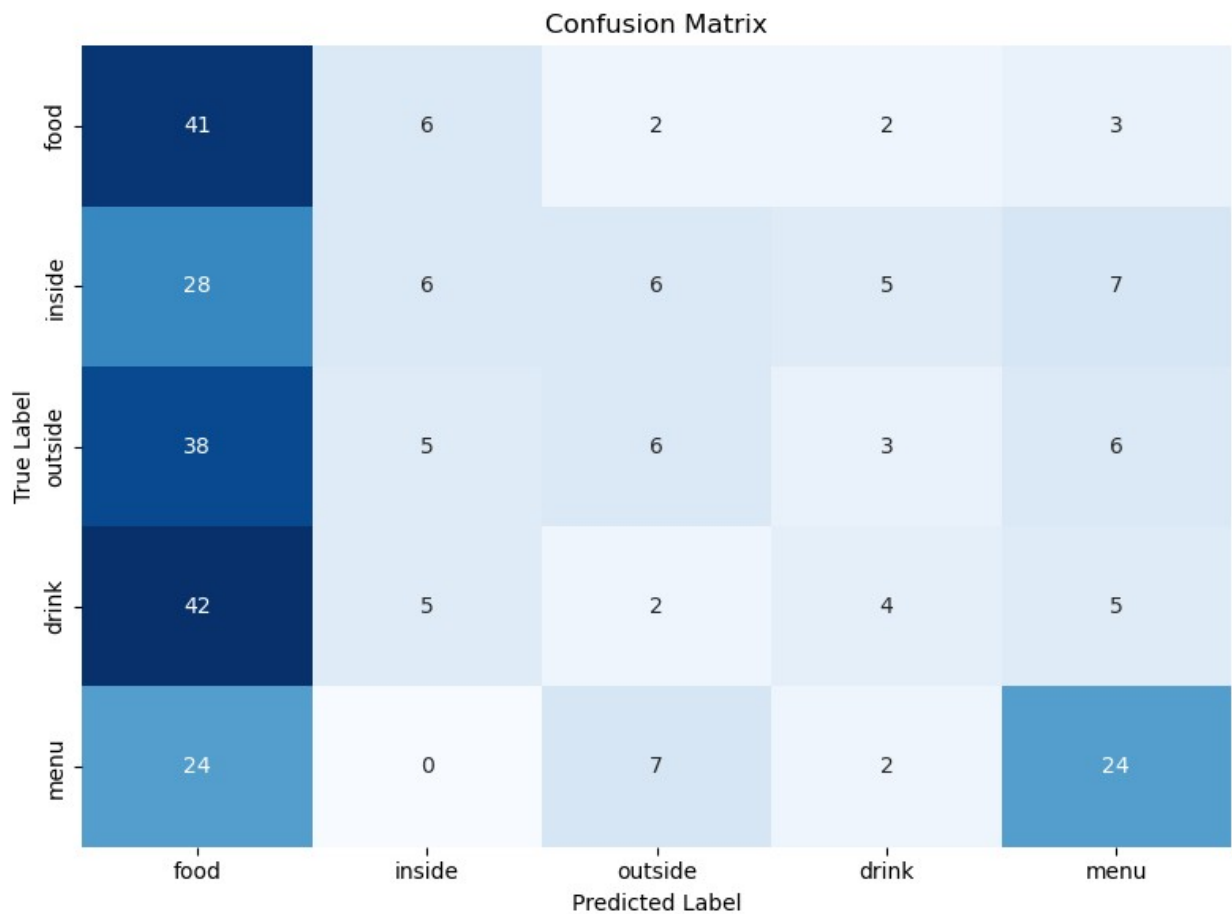


```
# Plot Confusion Matrix for Test Data
print("\nConfusion Matrix for Test Data:")
plot_confusion_matrix(np.argmax(y_test_onehot, axis=1),
y_test_pred_labels)
```

Confusion Matrix for Test Data:

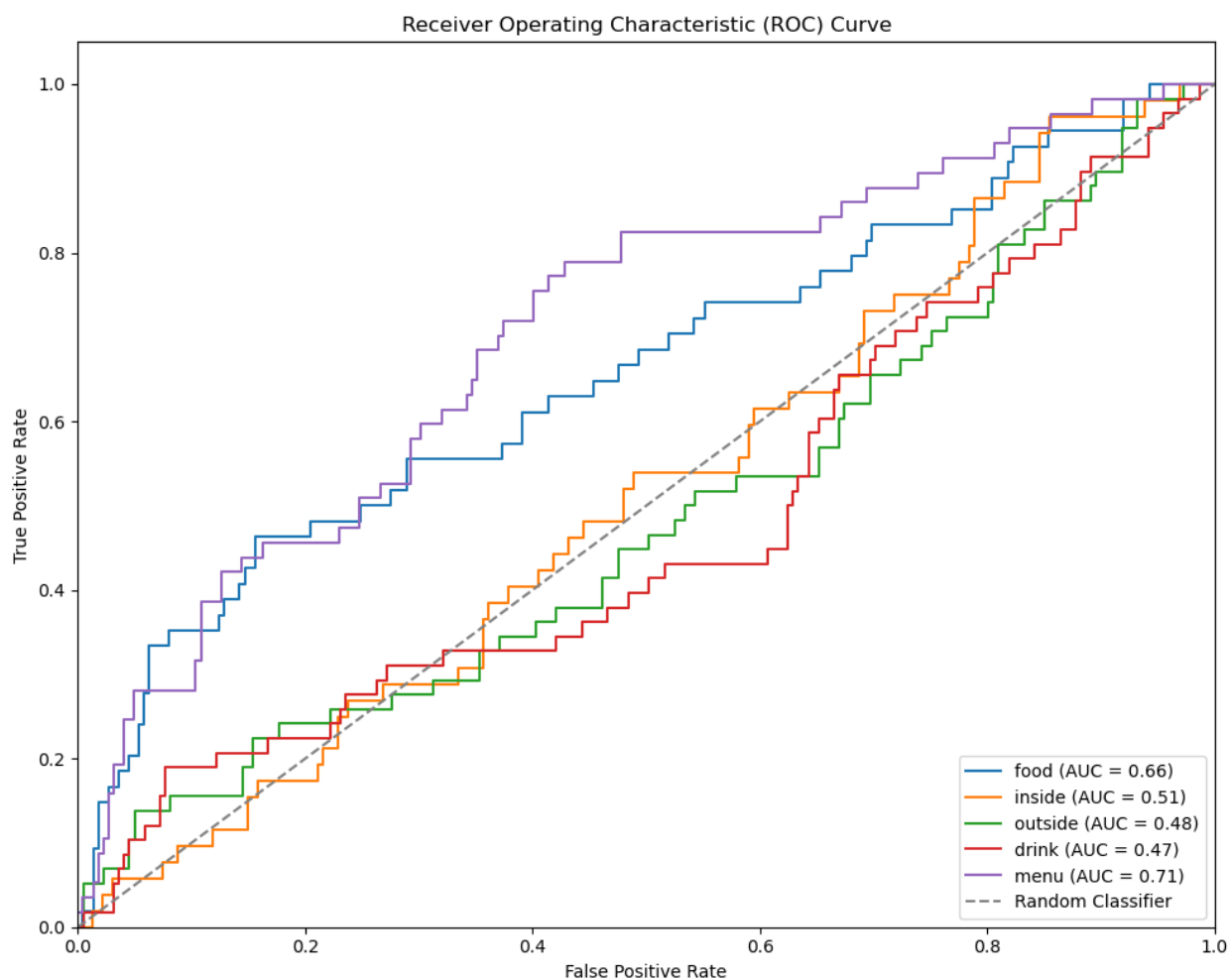
Confusion Matrix:

	food	inside	outside	drink	menu
food	41	6	2	2	3
inside	28	6	6	5	7
outside	38	5	6	3	6
drink	42	5	2	4	5
menu	24	0	7	2	24



```
# Plot ROC Curve for Test Data
print("\nROC Curve for Test Data:")
plot_roc_curve(np.argmax(y_test_onehot, axis=1), y_test_pred_probs)
```

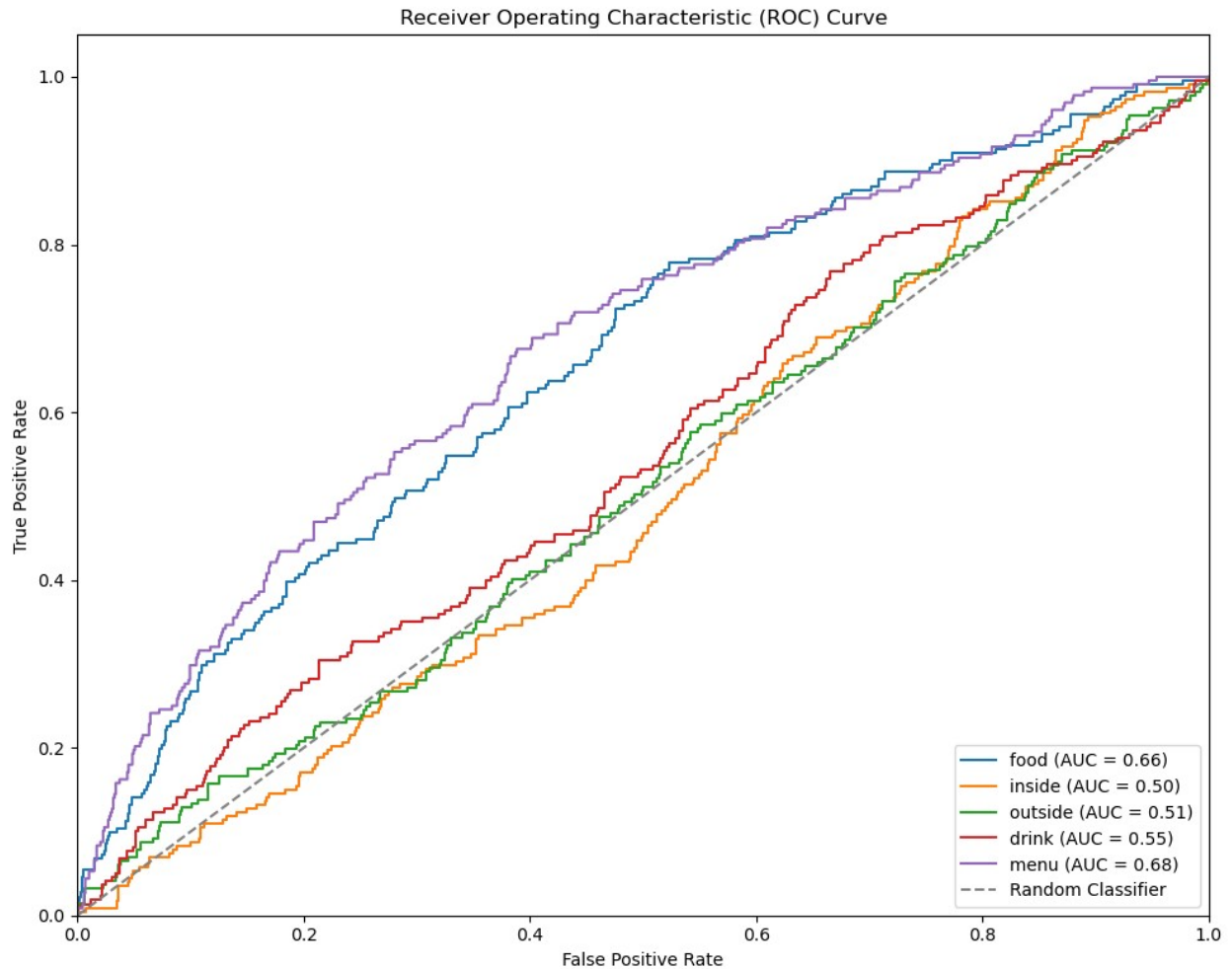
ROC Curve for Test Data:



AUC for class food: 0.66
AUC for class inside: 0.51
AUC for class outside: 0.48
AUC for class drink: 0.47
AUC for class menu: 0.71

```
# Plot ROC Curve for Training Data (Optional)
print("\nROC Curve for Training Data:")
plot_roc_curve(np.argmax(y_train_onehot, axis=1), y_train_pred_probs)
```

ROC Curve for Training Data:



```
AUC for class food: 0.66
AUC for class inside: 0.50
AUC for class outside: 0.51
AUC for class drink: 0.55
AUC for class menu: 0.68
```

lime

```
explain_with_lime(model, x_test[0], model_type="efficientnet")
{"model_id": "25f6485615cc40e7ac3a88825d944a2d", "version_major": 2, "version_minor": 0}
```

```
1/1 _____ 0s 217ms/step
1/1 _____ 0s 239ms/step
1/1 _____ 0s 246ms/step
1/1 _____ 0s 239ms/step
1/1 _____ 0s 208ms/step
1/1 _____ 0s 225ms/step
```

1/1	_____	0s 194ms/step
1/1	_____	0s 241ms/step
1/1	_____	0s 192ms/step
1/1	_____	0s 256ms/step
1/1	_____	0s 193ms/step
1/1	_____	0s 206ms/step
1/1	_____	0s 208ms/step
1/1	_____	0s 279ms/step
1/1	_____	0s 239ms/step
1/1	_____	0s 190ms/step
1/1	_____	0s 238ms/step
1/1	_____	0s 223ms/step
1/1	_____	0s 207ms/step
1/1	_____	0s 191ms/step
1/1	_____	0s 208ms/step
1/1	_____	0s 223ms/step
1/1	_____	0s 215ms/step
1/1	_____	0s 215ms/step
1/1	_____	0s 192ms/step
1/1	_____	0s 223ms/step
1/1	_____	0s 192ms/step
1/1	_____	0s 219ms/step
1/1	_____	0s 175ms/step
1/1	_____	0s 211ms/step
1/1	_____	0s 215ms/step
1/1	_____	0s 207ms/step
1/1	_____	0s 190ms/step
1/1	_____	0s 207ms/step
1/1	_____	0s 253ms/step
1/1	_____	0s 232ms/step
1/1	_____	0s 246ms/step
1/1	_____	0s 235ms/step
1/1	_____	0s 219ms/step
1/1	_____	0s 231ms/step
1/1	_____	0s 177ms/step
1/1	_____	0s 192ms/step
1/1	_____	0s 200ms/step
1/1	_____	0s 191ms/step
1/1	_____	0s 211ms/step
1/1	_____	0s 191ms/step
1/1	_____	0s 209ms/step
1/1	_____	0s 274ms/step
1/1	_____	0s 238ms/step
1/1	_____	0s 200ms/step
1/1	_____	0s 209ms/step
1/1	_____	0s 191ms/step
1/1	_____	0s 183ms/step
1/1	_____	0s 200ms/step
1/1	_____	0s 175ms/step

1/1	_____	0s	208ms/step
1/1	_____	0s	223ms/step
1/1	_____	0s	224ms/step
1/1	_____	0s	256ms/step
1/1	_____	0s	239ms/step
1/1	_____	0s	256ms/step
1/1	_____	0s	255ms/step
1/1	_____	0s	201ms/step
1/1	_____	0s	200ms/step
1/1	_____	0s	192ms/step
1/1	_____	0s	176ms/step
1/1	_____	0s	194ms/step
1/1	_____	0s	209ms/step
1/1	_____	0s	177ms/step
1/1	_____	0s	193ms/step
1/1	_____	0s	176ms/step
1/1	_____	0s	201ms/step
1/1	_____	0s	192ms/step
1/1	_____	0s	208ms/step
1/1	_____	0s	206ms/step
1/1	_____	0s	177ms/step
1/1	_____	0s	184ms/step
1/1	_____	0s	208ms/step
1/1	_____	0s	177ms/step
1/1	_____	0s	261ms/step
1/1	_____	0s	192ms/step
1/1	_____	0s	193ms/step
1/1	_____	0s	191ms/step
1/1	_____	0s	209ms/step
1/1	_____	0s	192ms/step
1/1	_____	0s	224ms/step
1/1	_____	0s	180ms/step
1/1	_____	0s	200ms/step
1/1	_____	0s	201ms/step
1/1	_____	0s	237ms/step
1/1	_____	0s	207ms/step
1/1	_____	0s	278ms/step
1/1	_____	0s	247ms/step
1/1	_____	0s	245ms/step
1/1	_____	0s	264ms/step
1/1	_____	0s	192ms/step
1/1	_____	0s	192ms/step
1/1	_____	0s	192ms/step
1/1	_____	0s	199ms/step
1/1	_____	0s	181ms/step

LIME Explanation for EFFICIENTNET - Class 0



efficientnet with all freeze layers and Image augmentation

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
```

```
# Load EfficientNetB0 without top layers
```



```

base_model = EfficientNetB0(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = False # Keep it frozen for now

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator with augmentation for the training data
train_datagen = ImageDataGenerator(
    rotation_range=40, # Random rotation between 0 and 40 degrees
    width_shift_range=0.2, # Horizontal shift
    height_shift_range=0.2, # Vertical shift
    shear_range=0.2, # Shearing transformations
    zoom_range=0.2, # Random zoom
    horizontal_flip=True, # Random horizontal flip
    fill_mode='nearest' # Filling mode for newly created pixels
)

# No augmentation for the test data, just use it as it is
test_datagen = ImageDataGenerator()

# Apply the augmentation to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Fine-tune the model with augmented images
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

```

```

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

print("confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels )
print("confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

print("ROC for train data")
plot_roc_curve(y_train, y_train_pred )
print("ROC for test data")
plot_roc_curve(y_test, y_test_pred )

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
  self._warn_if_super_not_called()

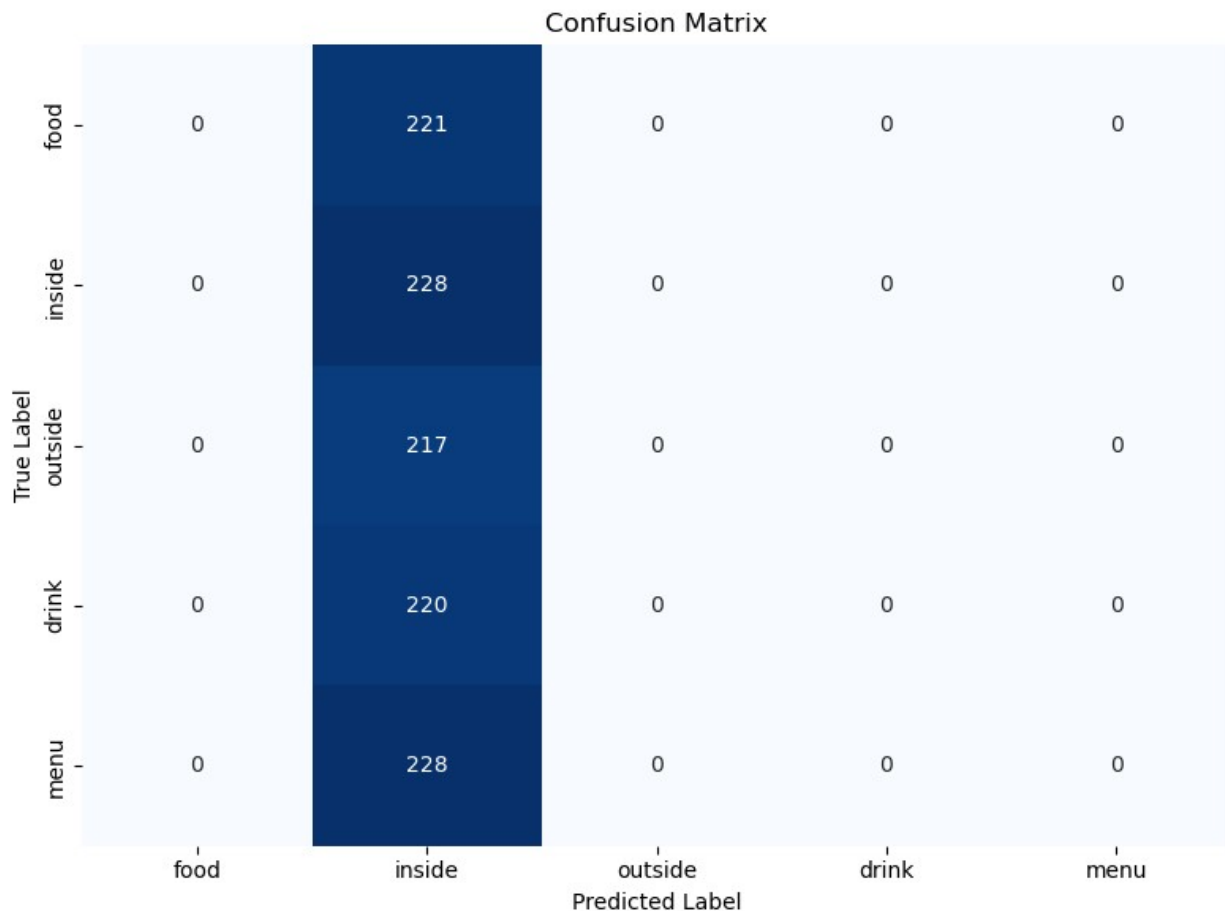
Epoch 1/10
35/35 _____ 35s 713ms/step - accuracy: 0.2214 - loss:
1.6334 - val_accuracy: 0.1935 - val_loss: 1.6139
Epoch 2/10
35/35 _____ 24s 700ms/step - accuracy: 0.1770 - loss:
1.6413 - val_accuracy: 0.2043 - val_loss: 1.6115
Epoch 3/10
35/35 _____ 26s 757ms/step - accuracy: 0.2045 - loss:
1.6232 - val_accuracy: 0.2043 - val_loss: 1.6122
Epoch 4/10
35/35 _____ 22s 641ms/step - accuracy: 0.1817 - loss:
1.6241 - val_accuracy: 0.1864 - val_loss: 1.6123
Epoch 5/10
35/35 _____ 23s 649ms/step - accuracy: 0.2166 - loss:
1.6148 - val_accuracy: 0.2043 - val_loss: 1.6117
Epoch 6/10
35/35 _____ 23s 658ms/step - accuracy: 0.2273 - loss:
1.6090 - val_accuracy: 0.2545 - val_loss: 1.6110
Epoch 7/10
35/35 _____ 26s 745ms/step - accuracy: 0.2123 - loss:
1.6144 - val_accuracy: 0.2079 - val_loss: 1.6113
Epoch 8/10
35/35 _____ 27s 760ms/step - accuracy: 0.1789 - loss:
1.6166 - val_accuracy: 0.1864 - val_loss: 1.6110

```

```

Epoch 9/10
35/35 _____ 26s 749ms/step - accuracy: 0.1968 - loss:
1.6165 - val_accuracy: 0.1864 - val_loss: 1.6111
Epoch 10/10
35/35 _____ 26s 731ms/step - accuracy: 0.1992 - loss:
1.6111 - val_accuracy: 0.1864 - val_loss: 1.6106
Training Time: 259.44 seconds
35/35 _____ 18s 456ms/step
9/9 _____ 3s 341ms/step
confusion Matrix for train data
Confusion Matrix:
      food  inside  outside  drink  menu
food      0     221      0      0      0
inside    0     228      0      0      0
outside   0     217      0      0      0
drink     0     220      0      0      0
menu      0     228      0      0      0

```

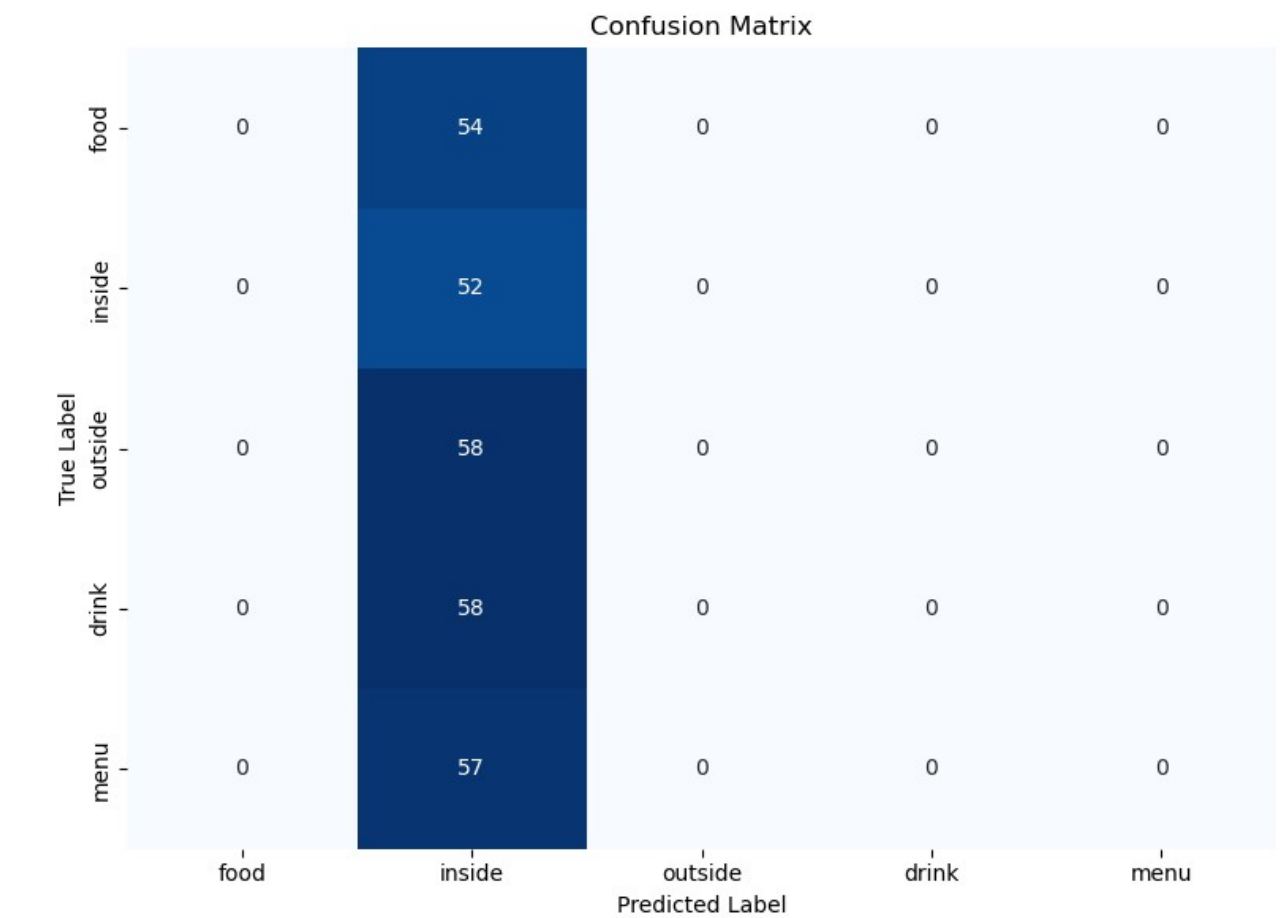


```

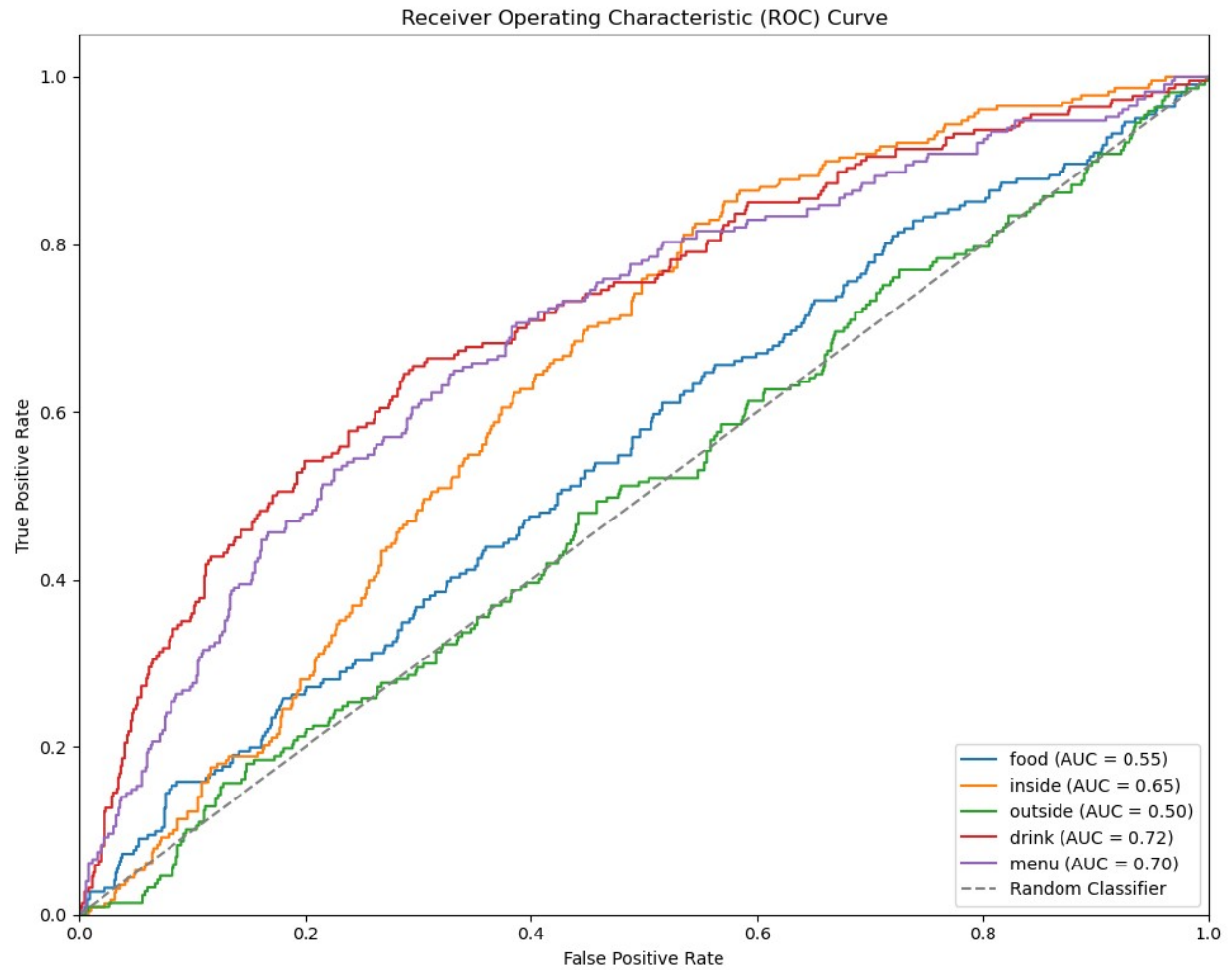
confusion Matrix for test data
Confusion Matrix:
      food  inside  outside  drink  menu

```

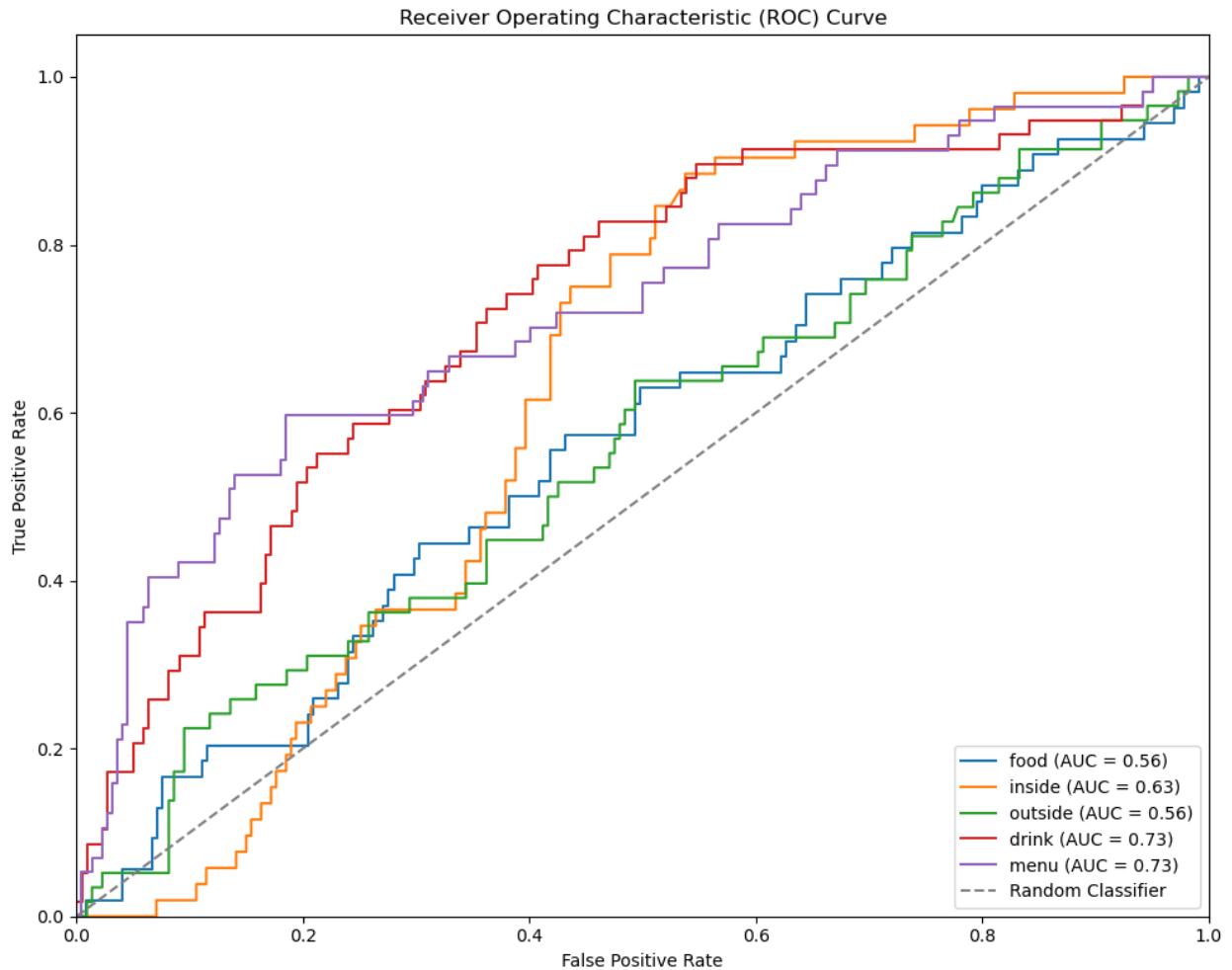
food	0	54	0	0	0
inside	0	52	0	0	0
outside	0	58	0	0	0
drink	0	58	0	0	0
menu	0	57	0	0	0



R0C for train data



```
AUC for class food: 0.55
AUC for class inside: 0.65
AUC for class outside: 0.50
AUC for class drink: 0.72
AUC for class menu: 0.70
ROC for test data
```



```
AUC for class food: 0.56
AUC for class inside: 0.63
AUC for class outside: 0.56
AUC for class drink: 0.73
AUC for class menu: 0.73
```

lime

```
explain_with_lime(model, x_test[0], model_type="efficientnet")
{"model_id": "306af5d287144f0a8cfc014eecb18709", "version_major": 2, "version_minor": 0}
```

```
1/1 _____ 0s 298ms/step
1/1 _____ 0s 291ms/step
1/1 _____ 0s 338ms/step
1/1 _____ 0s 267ms/step
1/1 _____ 0s 210ms/step
1/1 _____ 0s 237ms/step
```

1/1	_____	0s 276ms/step
1/1	_____	0s 221ms/step
1/1	_____	0s 232ms/step
1/1	_____	0s 201ms/step
1/1	_____	0s 250ms/step
1/1	_____	0s 220ms/step
1/1	_____	0s 260ms/step
1/1	_____	0s 291ms/step
1/1	_____	0s 216ms/step
1/1	_____	0s 389ms/step
1/1	_____	0s 375ms/step
1/1	_____	0s 256ms/step
1/1	_____	0s 242ms/step
1/1	_____	0s 379ms/step
1/1	_____	0s 419ms/step
1/1	_____	0s 336ms/step
1/1	_____	0s 312ms/step
1/1	_____	0s 410ms/step
1/1	_____	0s 217ms/step
1/1	_____	0s 376ms/step
1/1	_____	0s 375ms/step
1/1	_____	0s 267ms/step
1/1	_____	0s 221ms/step
1/1	_____	0s 285ms/step
1/1	_____	0s 442ms/step
1/1	_____	0s 302ms/step
1/1	_____	0s 377ms/step
1/1	_____	0s 396ms/step
1/1	_____	0s 273ms/step
1/1	_____	0s 340ms/step
1/1	_____	0s 434ms/step
1/1	_____	0s 325ms/step
1/1	_____	0s 397ms/step
1/1	_____	0s 296ms/step
1/1	_____	0s 302ms/step
1/1	_____	0s 416ms/step
1/1	_____	0s 303ms/step
1/1	_____	0s 373ms/step
1/1	_____	0s 286ms/step
1/1	_____	0s 448ms/step
1/1	_____	0s 293ms/step
1/1	_____	0s 408ms/step
1/1	_____	0s 326ms/step
1/1	_____	0s 385ms/step
1/1	_____	0s 273ms/step
1/1	_____	0s 215ms/step
1/1	_____	0s 402ms/step
1/1	_____	0s 355ms/step
1/1	_____	0s 374ms/step

1/1	_____	0s	259ms/step
1/1	_____	0s	202ms/step
1/1	_____	0s	316ms/step
1/1	_____	0s	327ms/step
1/1	_____	0s	315ms/step
1/1	_____	0s	337ms/step
1/1	_____	0s	211ms/step
1/1	_____	0s	299ms/step
1/1	_____	0s	236ms/step
1/1	_____	0s	183ms/step
1/1	_____	0s	321ms/step
1/1	_____	0s	329ms/step
1/1	_____	0s	251ms/step
1/1	_____	0s	241ms/step
1/1	_____	0s	278ms/step
1/1	_____	0s	290ms/step
1/1	_____	0s	209ms/step
1/1	_____	0s	228ms/step
1/1	_____	0s	254ms/step
1/1	_____	0s	331ms/step
1/1	_____	0s	320ms/step
1/1	_____	0s	256ms/step
1/1	_____	0s	342ms/step
1/1	_____	0s	342ms/step
1/1	_____	0s	330ms/step
1/1	_____	0s	313ms/step
1/1	_____	0s	335ms/step
1/1	_____	0s	214ms/step
1/1	_____	0s	233ms/step
1/1	_____	0s	228ms/step
1/1	_____	0s	372ms/step
1/1	_____	0s	251ms/step
1/1	_____	0s	279ms/step
1/1	_____	0s	433ms/step
1/1	_____	0s	280ms/step
1/1	_____	0s	269ms/step
1/1	_____	0s	428ms/step
1/1	_____	0s	292ms/step
1/1	_____	0s	287ms/step
1/1	_____	0s	285ms/step
1/1	_____	0s	252ms/step
1/1	_____	0s	279ms/step
1/1	_____	0s	307ms/step
1/1	_____	0s	224ms/step
1/1	_____	0s	258ms/step

LIME Explanation for EFFICIENTNET - Class 1



efficientnet with unfreeze some layers

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
```

```
# Load EfficientNetB0 without top layers
```

```

base_model = EfficientNetB0(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = True # Unfreeze the base model

# Unfreeze some layers of the base model
# Unfreeze the last few layers for fine-tuning
for layer in base_model.layers[:-20]: # Unfreeze the last 20 layers,
keep the earlier ones frozen
    layer.trainable = False

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=True) # Training=True to fine-
tune
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator with augmentation for the training
data
train_datagen = ImageDataGenerator(
    rotation_range=40, # Random rotation between 0 and 40 degrees
    width_shift_range=0.2, # Horizontal shift
    height_shift_range=0.2, # Vertical shift
    shear_range=0.2, # Shearing transformations
    zoom_range=0.2, # Random zoom
    horizontal_flip=True, # Random horizontal flip
    fill_mode='nearest' # Filling mode for newly created pixels
)

# No augmentation for the test data, just use it as it is
test_datagen = ImageDataGenerator()

# Apply the augmentation to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Fine-tune the model with augmented images
model.fit(train_generator, epochs=10,

```

```

validation_data=validation_generator)
end_time = time.time()

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)

print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)

print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
  self._warn_if_super_not_called()

Epoch 1/10
35/35 ————— 41s 826ms/step - accuracy: 0.1939 - loss:
1.6361 - val_accuracy: 0.2043 - val_loss: 1.6105
Epoch 2/10
35/35 ————— 26s 755ms/step - accuracy: 0.1817 - loss:
1.6447 - val_accuracy: 0.2043 - val_loss: 1.6096
Epoch 3/10
35/35 ————— 40s 732ms/step - accuracy: 0.2153 - loss:
1.6205 - val_accuracy: 0.2043 - val_loss: 1.6104
Epoch 4/10
35/35 ————— 25s 722ms/step - accuracy: 0.1743 - loss:
1.6360 - val_accuracy: 0.2079 - val_loss: 1.6135
Epoch 5/10

```

35/35 _____ 25s 716ms/step - accuracy: 0.2281 - loss: 1.6182 - val_accuracy: 0.1864 - val_loss: 1.6133

Epoch 6/10

35/35 _____ 39s 668ms/step - accuracy: 0.2191 - loss: 1.6170 - val_accuracy: 0.2043 - val_loss: 1.6125

Epoch 7/10

35/35 _____ 22s 615ms/step - accuracy: 0.2048 - loss: 1.6138 - val_accuracy: 0.1864 - val_loss: 1.6108

Epoch 8/10

35/35 _____ 24s 698ms/step - accuracy: 0.1980 - loss: 1.6197 - val_accuracy: 0.1864 - val_loss: 1.6123

Epoch 9/10

35/35 _____ 38s 605ms/step - accuracy: 0.2020 - loss: 1.6257 - val_accuracy: 0.1864 - val_loss: 1.6116

Epoch 10/10

35/35 _____ 21s 600ms/step - accuracy: 0.2040 - loss: 1.6242 - val_accuracy: 0.1935 - val_loss: 1.6097

Training Time: 323.61 seconds

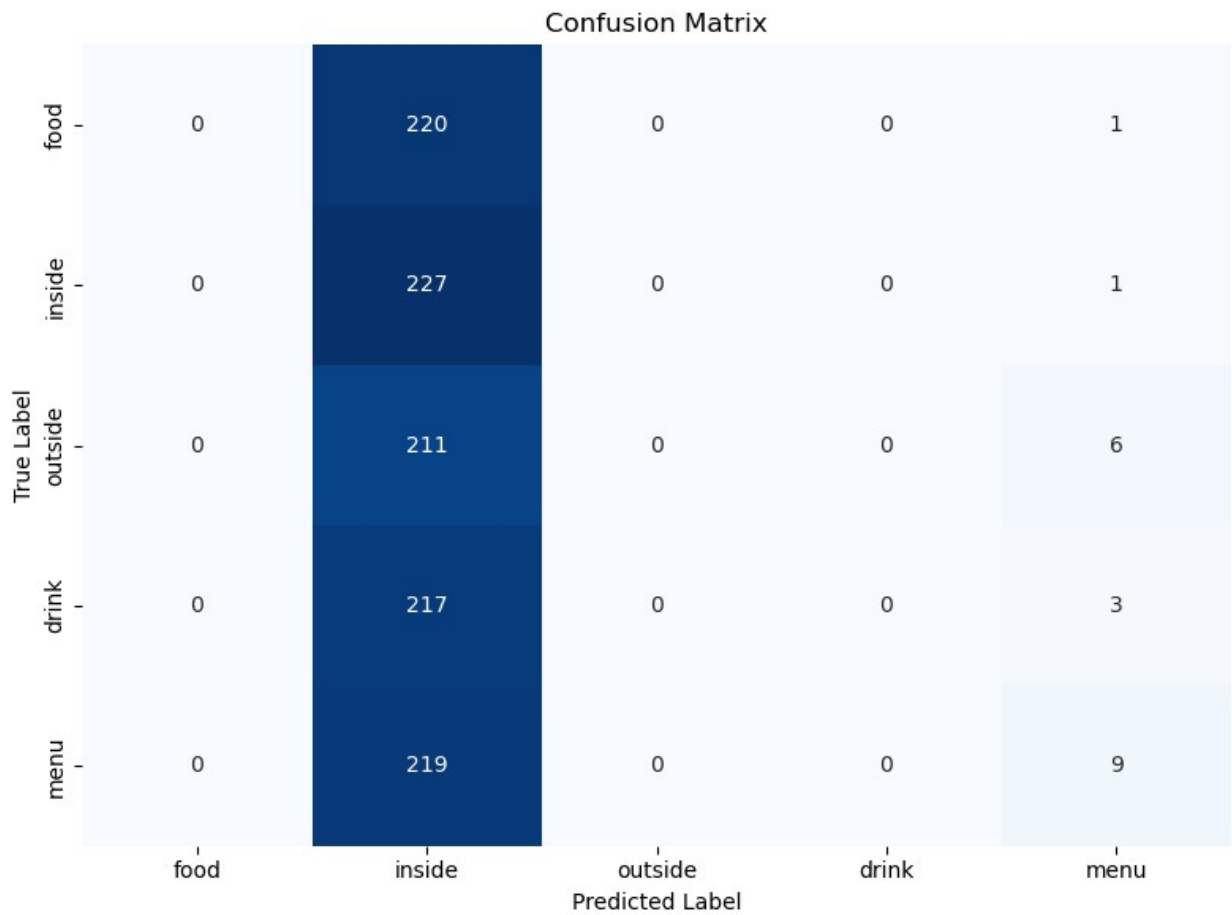
35/35 _____ 14s 339ms/step

9/9 _____ 2s 261ms/step

Confusion Matrix for train data

Confusion Matrix:

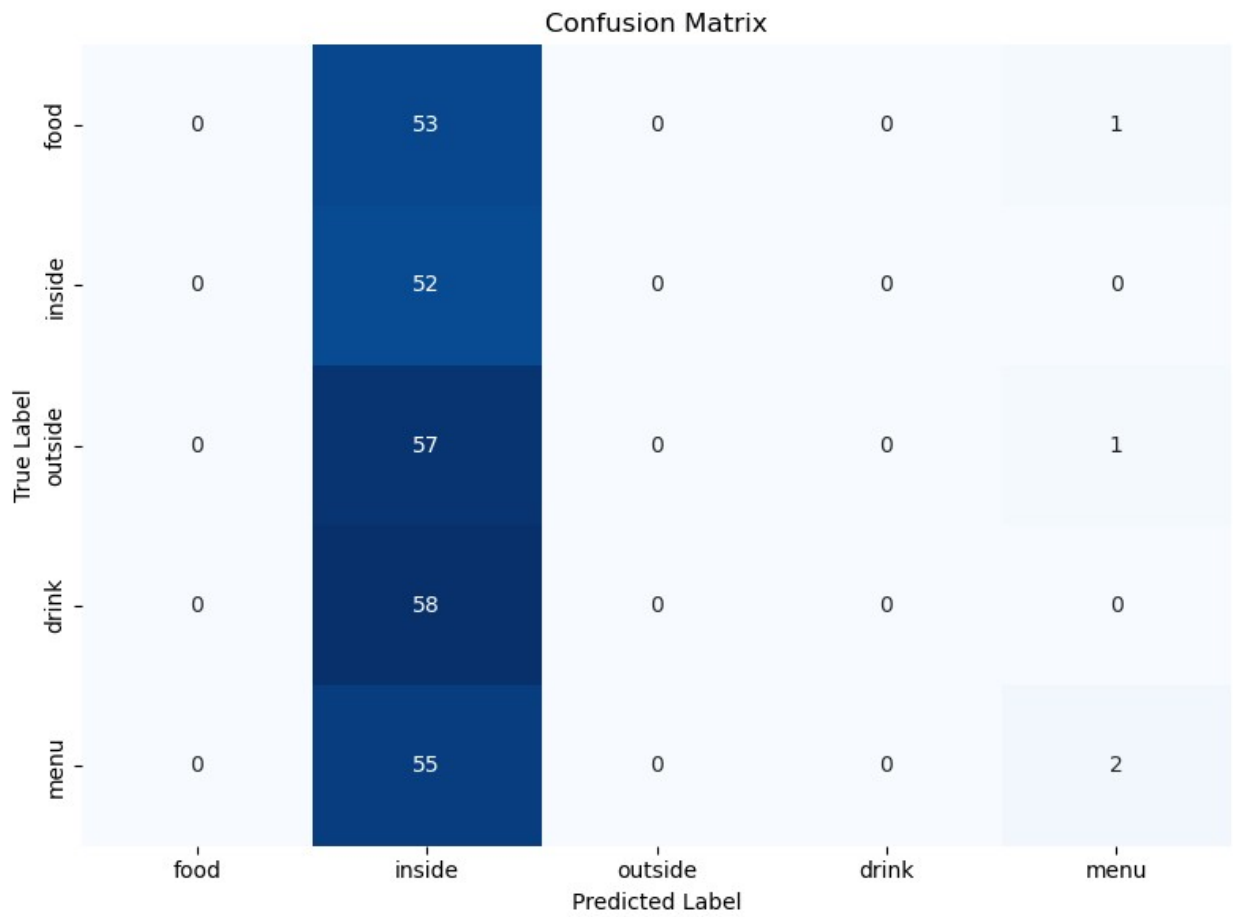
	food	inside	outside	drink	menu
food	0	220	0	0	1
inside	0	227	0	0	1
outside	0	211	0	0	6
drink	0	217	0	0	3
menu	0	219	0	0	9



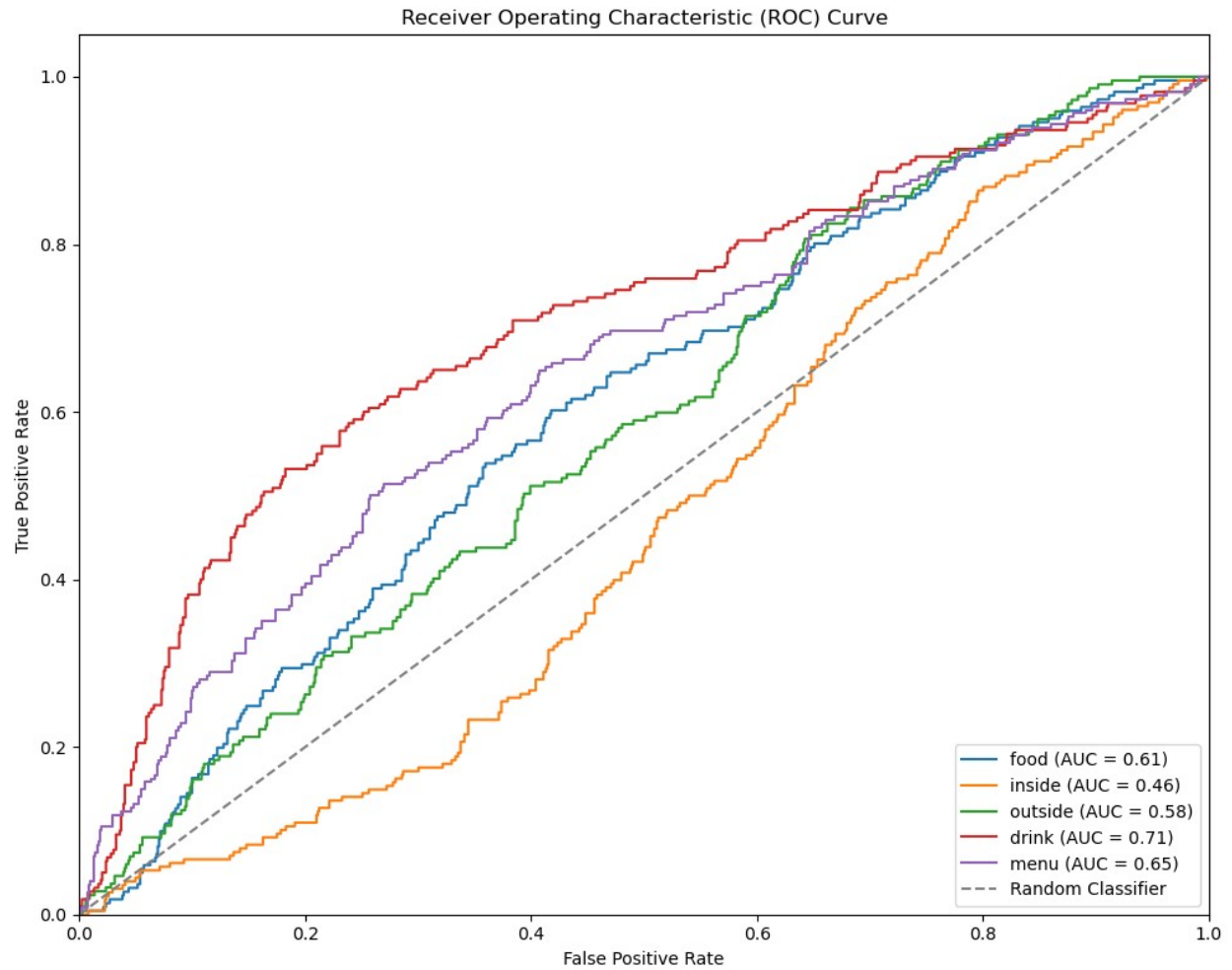
Confusion Matrix for test data

Confusion Matrix:

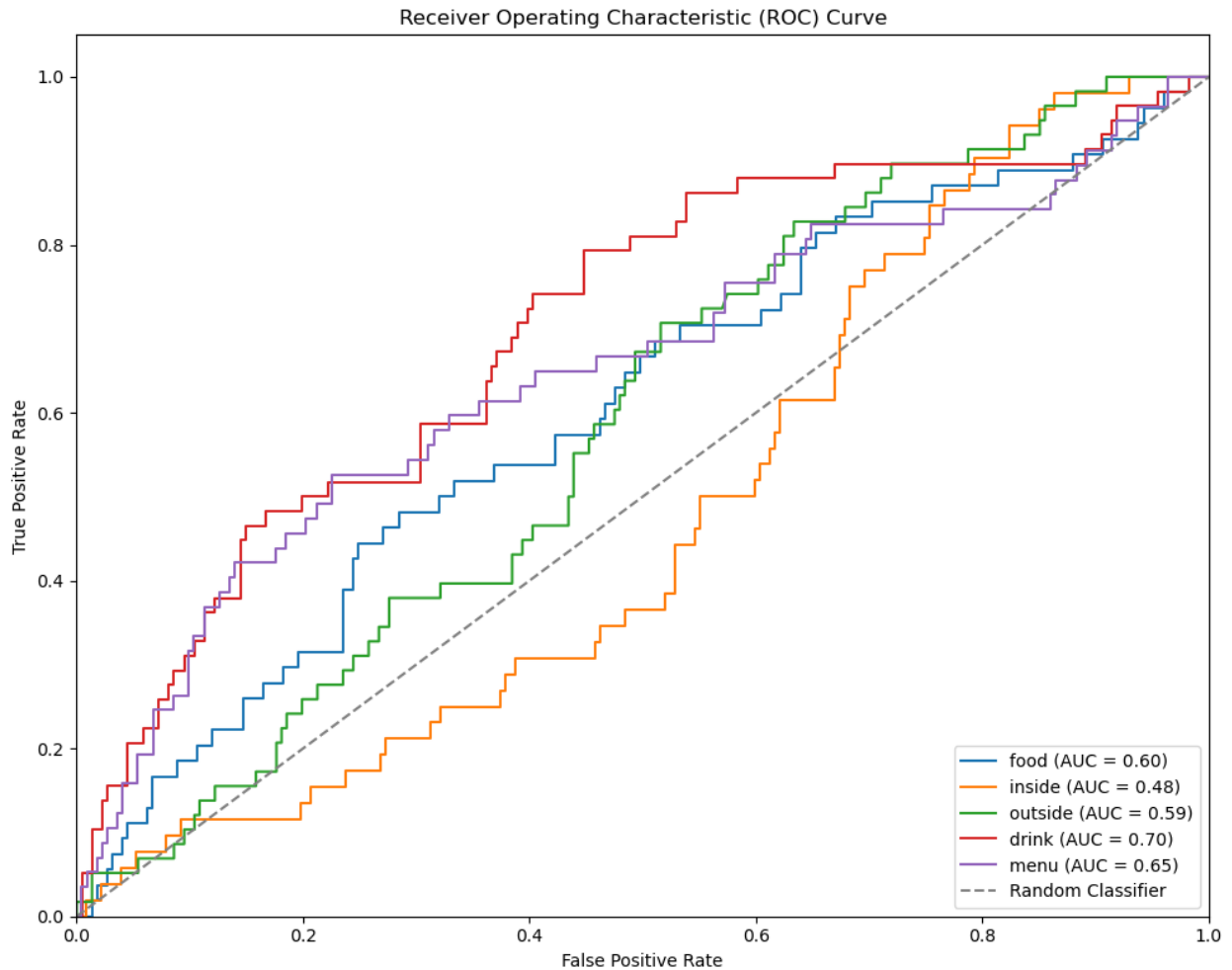
	food	inside	outside	drink	menu
food	0	53	0	0	1
inside	0	52	0	0	0
outside	0	57	0	0	1
drink	0	58	0	0	0
menu	0	55	0	0	2



ROC for train data



```
AUC for class food: 0.61
AUC for class inside: 0.46
AUC for class outside: 0.58
AUC for class drink: 0.71
AUC for class menu: 0.65
ROC for test data
```



```
AUC for class food: 0.60
AUC for class inside: 0.48
AUC for class outside: 0.59
AUC for class drink: 0.70
AUC for class menu: 0.65
```

lime

```
explain_with_lime(model, x_test[0], model_type="efficientnet")
{"model_id": "4d797bf121984f328bafale2e8bffae7", "version_major": 2, "version_minor": 0}
```

```
1/1 _____ 0s 211ms/step
1/1 _____ 0s 210ms/step
1/1 _____ 0s 350ms/step
1/1 _____ 0s 140ms/step
1/1 _____ 0s 319ms/step
1/1 _____ 0s 301ms/step
```


1/1	_____	0s 226ms/step
1/1	_____	0s 229ms/step
1/1	_____	0s 305ms/step
1/1	_____	0s 346ms/step
1/1	_____	0s 361ms/step
1/1	_____	0s 426ms/step
1/1	_____	0s 306ms/step
1/1	_____	0s 437ms/step
1/1	_____	0s 315ms/step
1/1	_____	0s 337ms/step
1/1	_____	0s 255ms/step
1/1	_____	0s 356ms/step
1/1	_____	0s 316ms/step
1/1	_____	0s 230ms/step
1/1	_____	0s 166ms/step
1/1	_____	0s 284ms/step
1/1	_____	0s 451ms/step
1/1	_____	0s 345ms/step
1/1	_____	0s 310ms/step
1/1	_____	0s 407ms/step
1/1	_____	0s 381ms/step
1/1	_____	0s 423ms/step
1/1	_____	0s 256ms/step
1/1	_____	0s 403ms/step
1/1	_____	0s 392ms/step
1/1	_____	0s 331ms/step
1/1	_____	0s 328ms/step
1/1	_____	0s 460ms/step
1/1	_____	0s 284ms/step
1/1	_____	0s 418ms/step
1/1	_____	0s 284ms/step
1/1	_____	0s 236ms/step
1/1	_____	0s 360ms/step
1/1	_____	0s 345ms/step
1/1	_____	0s 326ms/step
1/1	_____	0s 374ms/step
1/1	_____	0s 336ms/step
1/1	_____	0s 437ms/step
1/1	_____	0s 223ms/step
1/1	_____	0s 412ms/step
1/1	_____	0s 254ms/step
1/1	_____	0s 272ms/step
1/1	_____	0s 328ms/step
1/1	_____	0s 142ms/step
1/1	_____	0s 408ms/step
1/1	_____	0s 313ms/step
1/1	_____	0s 348ms/step
1/1	_____	0s 408ms/step
1/1	_____	0s 368ms/step

1/1	_____	0s	352ms/step
1/1	_____	0s	424ms/step
1/1	_____	0s	282ms/step
1/1	_____	0s	283ms/step
1/1	_____	0s	375ms/step
1/1	_____	0s	201ms/step
1/1	_____	0s	366ms/step
1/1	_____	0s	257ms/step
1/1	_____	0s	306ms/step
1/1	_____	0s	266ms/step
1/1	_____	0s	284ms/step
1/1	_____	0s	403ms/step
1/1	_____	0s	440ms/step
1/1	_____	0s	221ms/step
1/1	_____	0s	329ms/step
1/1	_____	0s	183ms/step
1/1	_____	0s	312ms/step
1/1	_____	0s	225ms/step
1/1	_____	0s	334ms/step
1/1	_____	0s	203ms/step
1/1	_____	0s	287ms/step
1/1	_____	0s	272ms/step
1/1	_____	0s	361ms/step
1/1	_____	0s	323ms/step
1/1	_____	0s	226ms/step
1/1	_____	0s	225ms/step
1/1	_____	0s	463ms/step
1/1	_____	0s	289ms/step
1/1	_____	0s	367ms/step
1/1	_____	0s	218ms/step
1/1	_____	0s	193ms/step
1/1	_____	0s	448ms/step
1/1	_____	0s	308ms/step
1/1	_____	0s	257ms/step
1/1	_____	0s	389ms/step
1/1	_____	0s	217ms/step
1/1	_____	0s	227ms/step
1/1	_____	0s	375ms/step
1/1	_____	0s	319ms/step
1/1	_____	0s	283ms/step
1/1	_____	0s	160ms/step
1/1	_____	0s	385ms/step
1/1	_____	0s	389ms/step
1/1	_____	0s	391ms/step
1/1	_____	0s	304ms/step

LIME Explanation for EFFICIENTNET - Class 1



efficientnet with unfreeze all layers and image augmentation

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

# Load EfficientNetB0 without top layers
```

```

base_model = EfficientNetB0(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Unfreeze all layers for fine-tuning
base_model.trainable = True

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=True) # Set training=True for
fine-tuning
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator with augmentation for the training
data
train_datagen = ImageDataGenerator(
    rotation_range=40, # Random rotation between 0 and 40 degrees
    width_shift_range=0.2, # Horizontal shift
    height_shift_range=0.2, # Vertical shift
    shear_range=0.2, # Shearing transformations
    zoom_range=0.2, # Random zoom
    horizontal_flip=True, # Random horizontal flip
    fill_mode='nearest' # Filling mode for newly created pixels
)

# No augmentation for the test data, just use it as it is
test_datagen = ImageDataGenerator()

# Apply the augmentation to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Fine-tune the model with augmented images
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()

# Calculate training time

```

```

training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)

print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)

print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)

```

```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
  self._warn_if_super_not_called()

```

```

Epoch 1/10
35/35 ━━━━━━━━━━━ 78s 1s/step - accuracy: 0.2445 - loss:
1.6299 - val_accuracy: 0.1900 - val_loss: 1.6226
Epoch 2/10
35/35 ━━━━━━━━━━━ 46s 1s/step - accuracy: 0.4852 - loss:
1.3494 - val_accuracy: 0.1864 - val_loss: 1.6233
Epoch 3/10
35/35 ━━━━━━━━━━━ 50s 1s/step - accuracy: 0.6951 - loss:
0.9927 - val_accuracy: 0.2043 - val_loss: 1.6164
Epoch 4/10
35/35 ━━━━━━━━━━━ 78s 1s/step - accuracy: 0.7674 - loss:
0.7022 - val_accuracy: 0.2330 - val_loss: 1.5975
Epoch 5/10
35/35 ━━━━━━━━━━━ 85s 1s/step - accuracy: 0.8204 - loss:
0.5408 - val_accuracy: 0.2258 - val_loss: 1.6016
Epoch 6/10
35/35 ━━━━━━━━━━━ 45s 1s/step - accuracy: 0.8690 - loss:
0.4055 - val_accuracy: 0.2258 - val_loss: 1.6121
Epoch 7/10

```

35/35 _____ 44s 1s/step - accuracy: 0.8643 - loss:
0.3984 - val_accuracy: 0.2258 - val_loss: 1.6041

Epoch 8/10

35/35 _____ 83s 1s/step - accuracy: 0.8959 - loss:
0.3339 - val_accuracy: 0.2294 - val_loss: 1.6044

Epoch 9/10

35/35 _____ 42s 1s/step - accuracy: 0.8998 - loss:
0.2998 - val_accuracy: 0.5412 - val_loss: 1.3205

Epoch 10/10

35/35 _____ 46s 1s/step - accuracy: 0.9187 - loss:
0.2419 - val_accuracy: 0.4516 - val_loss: 1.2993

Training Time: 596.47 seconds

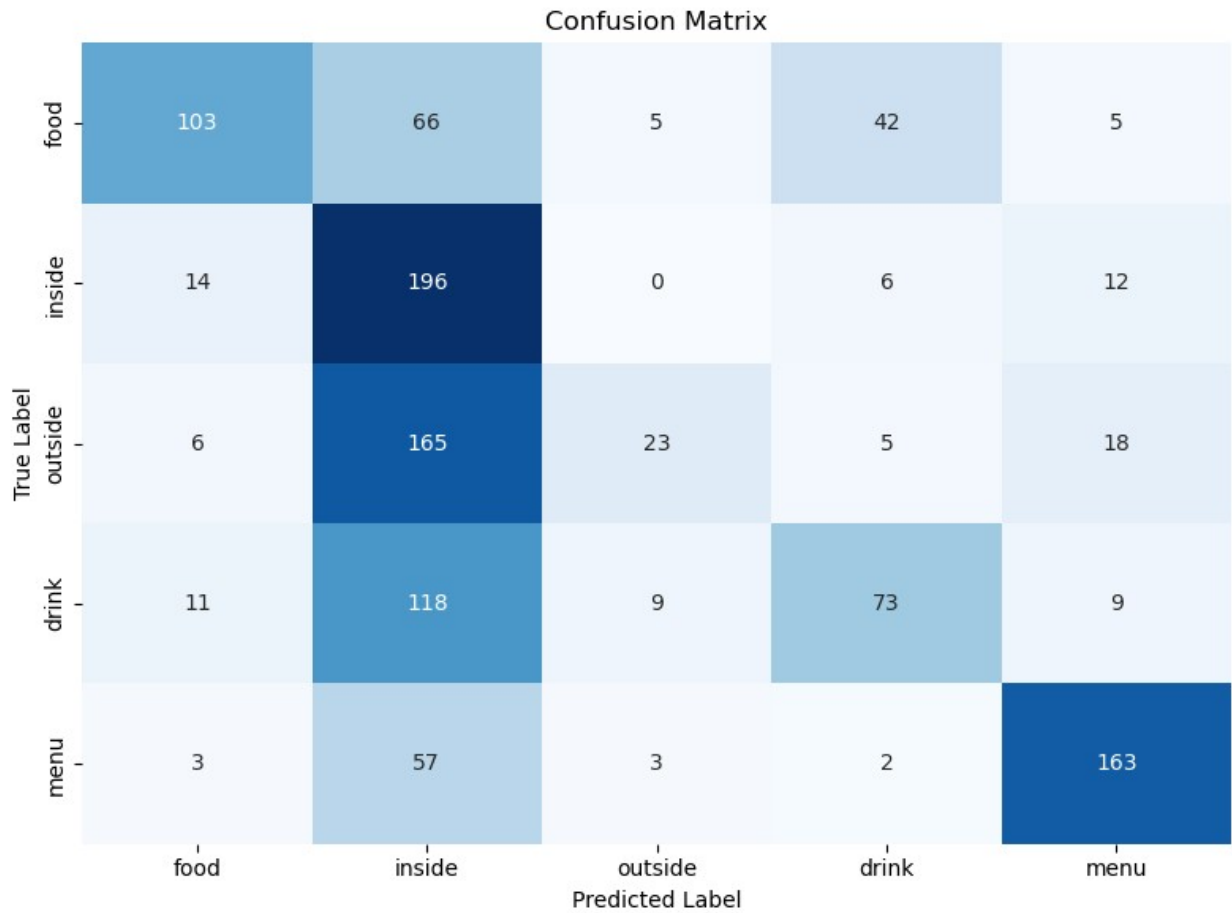
35/35 _____ 14s 339ms/step

9/9 _____ 2s 241ms/step

Confusion Matrix for train data

Confusion Matrix:

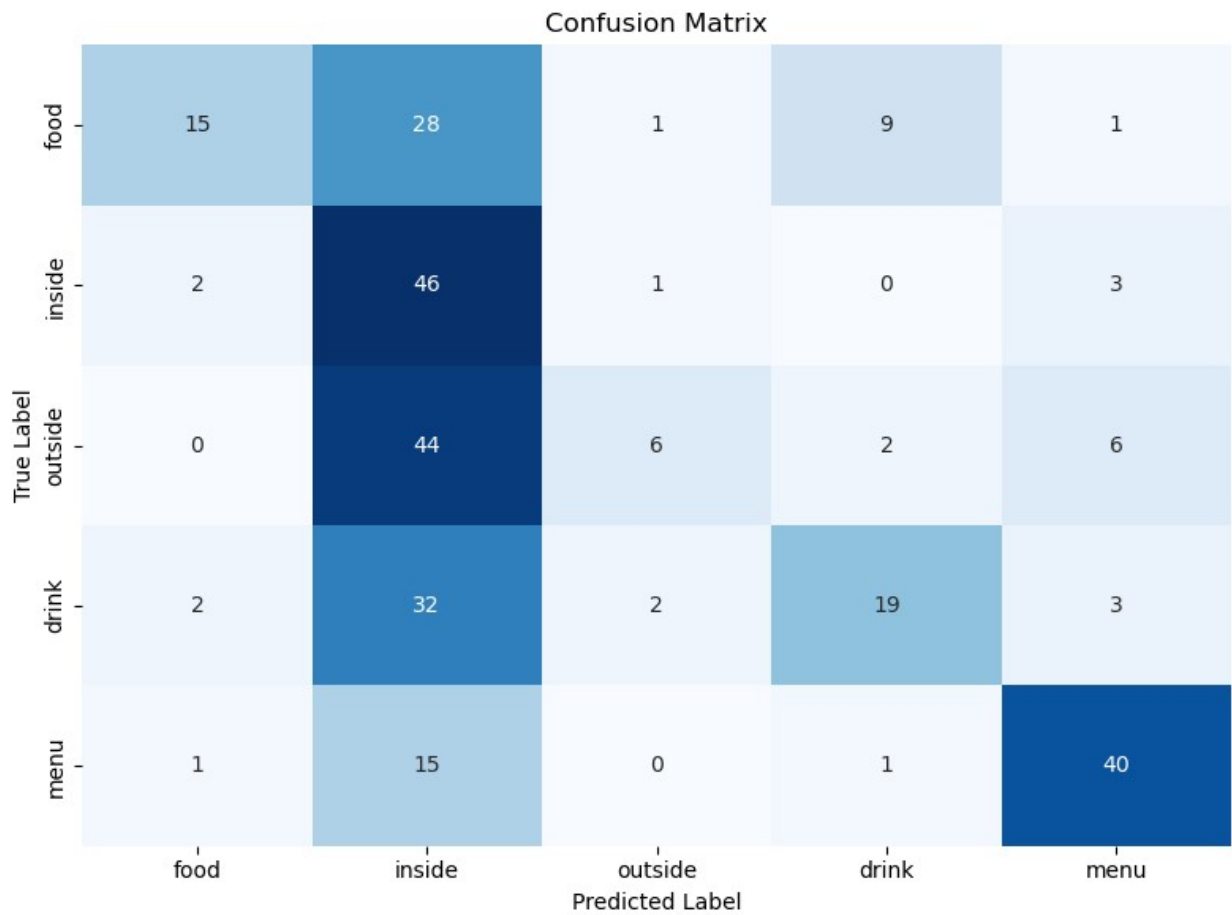
	food	inside	outside	drink	menu
food	103	66	5	42	5
inside	14	196	0	6	12
outside	6	165	23	5	18
drink	11	118	9	73	9
menu	3	57	3	2	163



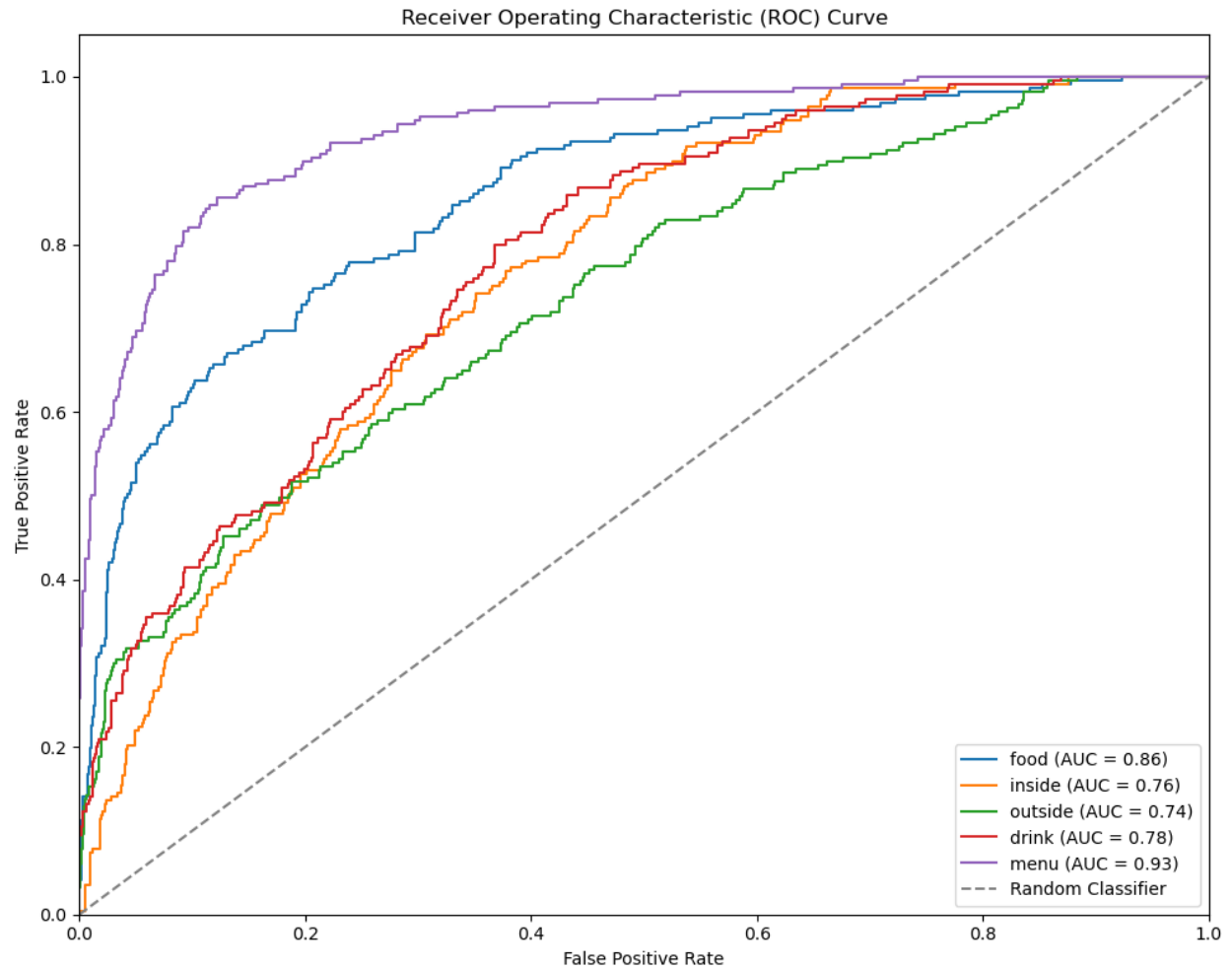
Confusion Matrix for test data

Confusion Matrix:

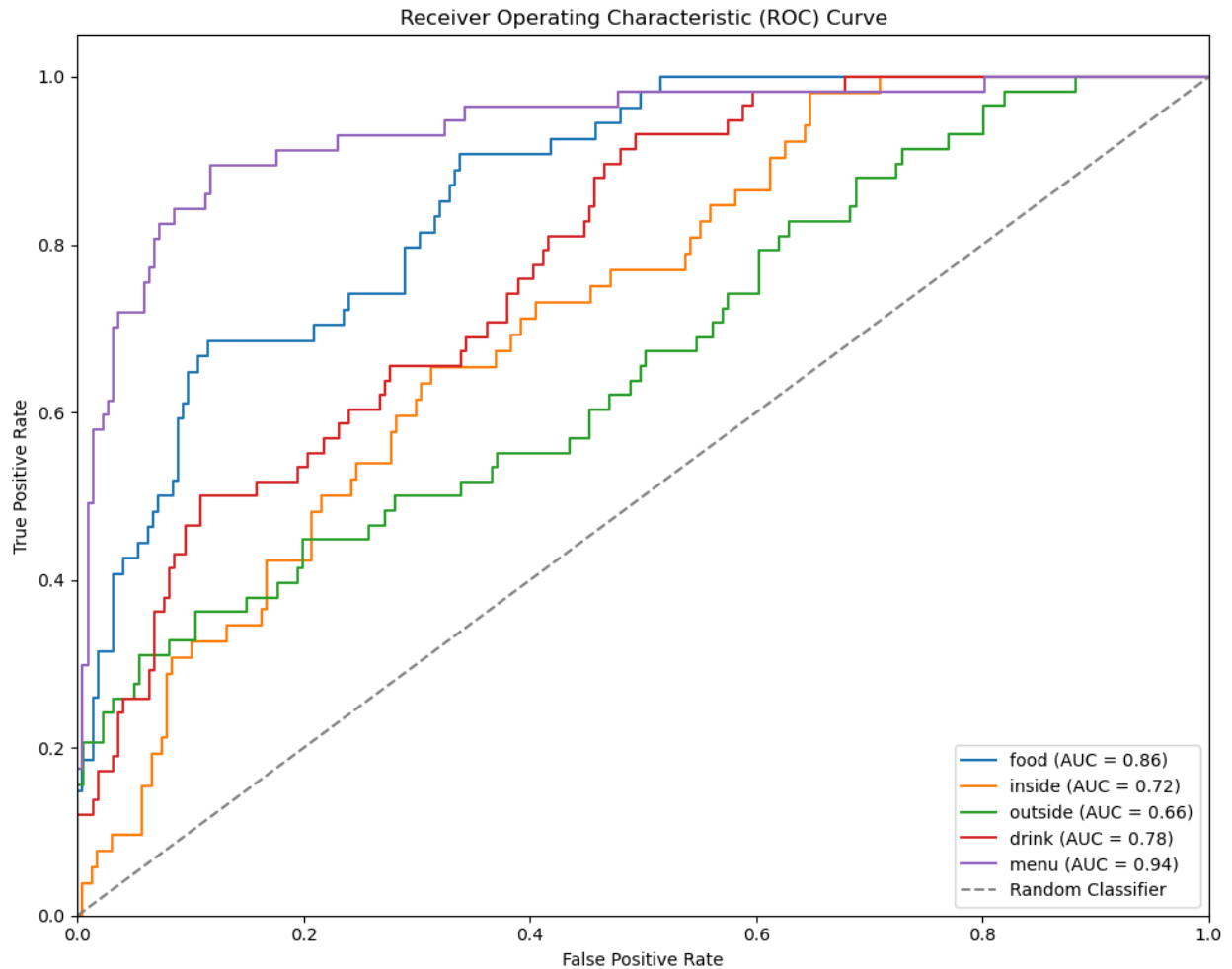
	food	inside	outside	drink	menu
food	15	28	1	9	1
inside	2	46	1	0	3
outside	0	44	6	2	6
drink	2	32	2	19	3
menu	1	15	0	1	40



R0C for train data



AUC for class food: 0.86
AUC for class inside: 0.76
AUC for class outside: 0.74
AUC for class drink: 0.78
AUC for class menu: 0.93
ROC for test data



```
AUC for class food: 0.86
AUC for class inside: 0.72
AUC for class outside: 0.66
AUC for class drink: 0.78
AUC for class menu: 0.94
```

lime

```
explain_with_lime(model, x_test[0], model_type="efficientnet")
{"model_id": "8c3057451ea24f959e33da59d0d7446e", "version_major": 2, "version_minor": 0}
```

```
1/1 _____ 0s 341ms/step
1/1 _____ 0s 279ms/step
1/1 _____ 0s 331ms/step
1/1 _____ 0s 226ms/step
1/1 _____ 0s 356ms/step
1/1 _____ 0s 394ms/step
```

1/1	_____	0s 373ms/step
1/1	_____	0s 344ms/step
1/1	_____	0s 316ms/step
1/1	_____	0s 212ms/step
1/1	_____	0s 364ms/step
1/1	_____	0s 185ms/step
1/1	_____	0s 398ms/step
1/1	_____	0s 220ms/step
1/1	_____	0s 198ms/step
1/1	_____	0s 177ms/step
1/1	_____	0s 191ms/step
1/1	_____	0s 241ms/step
1/1	_____	0s 338ms/step
1/1	_____	0s 367ms/step
1/1	_____	0s 381ms/step
1/1	_____	0s 344ms/step
1/1	_____	0s 328ms/step
1/1	_____	0s 273ms/step
1/1	_____	0s 220ms/step
1/1	_____	0s 153ms/step
1/1	_____	0s 176ms/step
1/1	_____	0s 322ms/step
1/1	_____	0s 298ms/step
1/1	_____	0s 164ms/step
1/1	_____	0s 365ms/step
1/1	_____	0s 336ms/step
1/1	_____	0s 193ms/step
1/1	_____	0s 352ms/step
1/1	_____	0s 407ms/step
1/1	_____	0s 325ms/step
1/1	_____	0s 275ms/step
1/1	_____	0s 274ms/step
1/1	_____	0s 254ms/step
1/1	_____	0s 184ms/step
1/1	_____	0s 238ms/step
1/1	_____	0s 136ms/step
1/1	_____	0s 230ms/step
1/1	_____	0s 167ms/step
1/1	_____	0s 332ms/step
1/1	_____	0s 327ms/step
1/1	_____	0s 365ms/step
1/1	_____	0s 315ms/step
1/1	_____	0s 399ms/step
1/1	_____	0s 214ms/step
1/1	_____	0s 175ms/step
1/1	_____	0s 289ms/step
1/1	_____	0s 343ms/step
1/1	_____	0s 165ms/step
1/1	_____	0s 180ms/step

1/1	_____	0s	275ms/step
1/1	_____	0s	175ms/step
1/1	_____	0s	191ms/step
1/1	_____	0s	160ms/step
1/1	_____	0s	361ms/step
1/1	_____	0s	162ms/step
1/1	_____	0s	317ms/step
1/1	_____	0s	182ms/step
1/1	_____	0s	254ms/step
1/1	_____	0s	317ms/step
1/1	_____	0s	144ms/step
1/1	_____	0s	153ms/step
1/1	_____	0s	317ms/step
1/1	_____	0s	229ms/step
1/1	_____	0s	171ms/step
1/1	_____	0s	267ms/step
1/1	_____	0s	254ms/step
1/1	_____	0s	196ms/step
1/1	_____	0s	196ms/step
1/1	_____	0s	204ms/step
1/1	_____	0s	179ms/step
1/1	_____	0s	287ms/step
1/1	_____	0s	229ms/step
1/1	_____	0s	232ms/step
1/1	_____	0s	217ms/step
1/1	_____	0s	208ms/step
1/1	_____	0s	204ms/step
1/1	_____	0s	167ms/step
1/1	_____	0s	184ms/step
1/1	_____	0s	169ms/step
1/1	_____	0s	355ms/step
1/1	_____	0s	191ms/step
1/1	_____	0s	208ms/step
1/1	_____	0s	216ms/step
1/1	_____	0s	223ms/step
1/1	_____	0s	342ms/step
1/1	_____	0s	180ms/step
1/1	_____	0s	175ms/step
1/1	_____	0s	351ms/step
1/1	_____	0s	322ms/step
1/1	_____	0s	211ms/step
1/1	_____	0s	188ms/step
1/1	_____	0s	153ms/step
1/1	_____	0s	211ms/step
1/1	_____	0s	234ms/step

LIME Explanation for EFFICIENTNET - Class 2



efficientnet with unfreeze all layers and image augmentation and gaussian blur

```
import tensorflow as tf
import cv2 # OpenCV for Gaussian Blur
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
```

```

# Function for Gaussian Blur using OpenCV
def gaussian_blur(image):
    """
    Applies Gaussian Blur to an image using OpenCV.

    Parameters:
    - image: Input image in NumPy array format.

    Returns:
    - Blurred image.
    """
    image = np.array(image) # Ensure it's a NumPy array
    blurred_image = cv2.GaussianBlur(image, (5, 5), 0) # Kernel size
    (5,5), standard deviation 0
    return blurred_image

# Combine Gaussian Blur into a preprocessing function
def preprocess_image(image):
    return gaussian_blur(image)

# Load EfficientNetB0 without top layers
base_model = EfficientNetB0(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = False # Freeze the base model

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator with augmentation for the training
data
train_datagen = ImageDataGenerator(
    rotation_range=40, # Random rotation between 0 and 40 degrees
    width_shift_range=0.2, # Horizontal shift
    height_shift_range=0.2, # Vertical shift
    shear_range=0.2, # Shearing transformations
    zoom_range=0.2, # Random zoom
    horizontal_flip=True, # Random horizontal flip

```

```

        fill_mode='nearest', # Filling mode for newly created pixels
        preprocessing_function=preprocess_image # Apply Gaussian Blur
    )

# No augmentation for the test data, just use it as it is
test_datagen = ImageDataGenerator()

# Apply the augmentation to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Fine-tune the model with augmented images
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)

print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)

print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will

```

be ignored.

```
self._warn_if_super_not_called()
```

Epoch 1/10

35/35 _____ 33s 650ms/step - accuracy: 0.1891 - loss: 1.6365 - val_accuracy: 0.2043 - val_loss: 1.6110

Epoch 2/10

35/35 _____ 22s 631ms/step - accuracy: 0.1851 - loss: 1.6343 - val_accuracy: 0.2043 - val_loss: 1.6100

Epoch 3/10

35/35 _____ 23s 658ms/step - accuracy: 0.2175 - loss: 1.6129 - val_accuracy: 0.2079 - val_loss: 1.6088

Epoch 4/10

35/35 _____ 22s 626ms/step - accuracy: 0.1856 - loss: 1.6329 - val_accuracy: 0.2079 - val_loss: 1.6085

Epoch 5/10

35/35 _____ 22s 641ms/step - accuracy: 0.1921 - loss: 1.6186 - val_accuracy: 0.2079 - val_loss: 1.6091

Epoch 6/10

35/35 _____ 22s 622ms/step - accuracy: 0.2025 - loss: 1.6220 - val_accuracy: 0.2043 - val_loss: 1.6089

Epoch 7/10

35/35 _____ 22s 637ms/step - accuracy: 0.2126 - loss: 1.6115 - val_accuracy: 0.2043 - val_loss: 1.6090

Epoch 8/10

35/35 _____ 22s 623ms/step - accuracy: 0.1817 - loss: 1.6155 - val_accuracy: 0.2043 - val_loss: 1.6101

Epoch 9/10

35/35 _____ 23s 657ms/step - accuracy: 0.2088 - loss: 1.6116 - val_accuracy: 0.2079 - val_loss: 1.6087

Epoch 10/10

35/35 _____ 22s 627ms/step - accuracy: 0.1938 - loss: 1.6152 - val_accuracy: 0.2043 - val_loss: 1.6098

Training Time: 233.79 seconds

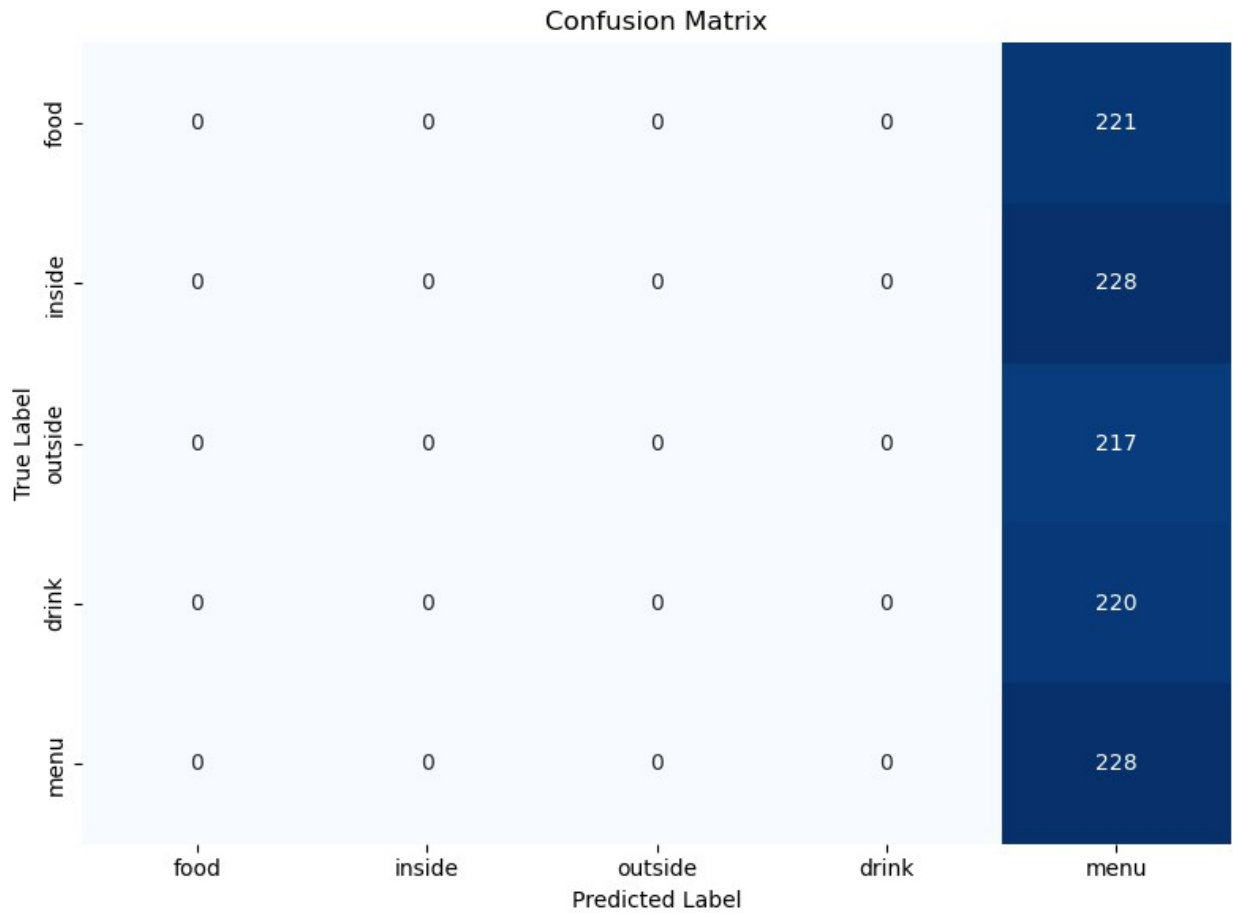
35/35 _____ 13s 308ms/step

9/9 _____ 2s 235ms/step

Confusion Matrix for train data

Confusion Matrix:

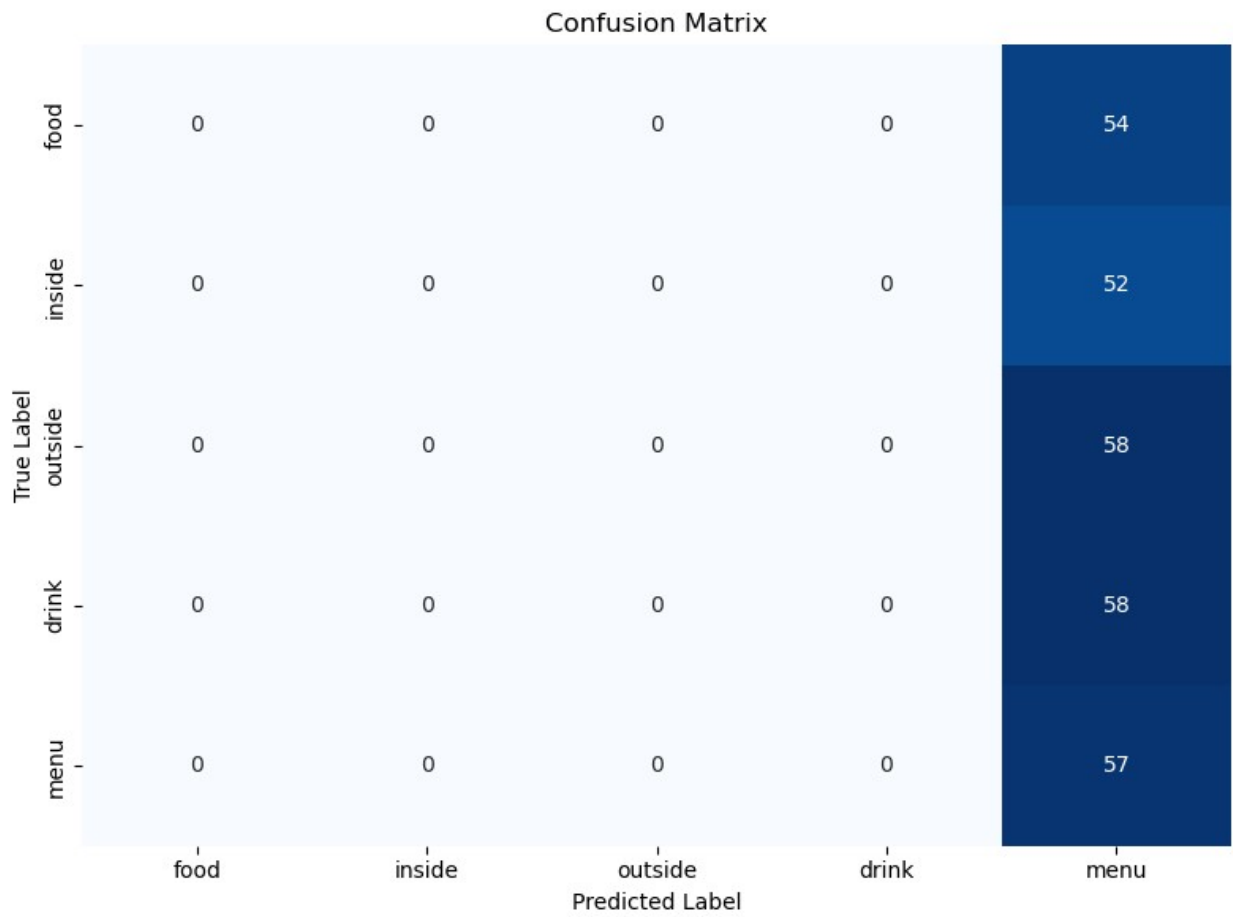
	food	inside	outside	drink	menu
food	0	0	0	0	221
inside	0	0	0	0	228
outside	0	0	0	0	217
drink	0	0	0	0	220
menu	0	0	0	0	228



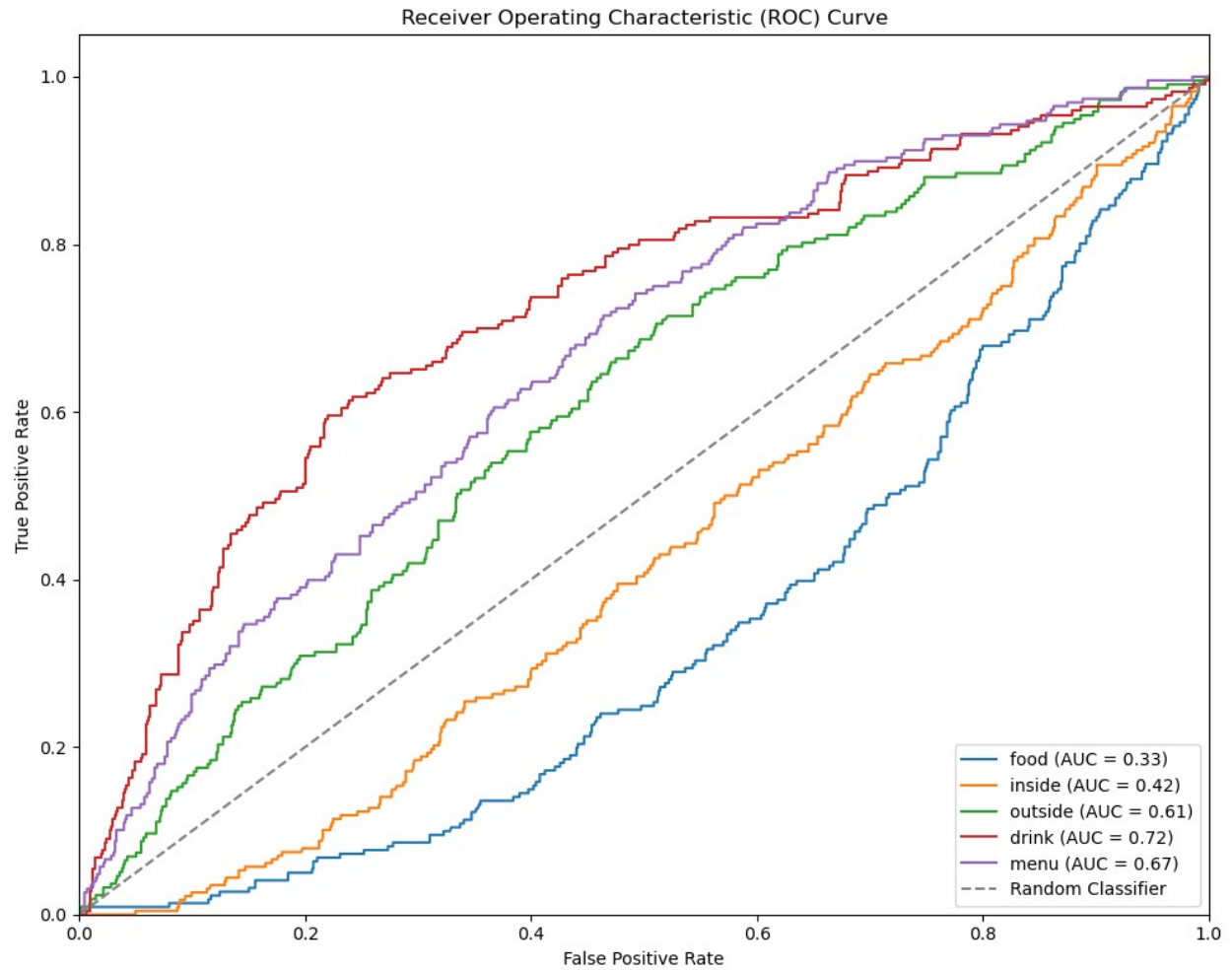
Confusion Matrix for test data

Confusion Matrix:

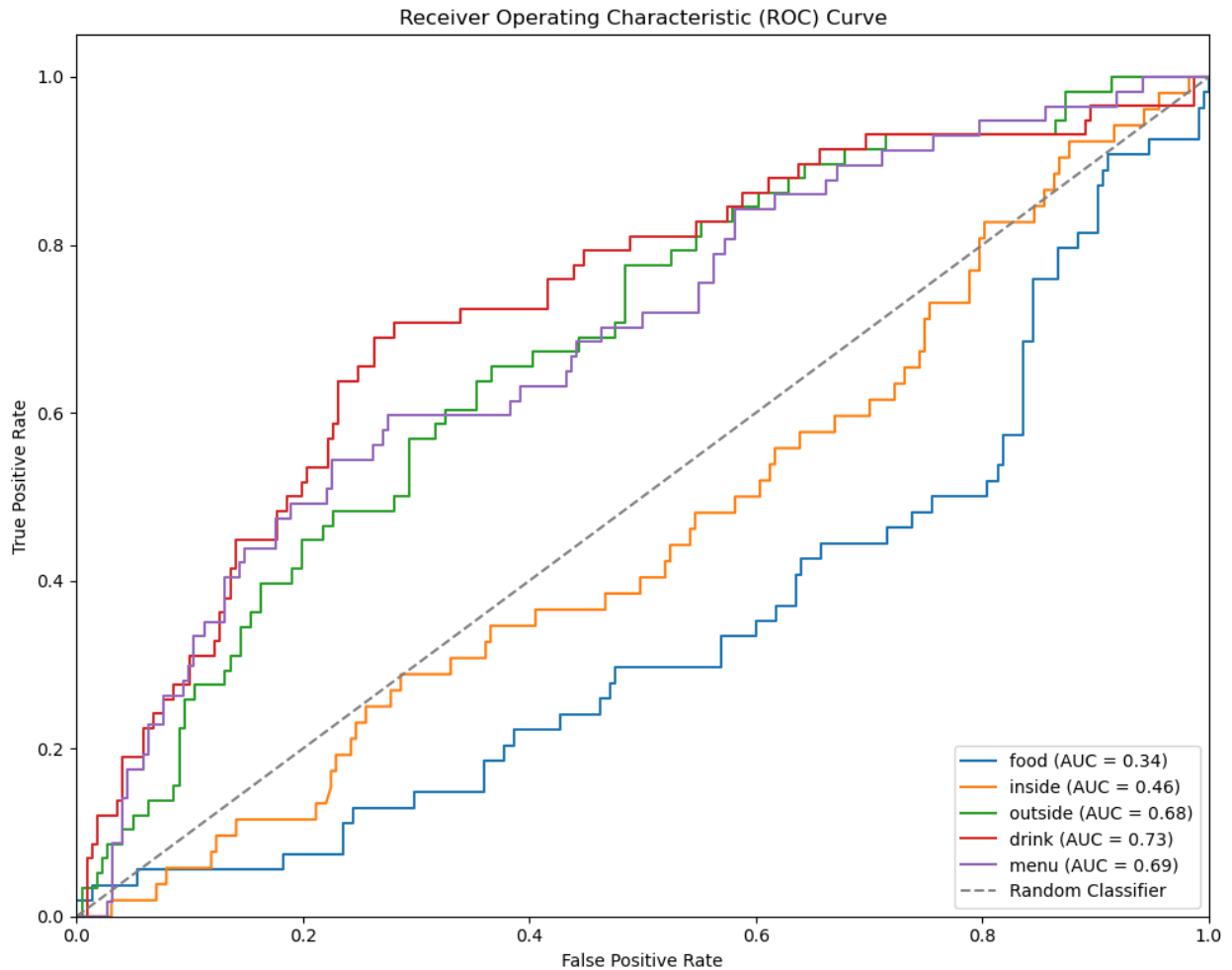
	food	inside	outside	drink	menu
food	0	0	0	0	54
inside	0	0	0	0	52
outside	0	0	0	0	58
drink	0	0	0	0	58
menu	0	0	0	0	57



R0C for train data



```
AUC for class food: 0.33
AUC for class inside: 0.42
AUC for class outside: 0.61
AUC for class drink: 0.72
AUC for class menu: 0.67
ROC for test data
```



```
AUC for class food: 0.34
AUC for class inside: 0.46
AUC for class outside: 0.68
AUC for class drink: 0.73
AUC for class menu: 0.69
```

lime

```
explain_with_lime(model, x_test[0], model_type="efficientnet")
{"model_id": "5df476899e78427bbf19eaedb76eb9e7", "version_major": 2, "version_minor": 0}
```

```
1/1 _____ 0s 396ms/step
1/1 _____ 0s 351ms/step
1/1 _____ 0s 348ms/step
1/1 _____ 0s 382ms/step
1/1 _____ 0s 358ms/step
1/1 _____ 0s 196ms/step
```

1/1	_____	0s 370ms/step
1/1	_____	0s 210ms/step
1/1	_____	0s 323ms/step
1/1	_____	0s 305ms/step
1/1	_____	0s 437ms/step
1/1	_____	0s 331ms/step
1/1	_____	0s 373ms/step
1/1	_____	0s 318ms/step
1/1	_____	0s 285ms/step
1/1	_____	0s 151ms/step
1/1	_____	0s 153ms/step
1/1	_____	0s 342ms/step
1/1	_____	0s 169ms/step
1/1	_____	0s 347ms/step
1/1	_____	0s 371ms/step
1/1	_____	0s 274ms/step
1/1	_____	0s 261ms/step
1/1	_____	0s 185ms/step
1/1	_____	0s 329ms/step
1/1	_____	0s 180ms/step
1/1	_____	0s 244ms/step
1/1	_____	0s 348ms/step
1/1	_____	0s 196ms/step
1/1	_____	0s 321ms/step
1/1	_____	0s 249ms/step
1/1	_____	0s 228ms/step
1/1	_____	0s 336ms/step
1/1	_____	0s 267ms/step
1/1	_____	0s 223ms/step
1/1	_____	0s 330ms/step
1/1	_____	0s 209ms/step
1/1	_____	0s 354ms/step
1/1	_____	0s 345ms/step
1/1	_____	0s 340ms/step
1/1	_____	0s 179ms/step
1/1	_____	0s 356ms/step
1/1	_____	0s 174ms/step
1/1	_____	0s 314ms/step
1/1	_____	0s 190ms/step
1/1	_____	0s 148ms/step
1/1	_____	0s 168ms/step
1/1	_____	0s 182ms/step
1/1	_____	0s 168ms/step
1/1	_____	0s 256ms/step
1/1	_____	0s 320ms/step
1/1	_____	0s 216ms/step
1/1	_____	0s 308ms/step
1/1	_____	0s 221ms/step
1/1	_____	0s 251ms/step

1/1	_____	0s	327ms/step
1/1	_____	0s	400ms/step
1/1	_____	0s	185ms/step
1/1	_____	0s	329ms/step
1/1	_____	0s	300ms/step
1/1	_____	0s	357ms/step
1/1	_____	0s	249ms/step
1/1	_____	0s	242ms/step
1/1	_____	0s	196ms/step
1/1	_____	0s	260ms/step
1/1	_____	0s	207ms/step
1/1	_____	0s	254ms/step
1/1	_____	0s	320ms/step
1/1	_____	0s	318ms/step
1/1	_____	0s	200ms/step
1/1	_____	0s	316ms/step
1/1	_____	0s	357ms/step
1/1	_____	0s	194ms/step
1/1	_____	0s	181ms/step
1/1	_____	0s	195ms/step
1/1	_____	0s	148ms/step
1/1	_____	0s	169ms/step
1/1	_____	0s	311ms/step
1/1	_____	0s	220ms/step
1/1	_____	0s	264ms/step
1/1	_____	0s	207ms/step
1/1	_____	0s	315ms/step
1/1	_____	0s	169ms/step
1/1	_____	0s	231ms/step
1/1	_____	0s	199ms/step
1/1	_____	0s	166ms/step
1/1	_____	0s	194ms/step
1/1	_____	0s	182ms/step
1/1	_____	0s	157ms/step
1/1	_____	0s	416ms/step
1/1	_____	0s	202ms/step
1/1	_____	0s	266ms/step
1/1	_____	0s	201ms/step
1/1	_____	0s	208ms/step
1/1	_____	0s	211ms/step
1/1	_____	0s	322ms/step
1/1	_____	0s	230ms/step
1/1	_____	0s	230ms/step
1/1	_____	0s	425ms/step
1/1	_____	0s	288ms/step

LIME Explanation for EFFICIENTNET - Class 4



Vgg

```
from tensorflow.keras.applications.vgg16 import preprocess_input as  
vgg_preprocess  
  
# Apply VGG16 preprocessing to the entire batch of images  
preprocessed_images = vgg_preprocess(x_images) # This scales and  
subtracts the mean values  
  
x_train, x_test, y_train, y_test =  
train_test_split(preprocessed_images, y, test_size=0.2,  
random_state=42)
```

```

print("Shape of x_train: ", x_train.shape) # Should print
(num_samples, 224, 224, 3)
print("Shape of y_train: ", y_train.shape)
print("Shape of x_test: ", x_test.shape)
print("Shape of y_test: ", y_test.shape)

```

```

Shape of x_train: (1114, 224, 224, 3)
Shape of y_train: (1114,)
Shape of x_test: (279, 224, 224, 3)
Shape of y_test: (279,)

```

vgg freeze all layers

```

import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

# Load VGG16 without top layers
base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Freeze all layers of VGG16 (no fine-tuning)
base_model.trainable = False

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=False) # Use training=False to
keep frozen layers stable
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator (without augmentation for training
data)
train_datagen = ImageDataGenerator()
test_datagen = ImageDataGenerator()

```



```
# Apply the generator to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)
```

```
start_time = time.time()
# Train the model (only the custom dense layers will be updated)
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()
```

```
# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")
```

```
# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)
```

```
# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)
```

```
# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)
```

```
print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)
```

```
# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)
```

```
print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)
```

Epoch 1/10

```
C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
```

```
self._warn_if_super_not_called()
```

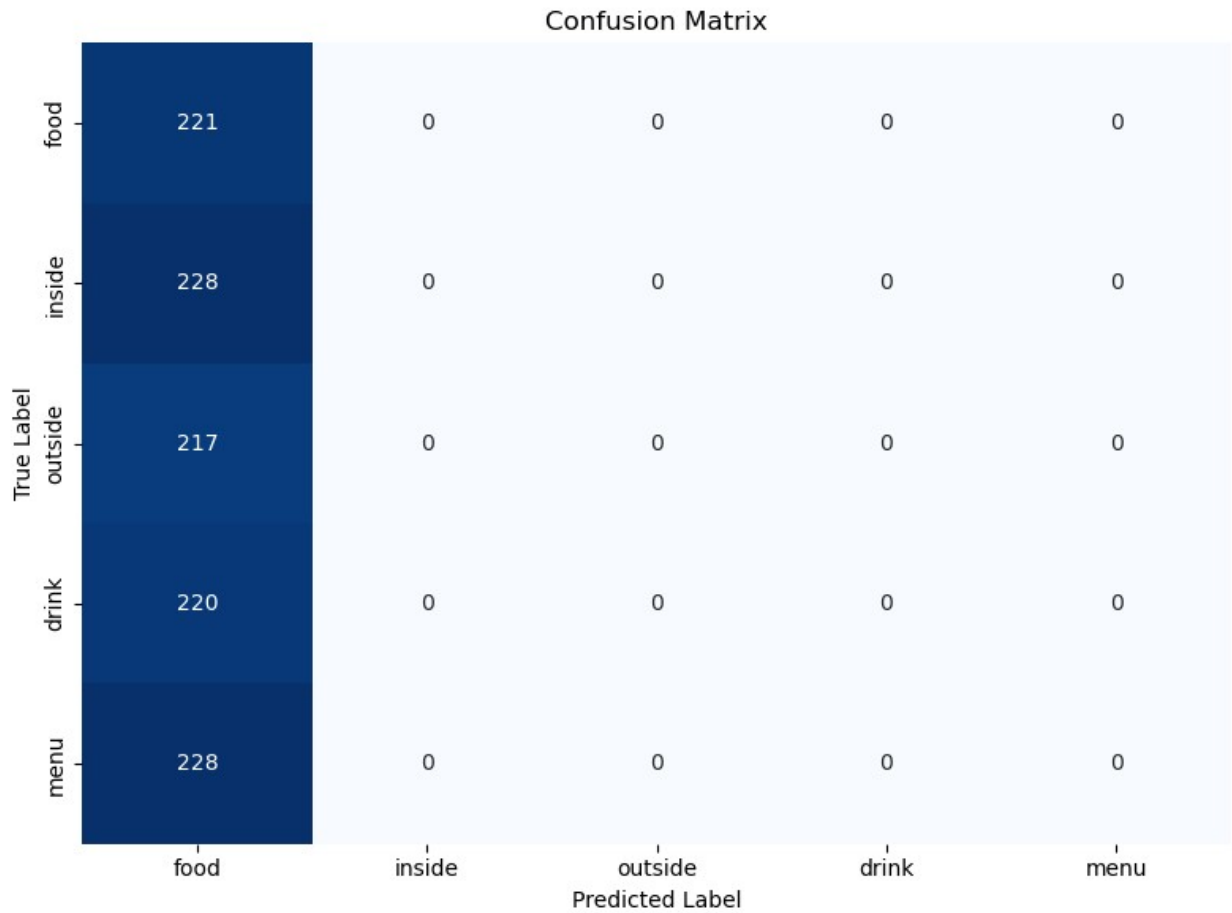
```
35/35 ————— 61s 2s/step - accuracy: 0.1818 - loss:
2.7214 - val_accuracy: 0.1935 - val_loss: 1.6200
Epoch 2/10
```

```

35/35 _____ 61s 2s/step - accuracy: 0.1839 - loss:
2.2806 - val_accuracy: 0.1935 - val_loss: 1.6428
Epoch 3/10
35/35 _____ 61s 2s/step - accuracy: 0.1880 - loss:
2.0805 - val_accuracy: 0.1935 - val_loss: 1.6377
Epoch 4/10
35/35 _____ 62s 2s/step - accuracy: 0.1831 - loss:
2.0055 - val_accuracy: 0.1935 - val_loss: 1.6403
Epoch 5/10
35/35 _____ 62s 2s/step - accuracy: 0.2038 - loss:
1.9080 - val_accuracy: 0.1935 - val_loss: 1.6316
Epoch 6/10
35/35 _____ 62s 2s/step - accuracy: 0.1949 - loss:
1.8887 - val_accuracy: 0.1935 - val_loss: 1.6321
Epoch 7/10
35/35 _____ 62s 2s/step - accuracy: 0.2020 - loss:
1.8335 - val_accuracy: 0.1935 - val_loss: 1.6345
Epoch 8/10
35/35 _____ 63s 2s/step - accuracy: 0.2257 - loss:
1.7883 - val_accuracy: 0.1935 - val_loss: 1.6314
Epoch 9/10
35/35 _____ 62s 2s/step - accuracy: 0.1896 - loss:
1.7866 - val_accuracy: 0.1935 - val_loss: 1.6262
Epoch 10/10
35/35 _____ 62s 2s/step - accuracy: 0.2162 - loss:
1.7601 - val_accuracy: 0.1935 - val_loss: 1.6192
Training Time: 616.91 seconds
35/35 _____ 49s 1s/step
9/9 _____ 12s 1s/step
Confusion Matrix for train data
Confusion Matrix:

```

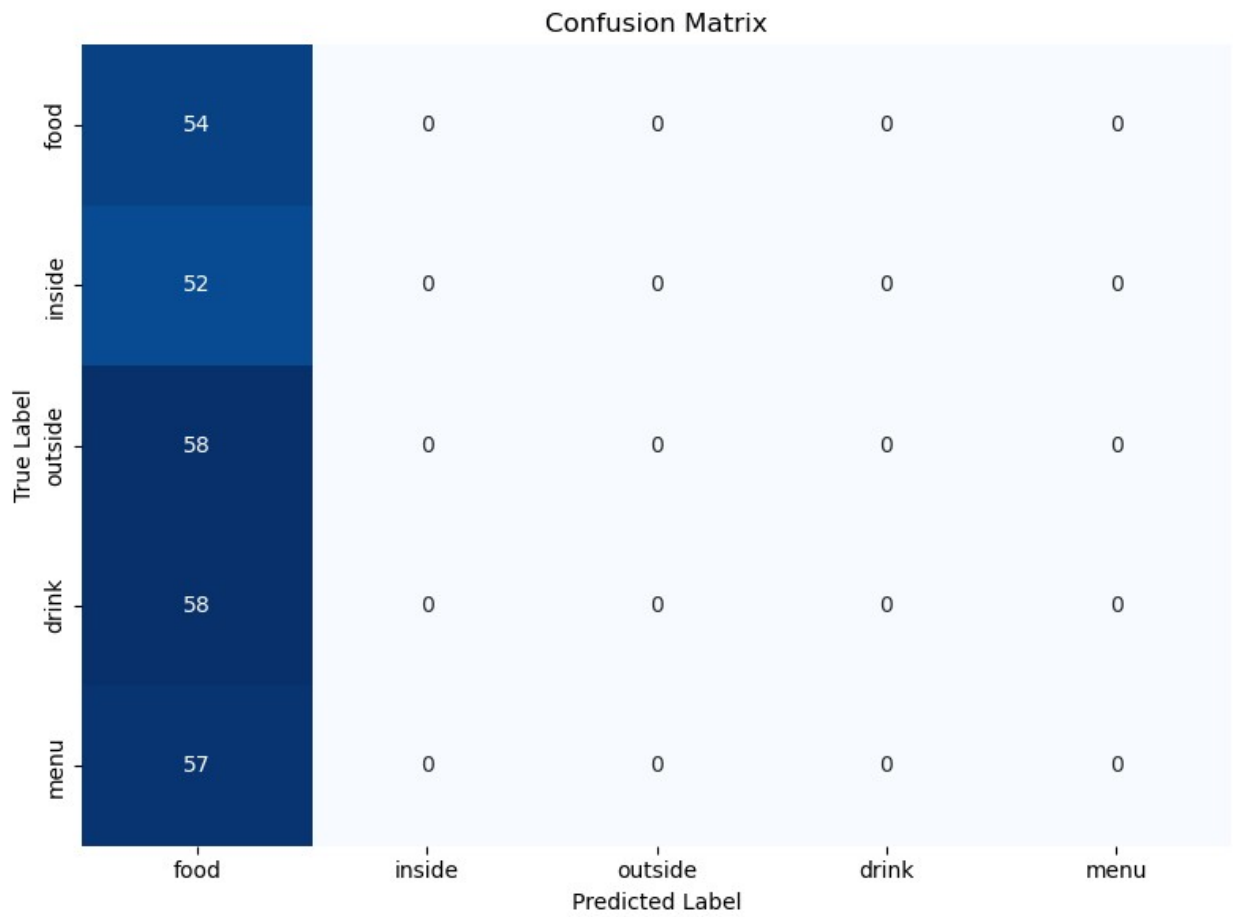
	food	inside	outside	drink	menu
food	221	0	0	0	0
inside	228	0	0	0	0
outside	217	0	0	0	0
drink	220	0	0	0	0
menu	228	0	0	0	0



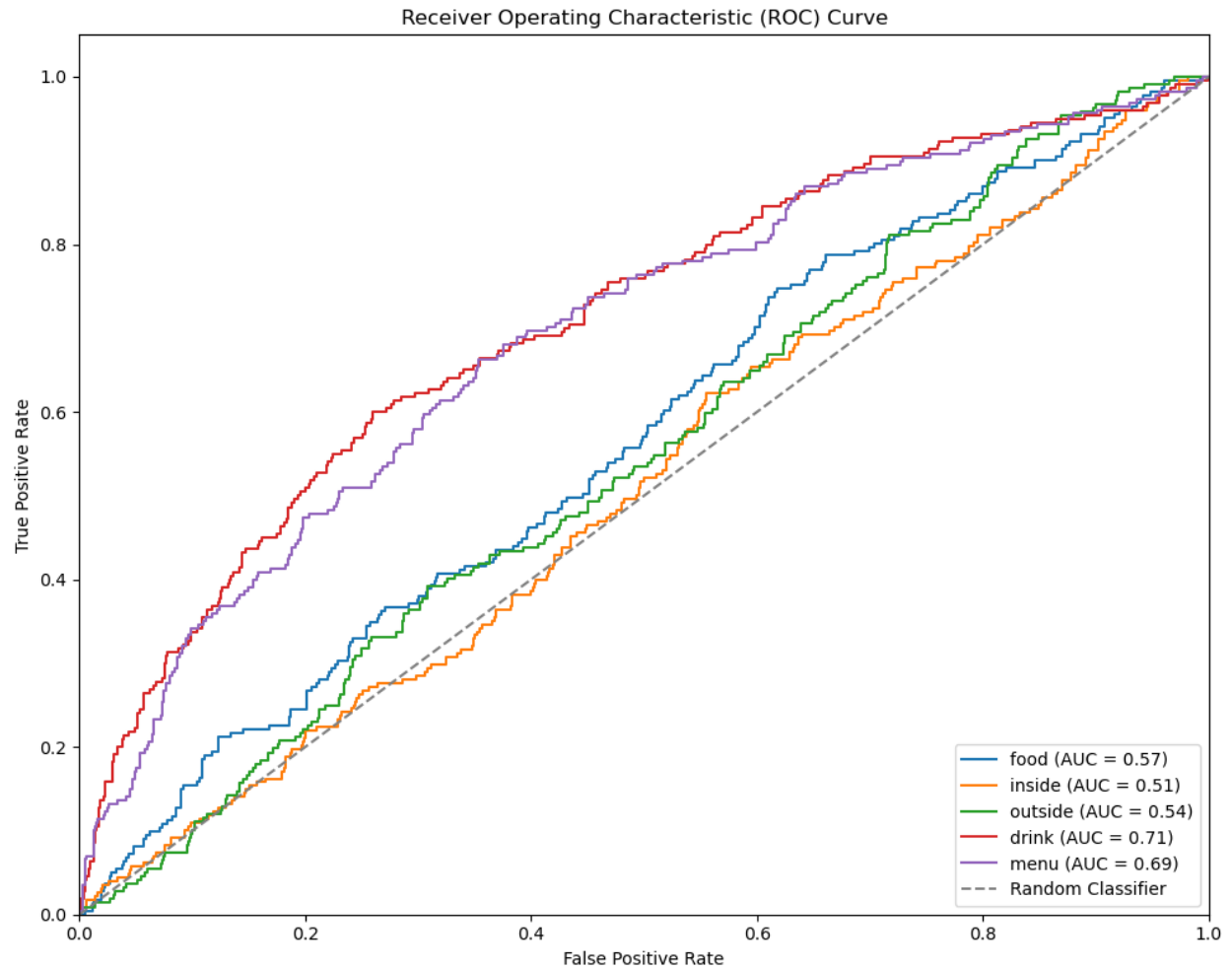
Confusion Matrix for test data

Confusion Matrix:

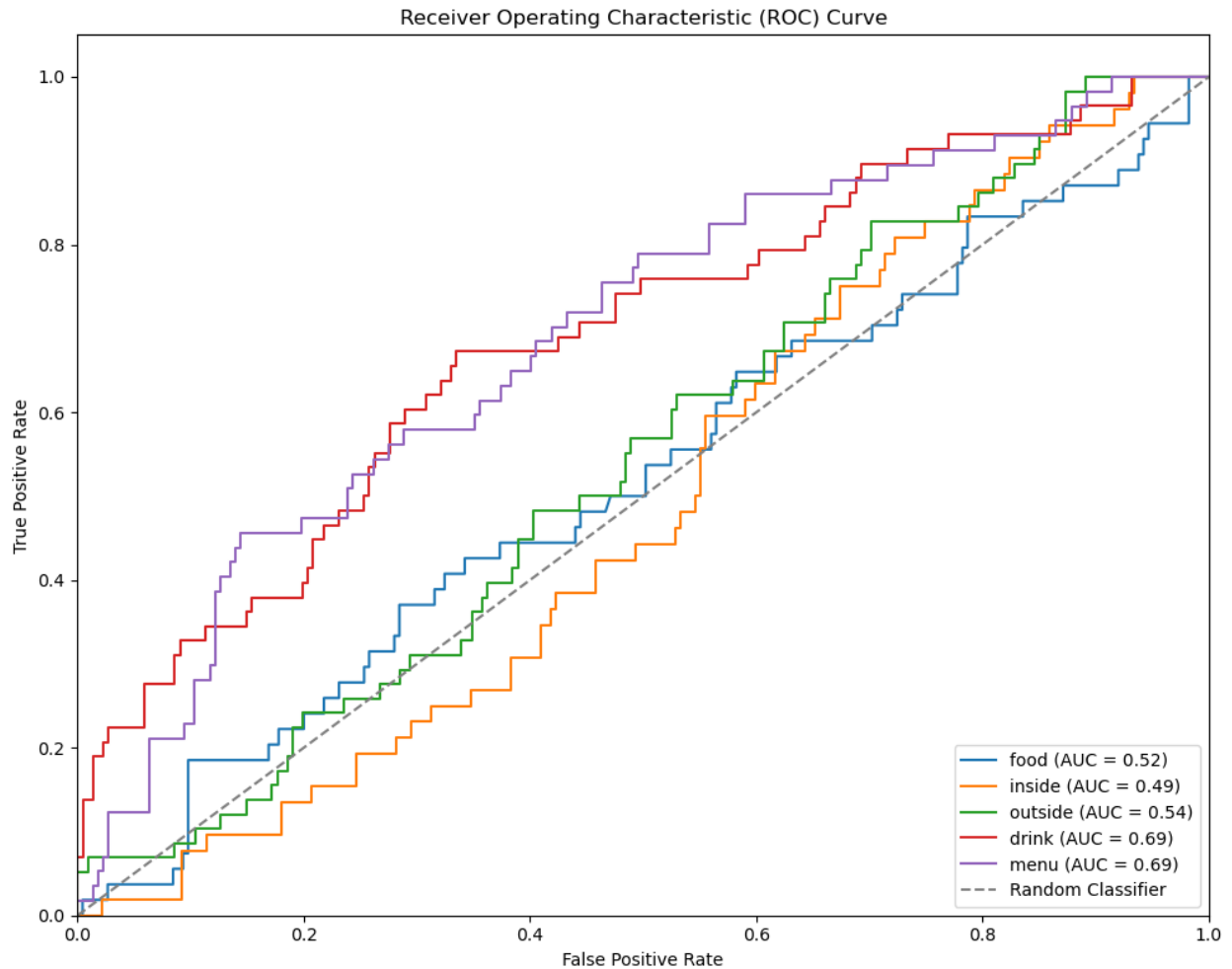
	food	inside	outside	drink	menu
food	54	0	0	0	0
inside	52	0	0	0	0
outside	58	0	0	0	0
drink	58	0	0	0	0
menu	57	0	0	0	0



R0C for train data



```
AUC for class food: 0.57
AUC for class inside: 0.51
AUC for class outside: 0.54
AUC for class drink: 0.71
AUC for class menu: 0.69
ROC for test data
```



```
AUC for class food: 0.52
AUC for class inside: 0.49
AUC for class outside: 0.54
AUC for class drink: 0.69
AUC for class menu: 0.69
```

lime

```
explain_with_lime(model, x_test[0], model_type="vgg")
{"model_id": "678f78e4126947deb630640deefa7e8", "version_major": 2, "version_minor": 0}
```

```
1/1 _____ 1s 528ms/step
1/1 _____ 1s 547ms/step
1/1 _____ 0s 480ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 1s 575ms/step
1/1 _____ 1s 540ms/step
```

1/1	_____	1s	545ms/step
1/1	_____	1s	561ms/step
1/1	_____	1s	545ms/step
1/1	_____	1s	539ms/step
1/1	_____	1s	558ms/step
1/1	_____	1s	541ms/step
1/1	_____	1s	554ms/step
1/1	_____	1s	543ms/step
1/1	_____	1s	573ms/step
1/1	_____	1s	540ms/step
1/1	_____	1s	516ms/step
1/1	_____	1s	568ms/step
1/1	_____	1s	518ms/step
1/1	_____	1s	552ms/step
1/1	_____	0s	494ms/step
1/1	_____	1s	545ms/step
1/1	_____	1s	503ms/step
1/1	_____	1s	550ms/step
1/1	_____	1s	556ms/step
1/1	_____	1s	534ms/step
1/1	_____	1s	550ms/step
1/1	_____	0s	491ms/step
1/1	_____	1s	541ms/step
1/1	_____	1s	557ms/step
1/1	_____	1s	558ms/step
1/1	_____	1s	555ms/step
1/1	_____	1s	555ms/step
1/1	_____	1s	527ms/step
1/1	_____	1s	568ms/step
1/1	_____	1s	518ms/step
1/1	_____	1s	535ms/step
1/1	_____	1s	564ms/step
1/1	_____	1s	555ms/step
1/1	_____	1s	556ms/step
1/1	_____	1s	526ms/step
1/1	_____	1s	532ms/step
1/1	_____	1s	511ms/step
1/1	_____	0s	487ms/step
1/1	_____	0s	487ms/step
1/1	_____	1s	512ms/step
1/1	_____	0s	465ms/step
1/1	_____	1s	552ms/step
1/1	_____	1s	553ms/step
1/1	_____	1s	556ms/step
1/1	_____	1s	528ms/step
1/1	_____	1s	536ms/step
1/1	_____	1s	550ms/step
1/1	_____	1s	553ms/step
1/1	_____	1s	557ms/step

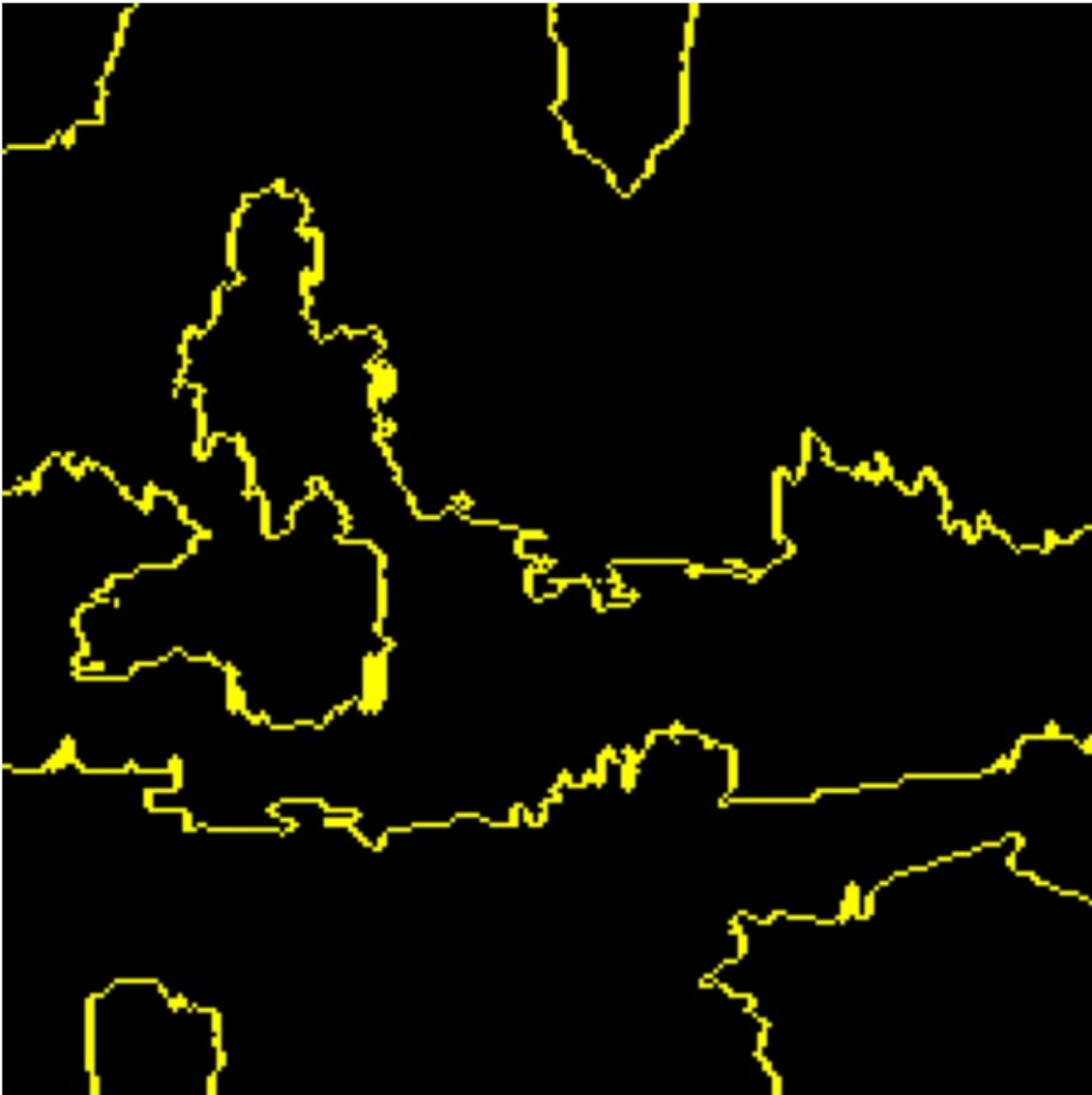
```

1/1 _____ 1s 563ms/step
1/1 _____ 1s 539ms/step
1/1 _____ 0s 495ms/step
1/1 _____ 1s 529ms/step
1/1 _____ 1s 516ms/step
1/1 _____ 0s 490ms/step
1/1 _____ 0s 484ms/step
1/1 _____ 1s 519ms/step
1/1 _____ 1s 527ms/step
1/1 _____ 1s 556ms/step
1/1 _____ 1s 545ms/step
1/1 _____ 1s 537ms/step
1/1 _____ 1s 547ms/step
1/1 _____ 0s 495ms/step
1/1 _____ 1s 548ms/step
1/1 _____ 0s 486ms/step
1/1 _____ 1s 511ms/step
1/1 _____ 1s 524ms/step
1/1 _____ 1s 550ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 1s 545ms/step
1/1 _____ 1s 552ms/step
1/1 _____ 0s 477ms/step
1/1 _____ 1s 524ms/step
1/1 _____ 1s 558ms/step
1/1 _____ 1s 556ms/step
1/1 _____ 1s 553ms/step
1/1 _____ 1s 551ms/step
1/1 _____ 1s 514ms/step
1/1 _____ 0s 495ms/step
1/1 _____ 1s 548ms/step
1/1 _____ 1s 523ms/step
1/1 _____ 1s 528ms/step
1/1 _____ 1s 504ms/step
1/1 _____ 1s 514ms/step
1/1 _____ 1s 548ms/step
1/1 _____ 1s 530ms/step
1/1 _____ 1s 518ms/step
1/1 _____ 1s 516ms/step
1/1 _____ 0s 496ms/step
1/1 _____ 0s 497ms/step
1/1 _____ 0s 479ms/step
1/1 _____ 1s 559ms/step
1/1 _____ 0s 473ms/step
1/1 _____ 1s 509ms/step

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-233.55799865722656..1.0].

LIME Explanation for VGG - Class 0



vgg freeze all layers with image augmentation

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

# Load VGG16 without top layers
```

```

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Freeze all layers of VGG16
base_model.trainable = False

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=False) # Keep batch norm layers
in inference mode
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator with augmentation for the training
data
train_datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# No augmentation for the test data
test_datagen = ImageDataGenerator()

# Apply augmentation to training data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Train the model (only the custom dense layers will be updated)
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()

# Calculate training time

```

```

training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)

print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)

print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)

```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```
self._warn_if_super_not_called()
```

```

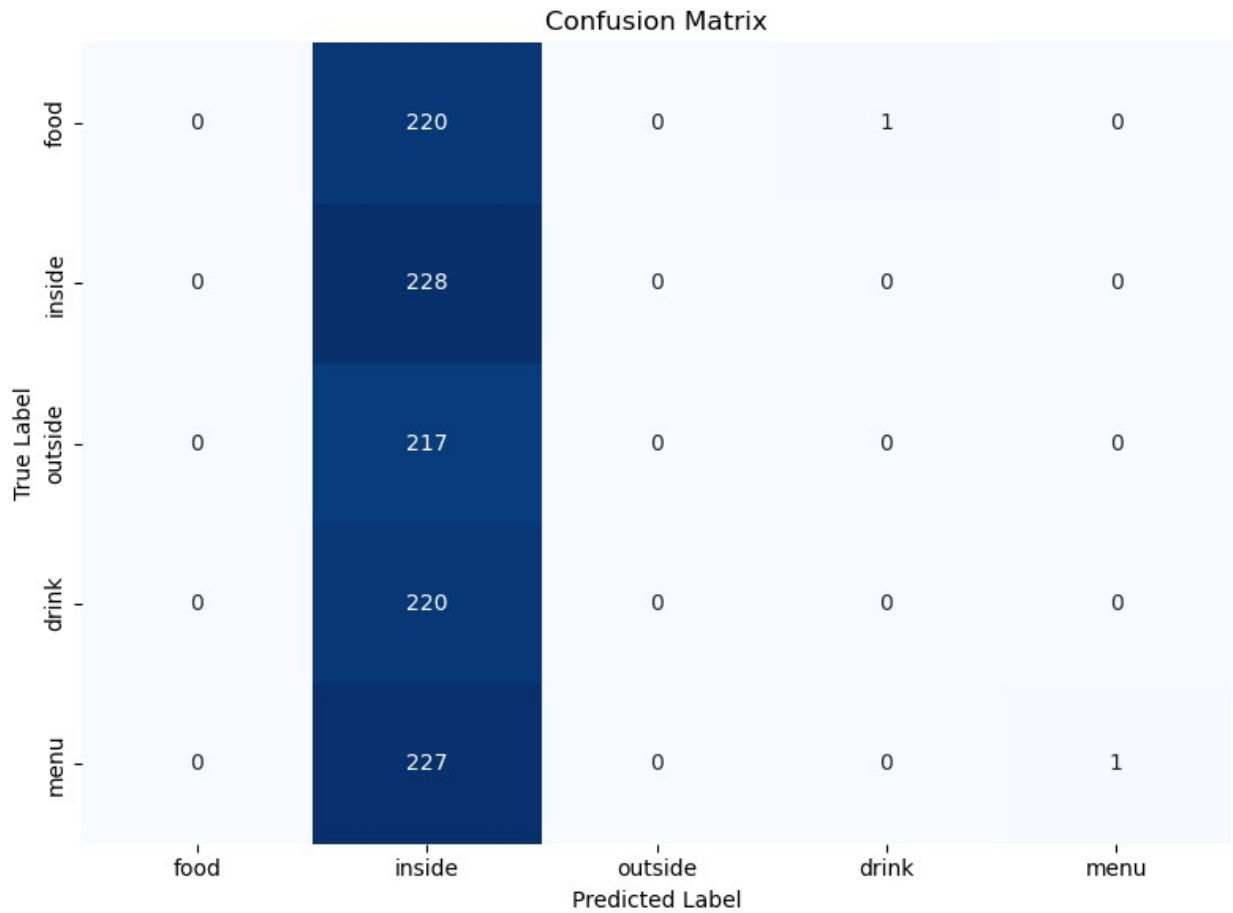
Epoch 1/10
35/35 ━━━━━━━━━━━ 64s 2s/step - accuracy: 0.2069 - loss:
2.4304 - val_accuracy: 0.1971 - val_loss: 1.6325
Epoch 2/10
35/35 ━━━━━━━━━━━ 62s 2s/step - accuracy: 0.2187 - loss:
2.0966 - val_accuracy: 0.1864 - val_loss: 1.6447
Epoch 3/10
35/35 ━━━━━━━━━━━ 62s 2s/step - accuracy: 0.2125 - loss:
2.0133 - val_accuracy: 0.1756 - val_loss: 1.6331
Epoch 4/10
35/35 ━━━━━━━━━━━ 62s 2s/step - accuracy: 0.2046 - loss:
1.9147 - val_accuracy: 0.2007 - val_loss: 1.6223
Epoch 5/10
35/35 ━━━━━━━━━━━ 62s 2s/step - accuracy: 0.1932 - loss:
1.8939 - val_accuracy: 0.1935 - val_loss: 1.6249
Epoch 6/10
35/35 ━━━━━━━━━━━ 62s 2s/step - accuracy: 0.2055 - loss:

```

```

1.7781 - val_accuracy: 0.1720 - val_loss: 1.6201
Epoch 7/10
35/35 _____ 62s 2s/step - accuracy: 0.1980 - loss:
1.8237 - val_accuracy: 0.1613 - val_loss: 1.6214
Epoch 8/10
35/35 _____ 63s 2s/step - accuracy: 0.1723 - loss:
1.8235 - val_accuracy: 0.2007 - val_loss: 1.6237
Epoch 9/10
35/35 _____ 61s 2s/step - accuracy: 0.1597 - loss:
1.7849 - val_accuracy: 0.1864 - val_loss: 1.6190
Epoch 10/10
35/35 _____ 60s 2s/step - accuracy: 0.1961 - loss:
1.7560 - val_accuracy: 0.1864 - val_loss: 1.6167
Training Time: 622.15 seconds
35/35 _____ 48s 1s/step
9/9 _____ 12s 1s/step
Confusion Matrix for train data
Confusion Matrix:
      food  inside  outside  drink  menu
food      0     220        0       1     0
inside    0     228        0       0     0
outside   0     217        0       0     0
drink     0     220        0       0     0
menu      0     227        0       0     1

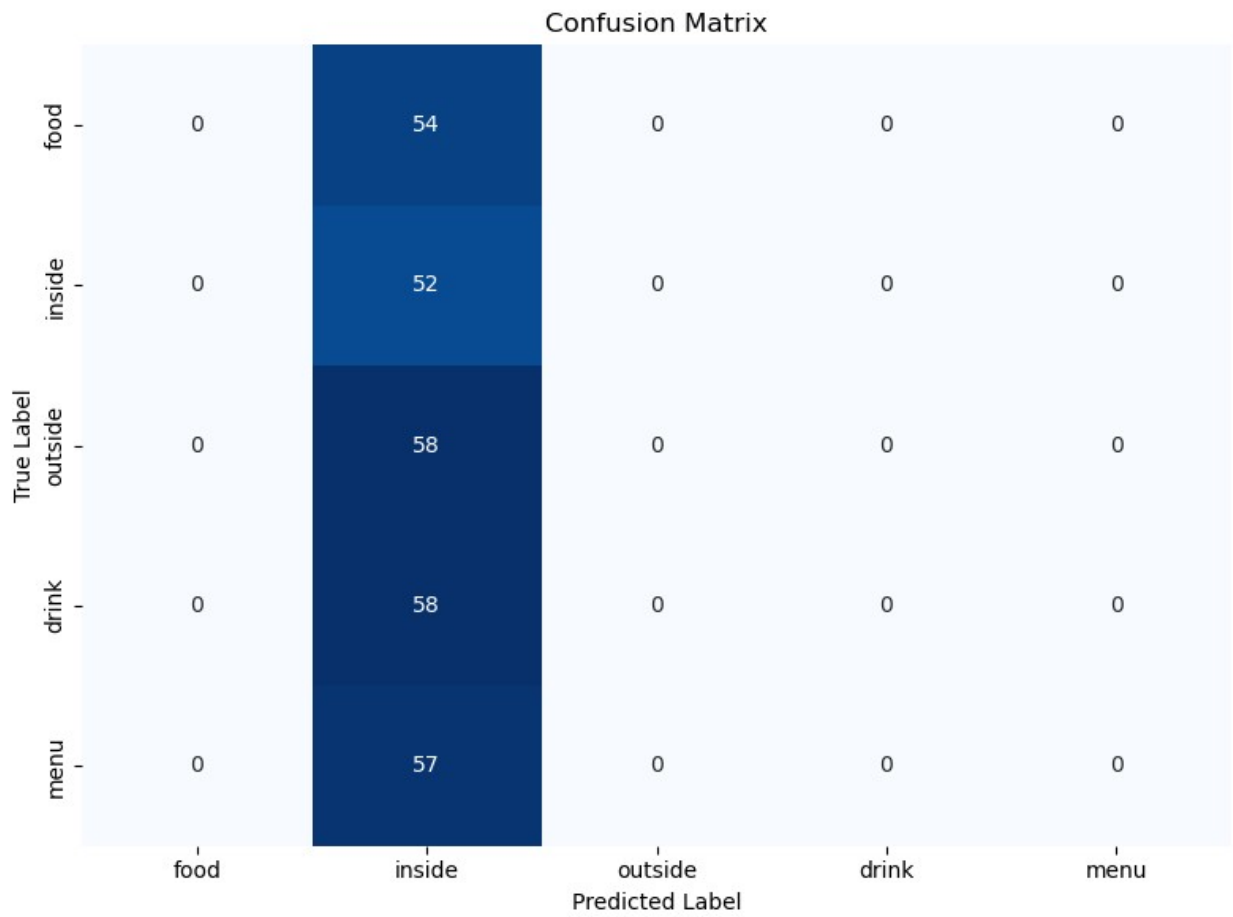
```



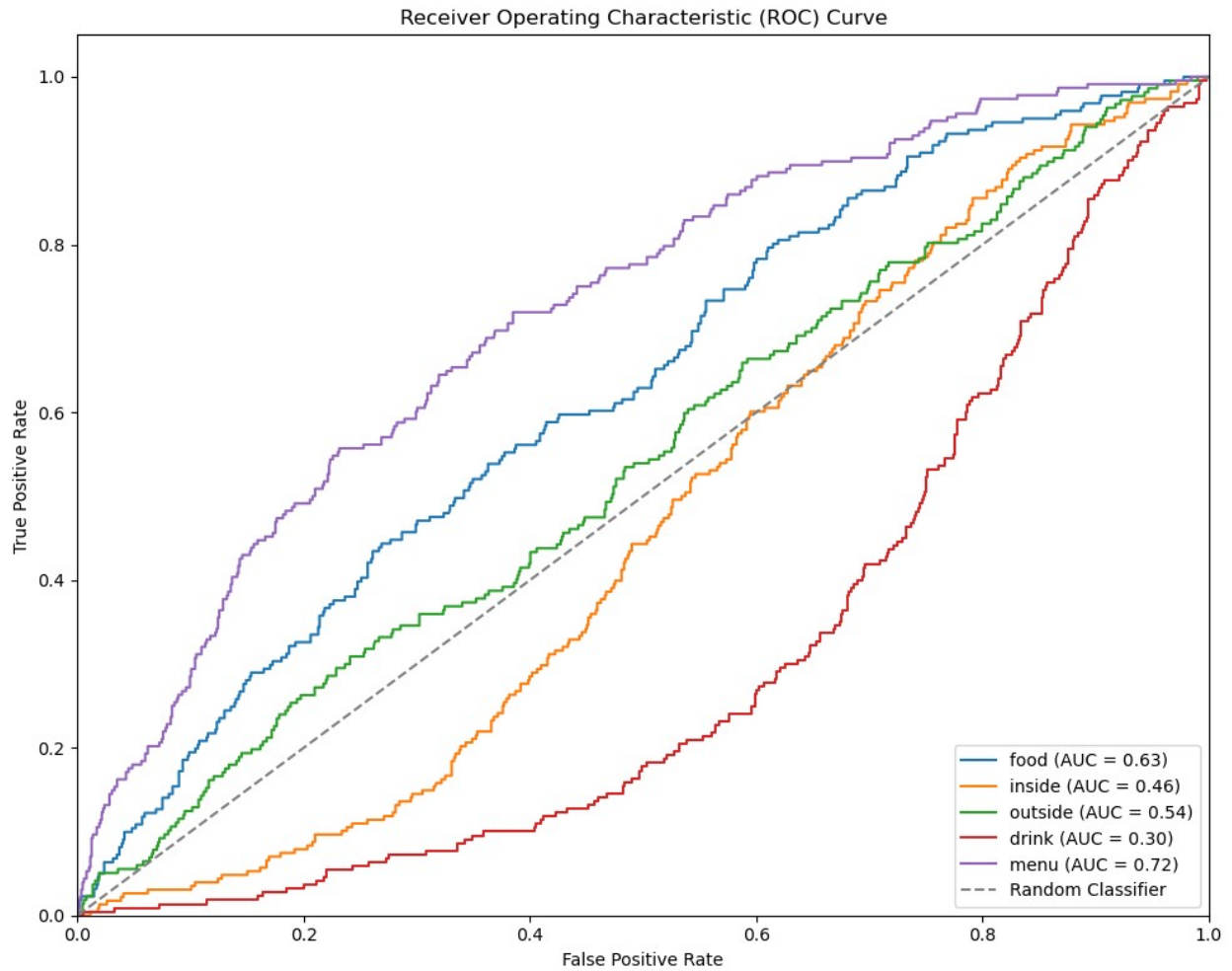
Confusion Matrix for test data

Confusion Matrix:

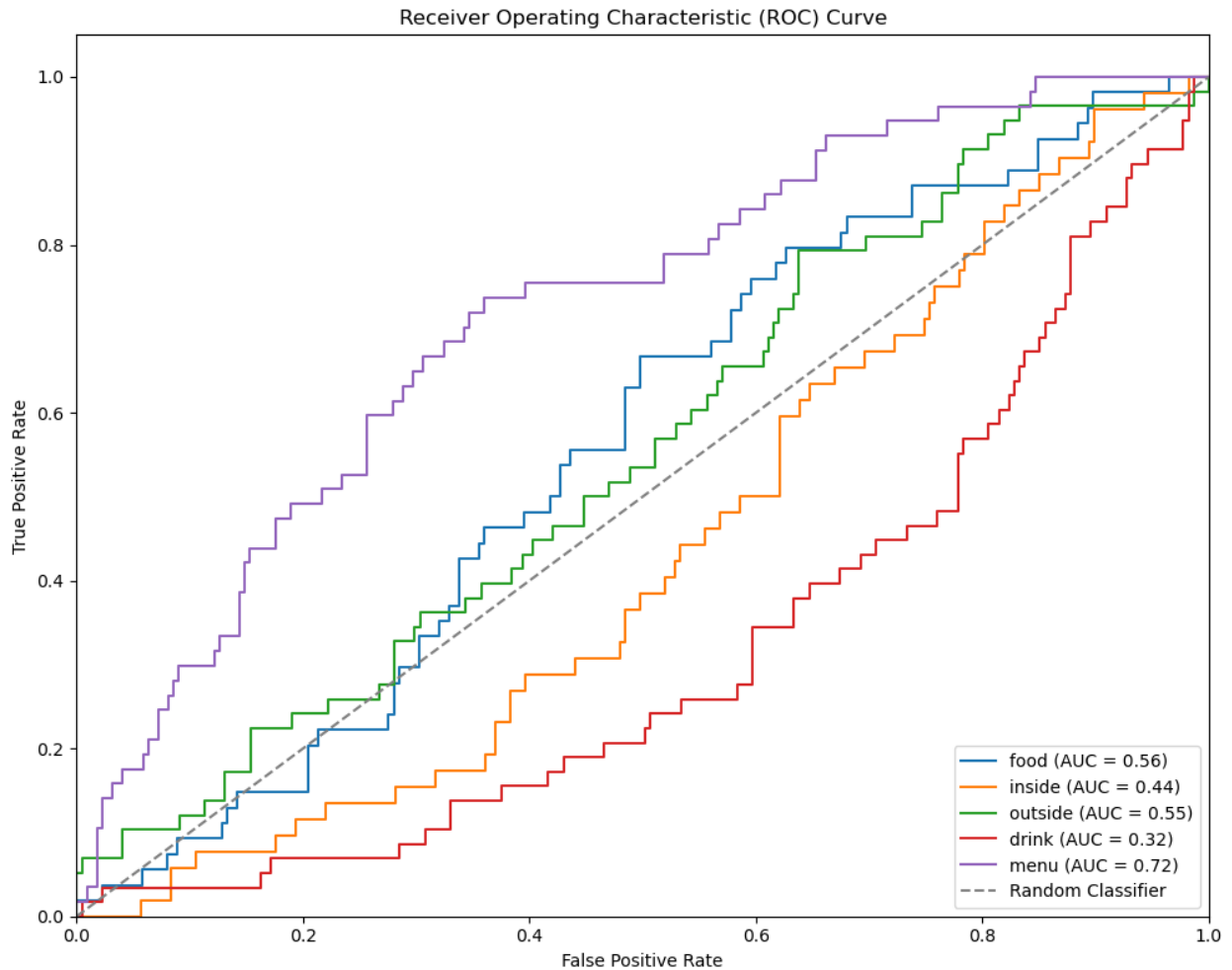
	food	inside	outside	drink	menu
food	0	54	0	0	0
inside	0	52	0	0	0
outside	0	58	0	0	0
drink	0	58	0	0	0
menu	0	57	0	0	0



ROC for train data



```
AUC for class food: 0.63
AUC for class inside: 0.46
AUC for class outside: 0.54
AUC for class drink: 0.30
AUC for class menu: 0.72
ROC for test data
```



```
AUC for class food: 0.56
AUC for class inside: 0.44
AUC for class outside: 0.55
AUC for class drink: 0.32
AUC for class menu: 0.72
```

lime

```
explain_with_lime(model, x_test[0], model_type="vgg")
{"model_id": "8c01d3048059498883635de2740b7f4a", "version_major": 2, "version_minor": 0}
```

```
1/1 _____ 1s 556ms/step
1/1 _____ 1s 549ms/step
1/1 _____ 0s 499ms/step
1/1 _____ 1s 505ms/step
1/1 _____ 1s 543ms/step
1/1 _____ 1s 525ms/step
```


1/1	_____	1s	562ms/step
1/1	_____	1s	501ms/step
1/1	_____	0s	463ms/step
1/1	_____	1s	541ms/step
1/1	_____	1s	546ms/step
1/1	_____	1s	545ms/step
1/1	_____	1s	535ms/step
1/1	_____	1s	567ms/step
1/1	_____	0s	457ms/step
1/1	_____	1s	510ms/step
1/1	_____	0s	473ms/step
1/1	_____	0s	455ms/step
1/1	_____	1s	513ms/step
1/1	_____	1s	541ms/step
1/1	_____	1s	521ms/step
1/1	_____	1s	547ms/step
1/1	_____	1s	510ms/step
1/1	_____	0s	449ms/step
1/1	_____	0s	450ms/step
1/1	_____	0s	451ms/step
1/1	_____	1s	552ms/step
1/1	_____	0s	460ms/step
1/1	_____	0s	498ms/step
1/1	_____	1s	550ms/step
1/1	_____	1s	525ms/step
1/1	_____	1s	535ms/step
1/1	_____	1s	547ms/step
1/1	_____	1s	548ms/step
1/1	_____	0s	453ms/step
1/1	_____	0s	464ms/step
1/1	_____	1s	505ms/step
1/1	_____	0s	479ms/step
1/1	_____	1s	522ms/step
1/1	_____	1s	541ms/step
1/1	_____	0s	456ms/step
1/1	_____	1s	541ms/step
1/1	_____	1s	544ms/step
1/1	_____	0s	453ms/step
1/1	_____	1s	549ms/step
1/1	_____	1s	523ms/step
1/1	_____	1s	532ms/step
1/1	_____	0s	491ms/step
1/1	_____	1s	542ms/step
1/1	_____	1s	550ms/step
1/1	_____	1s	556ms/step
1/1	_____	1s	535ms/step
1/1	_____	1s	551ms/step
1/1	_____	1s	553ms/step
1/1	_____	1s	523ms/step

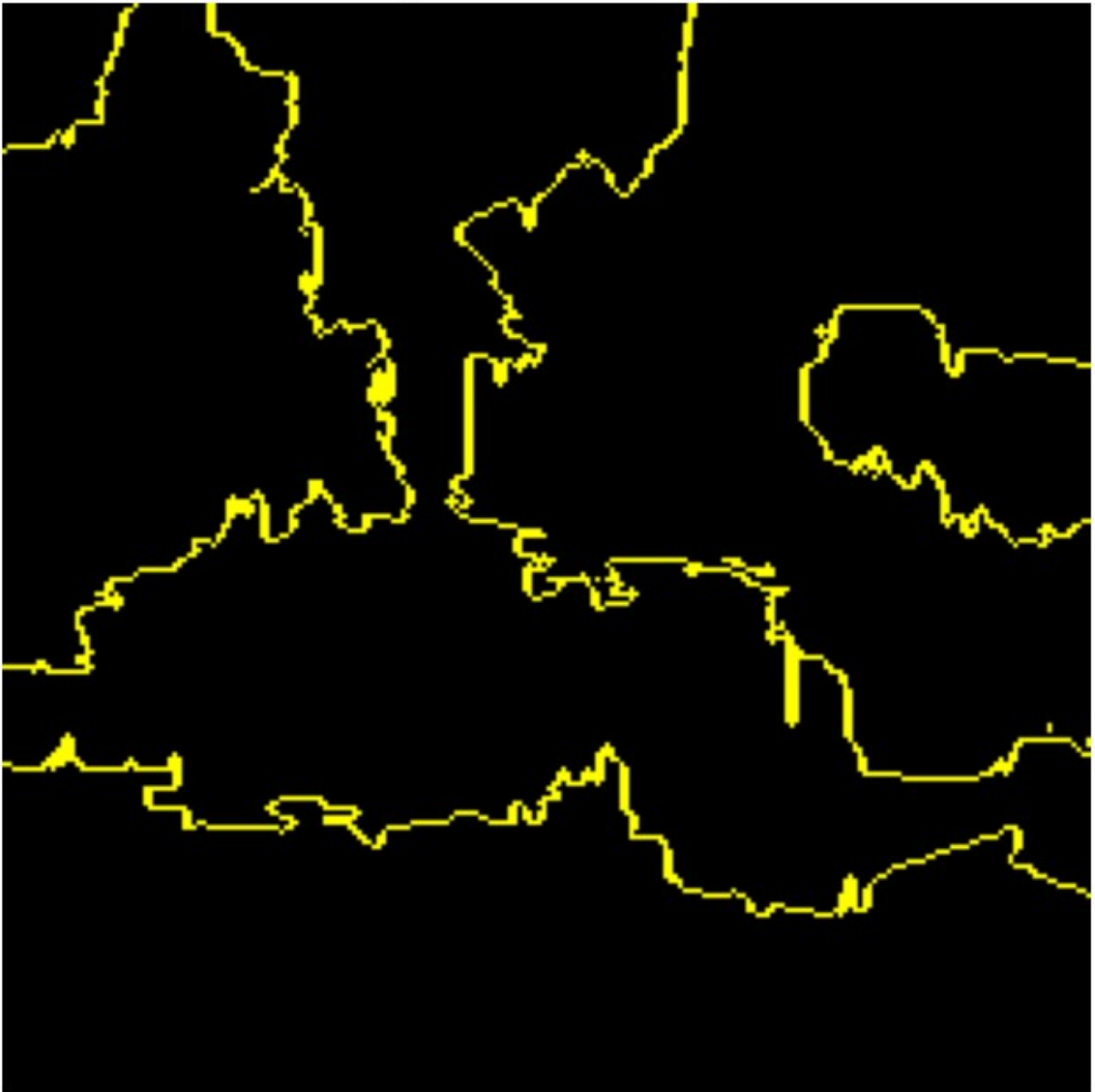
```

1/1 _____ 1s 530ms/step
1/1 _____ 1s 535ms/step
1/1 _____ 0s 469ms/step
1/1 _____ 0s 499ms/step
1/1 _____ 0s 494ms/step
1/1 _____ 1s 541ms/step
1/1 _____ 1s 551ms/step
1/1 _____ 1s 512ms/step
1/1 _____ 1s 541ms/step
1/1 _____ 1s 528ms/step
1/1 _____ 0s 454ms/step
1/1 _____ 0s 465ms/step
1/1 _____ 1s 501ms/step
1/1 _____ 1s 509ms/step
1/1 _____ 0s 476ms/step
1/1 _____ 0s 448ms/step
1/1 _____ 1s 508ms/step
1/1 _____ 1s 508ms/step
1/1 _____ 0s 459ms/step
1/1 _____ 1s 539ms/step
1/1 _____ 1s 558ms/step
1/1 _____ 1s 532ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 0s 469ms/step
1/1 _____ 1s 531ms/step
1/1 _____ 1s 551ms/step
1/1 _____ 1s 521ms/step
1/1 _____ 1s 524ms/step
1/1 _____ 1s 529ms/step
1/1 _____ 0s 465ms/step
1/1 _____ 0s 477ms/step
1/1 _____ 1s 540ms/step
1/1 _____ 1s 535ms/step
1/1 _____ 0s 460ms/step
1/1 _____ 0s 457ms/step
1/1 _____ 1s 532ms/step
1/1 _____ 0s 459ms/step
1/1 _____ 0s 495ms/step
1/1 _____ 1s 508ms/step
1/1 _____ 1s 522ms/step
1/1 _____ 0s 495ms/step
1/1 _____ 1s 528ms/step
1/1 _____ 0s 459ms/step
1/1 _____ 1s 552ms/step
1/1 _____ 0s 446ms/step

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-351.29901123046875..1.0].

LIME Explanation for VGG - Class 3



vgg with some unfreeze layers and image augmentation

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

# Load VGG16 without top layers
```

```

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Unfreeze the last few layers of the base model and freeze the rest
for layer in base_model.layers[:-4]: # Freeze all layers except the
last 4
    layer.trainable = False

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=True) # Training=True to fine-
tune the unfrozen layers
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator with augmentation for the training
data
train_datagen = ImageDataGenerator(
    rotation_range=40, # Random rotation between 0 and 40 degrees
    width_shift_range=0.2, # Horizontal shift
    height_shift_range=0.2, # Vertical shift
    shear_range=0.2, # Shearing transformations
    zoom_range=0.2, # Random zoom
    horizontal_flip=True, # Random horizontal flip
    fill_mode='nearest' # Filling mode for newly created pixels
)

# No augmentation for the test data, just use it as it is
test_datagen = ImageDataGenerator()

# Apply the augmentation to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Fine-tune the model with augmented images
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()

```

```

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)

print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)

print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)

```

```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.

```

```

    self._warn_if_super_not_called()

```

```

Epoch 1/10
35/35 ━━━━━━━━━━━ 69s 2s/step - accuracy: 0.2143 - loss:
1.8468 - val_accuracy: 0.1900 - val_loss: 1.6092
Epoch 2/10
35/35 ━━━━━━━━━━━ 68s 2s/step - accuracy: 0.2016 - loss:
1.6098 - val_accuracy: 0.1864 - val_loss: 1.6097
Epoch 3/10
35/35 ━━━━━━━━━━━ 67s 2s/step - accuracy: 0.1752 - loss:
1.6099 - val_accuracy: 0.1864 - val_loss: 1.6098
Epoch 4/10
35/35 ━━━━━━━━━━━ 67s 2s/step - accuracy: 0.2072 - loss:
1.6096 - val_accuracy: 0.1864 - val_loss: 1.6096
Epoch 5/10
35/35 ━━━━━━━━━━━ 67s 2s/step - accuracy: 0.2057 - loss:
1.6095 - val_accuracy: 0.1864 - val_loss: 1.6096

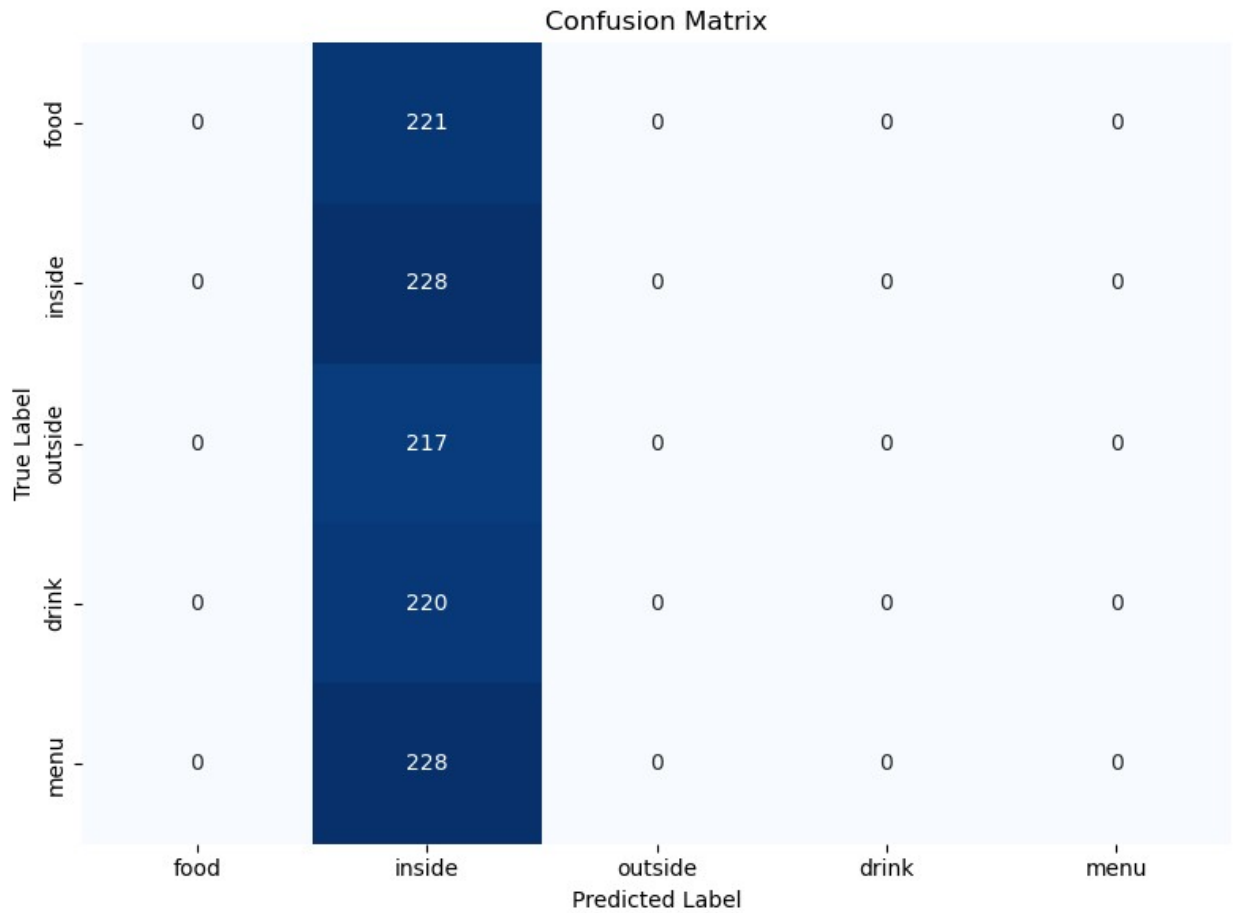
```

```

Epoch 6/10
35/35 _____ 67s 2s/step - accuracy: 0.2009 - loss:
1.6095 - val_accuracy: 0.1864 - val_loss: 1.6096
Epoch 7/10
35/35 _____ 67s 2s/step - accuracy: 0.1874 - loss:
1.6095 - val_accuracy: 0.1864 - val_loss: 1.6096
Epoch 8/10
35/35 _____ 67s 2s/step - accuracy: 0.1901 - loss:
1.6094 - val_accuracy: 0.1864 - val_loss: 1.6097
Epoch 9/10
35/35 _____ 67s 2s/step - accuracy: 0.1789 - loss:
1.6096 - val_accuracy: 0.1864 - val_loss: 1.6097
Epoch 10/10
35/35 _____ 67s 2s/step - accuracy: 0.2081 - loss:
1.6090 - val_accuracy: 0.1864 - val_loss: 1.6097
Training Time: 676.23 seconds
35/35 _____ 47s 1s/step
9/9 _____ 11s 1s/step
Confusion Matrix for train data
Confusion Matrix:

```

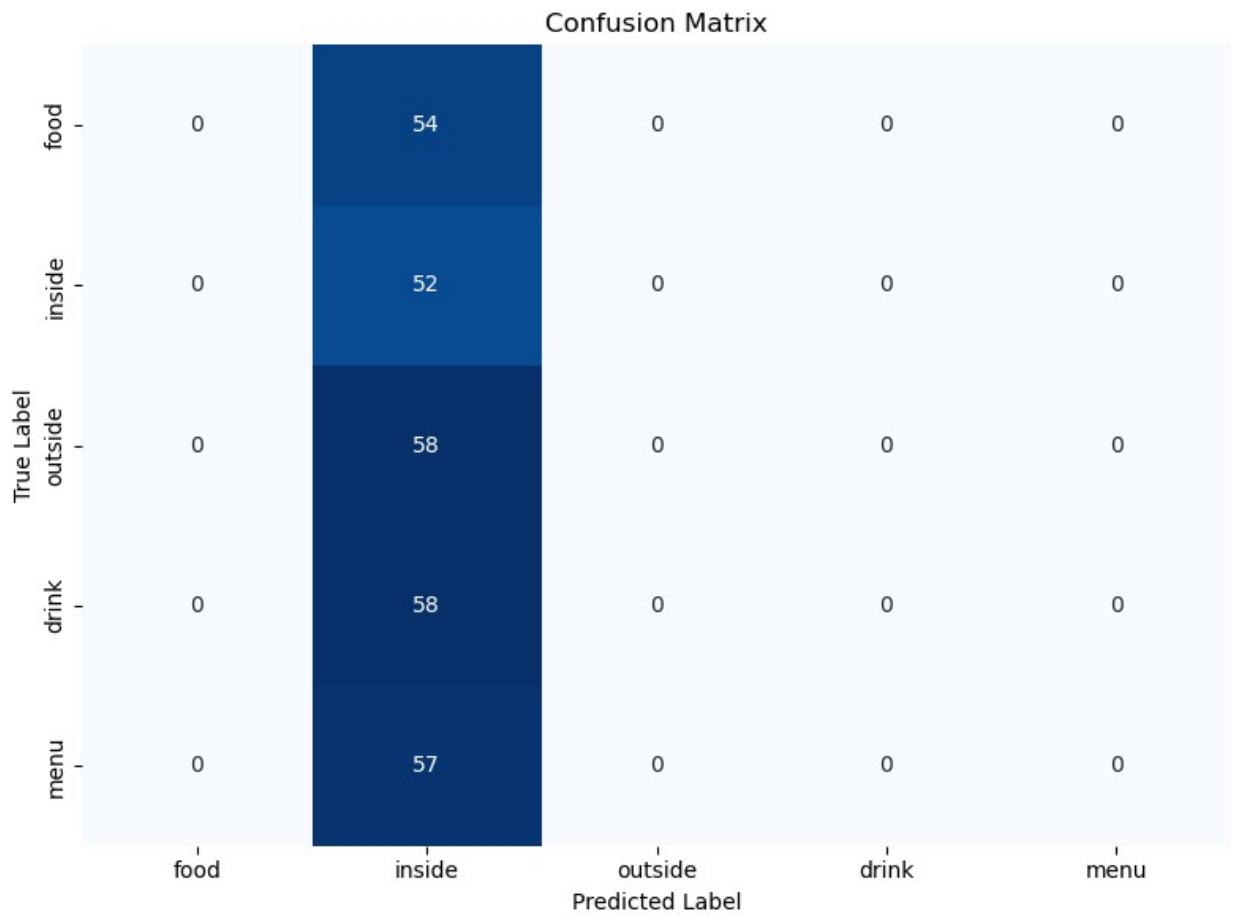
	food	inside	outside	drink	menu
food	0	221	0	0	0
inside	0	228	0	0	0
outside	0	217	0	0	0
drink	0	220	0	0	0
menu	0	228	0	0	0



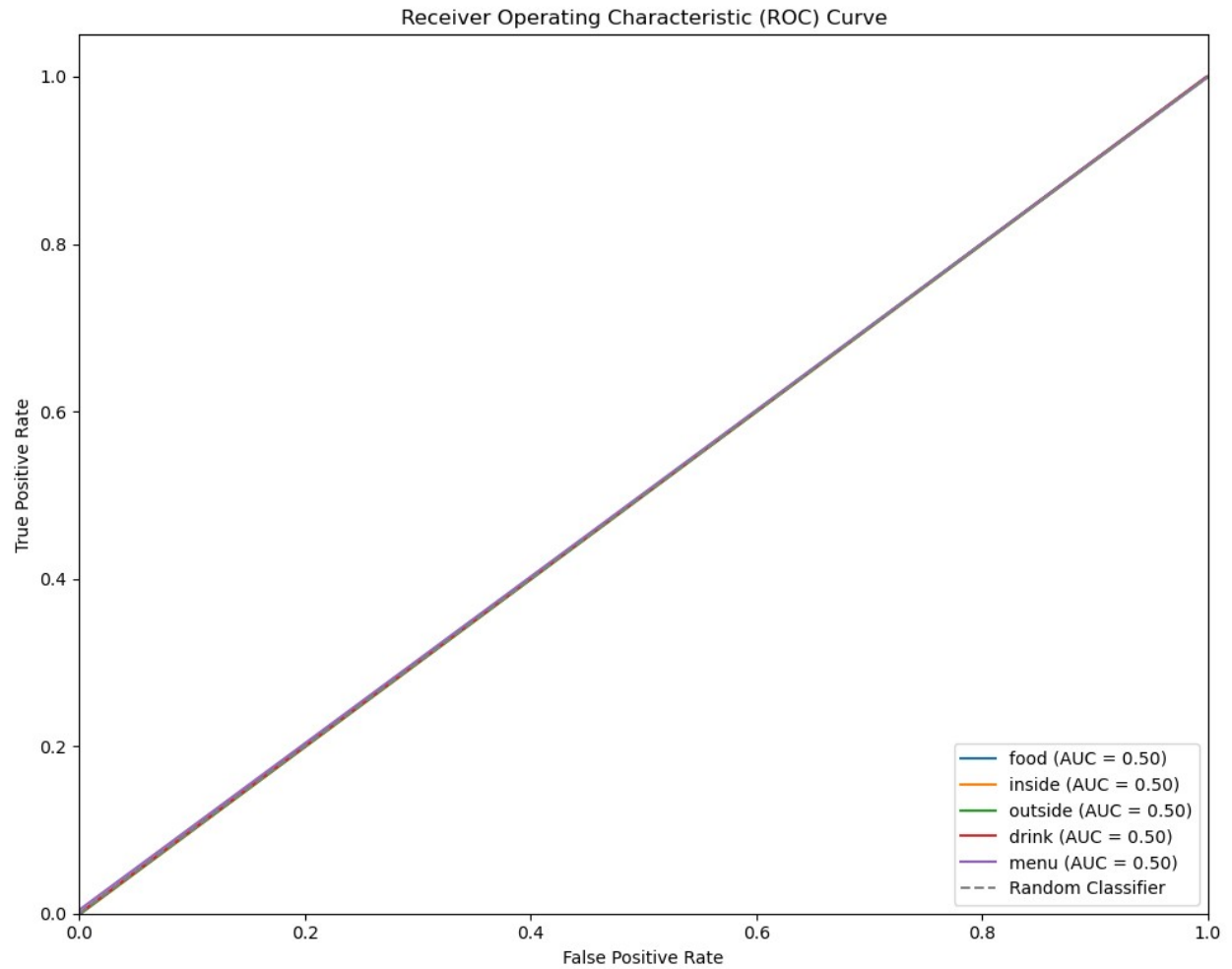
Confusion Matrix for test data

Confusion Matrix:

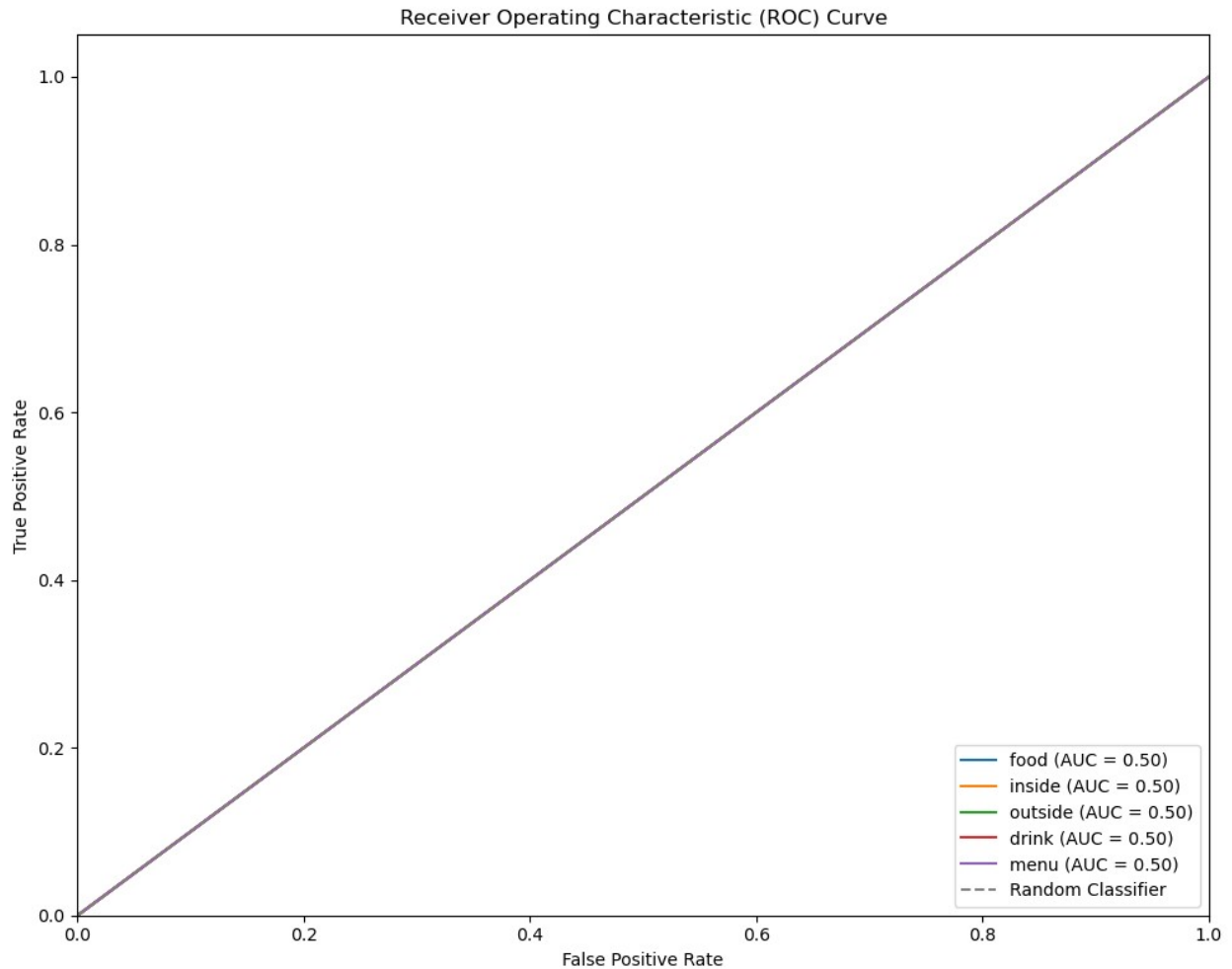
	food	inside	outside	drink	menu
food	0	54	0	0	0
inside	0	52	0	0	0
outside	0	58	0	0	0
drink	0	58	0	0	0
menu	0	57	0	0	0



ROC for train data



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.50
AUC for class drink: 0.50
AUC for class menu: 0.50
ROC for test data
```



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.50
AUC for class drink: 0.50
AUC for class menu: 0.50
```

lime

```
explain_with_lime(model, x_test[0], model_type="vgg")
{"model_id": "7b1080c0ad7a46d29cf51bec5222ee36", "version_major": 2, "version_minor": 0}

1/1 _____ 0s 449ms/step
1/1 _____ 0s 471ms/step
1/1 _____ 0s 492ms/step
1/1 _____ 0s 487ms/step
1/1 _____ 0s 434ms/step
1/1 _____ 0s 490ms/step
```

1/1	_____	0s	498ms/step
1/1	_____	1s	521ms/step
1/1	_____	0s	480ms/step
1/1	_____	1s	524ms/step
1/1	_____	1s	501ms/step
1/1	_____	1s	522ms/step
1/1	_____	1s	539ms/step
1/1	_____	1s	533ms/step
1/1	_____	1s	509ms/step
1/1	_____	1s	517ms/step
1/1	_____	1s	514ms/step
1/1	_____	1s	508ms/step
1/1	_____	1s	513ms/step
1/1	_____	0s	493ms/step
1/1	_____	0s	436ms/step
1/1	_____	0s	494ms/step
1/1	_____	0s	490ms/step
1/1	_____	0s	445ms/step
1/1	_____	0s	492ms/step
1/1	_____	1s	509ms/step
1/1	_____	0s	493ms/step
1/1	_____	1s	526ms/step
1/1	_____	0s	429ms/step
1/1	_____	0s	500ms/step
1/1	_____	1s	540ms/step
1/1	_____	1s	509ms/step
1/1	_____	1s	526ms/step
1/1	_____	0s	485ms/step
1/1	_____	0s	433ms/step
1/1	_____	0s	443ms/step
1/1	_____	1s	522ms/step
1/1	_____	1s	509ms/step
1/1	_____	0s	448ms/step
1/1	_____	1s	524ms/step
1/1	_____	1s	525ms/step
1/1	_____	0s	484ms/step
1/1	_____	1s	510ms/step
1/1	_____	0s	444ms/step
1/1	_____	0s	429ms/step
1/1	_____	1s	520ms/step
1/1	_____	1s	521ms/step
1/1	_____	1s	525ms/step
1/1	_____	1s	522ms/step
1/1	_____	1s	532ms/step
1/1	_____	1s	545ms/step
1/1	_____	1s	516ms/step
1/1	_____	1s	541ms/step
1/1	_____	0s	497ms/step
1/1	_____	1s	509ms/step

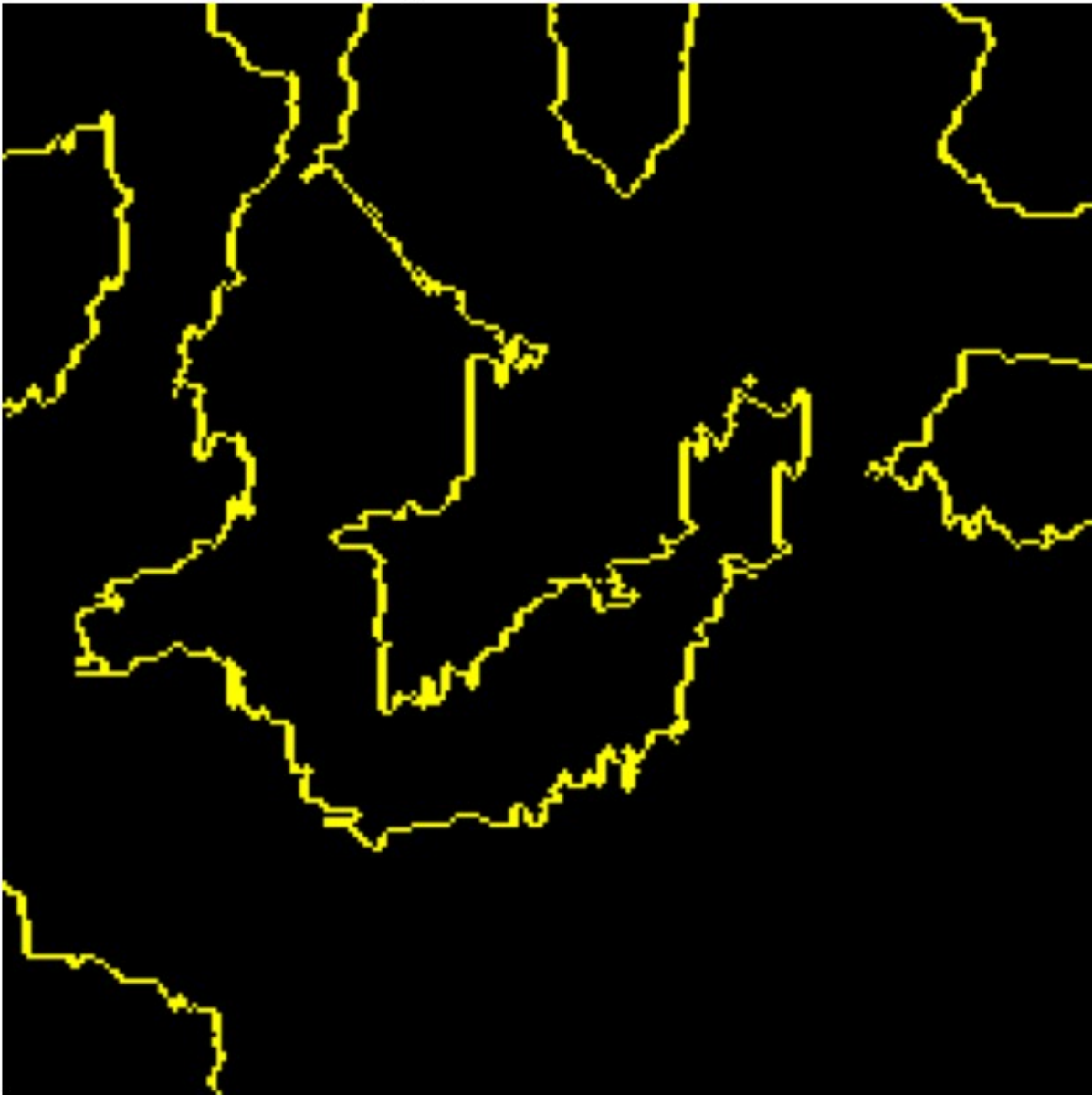
```

1/1 _____ 0s 467ms/step
1/1 _____ 1s 515ms/step
1/1 _____ 0s 452ms/step
1/1 _____ 0s 441ms/step
1/1 _____ 1s 545ms/step
1/1 _____ 0s 442ms/step
1/1 _____ 0s 456ms/step
1/1 _____ 0s 482ms/step
1/1 _____ 1s 519ms/step
1/1 _____ 0s 429ms/step
1/1 _____ 0s 427ms/step
1/1 _____ 1s 517ms/step
1/1 _____ 1s 522ms/step
1/1 _____ 0s 455ms/step
1/1 _____ 0s 475ms/step
1/1 _____ 0s 433ms/step
1/1 _____ 1s 541ms/step
1/1 _____ 1s 524ms/step
1/1 _____ 0s 445ms/step
1/1 _____ 0s 429ms/step
1/1 _____ 0s 436ms/step
1/1 _____ 0s 472ms/step
1/1 _____ 0s 485ms/step
1/1 _____ 0s 453ms/step
1/1 _____ 0s 453ms/step
1/1 _____ 0s 479ms/step
1/1 _____ 0s 476ms/step
1/1 _____ 1s 500ms/step
1/1 _____ 0s 444ms/step
1/1 _____ 0s 484ms/step
1/1 _____ 0s 445ms/step
1/1 _____ 0s 494ms/step
1/1 _____ 0s 468ms/step
1/1 _____ 1s 508ms/step
1/1 _____ 1s 515ms/step
1/1 _____ 0s 450ms/step
1/1 _____ 1s 518ms/step
1/1 _____ 1s 506ms/step
1/1 _____ 1s 524ms/step
1/1 _____ 1s 516ms/step
1/1 _____ 0s 461ms/step
1/1 _____ 1s 547ms/step
1/1 _____ 0s 484ms/step
1/1 _____ 0s 471ms/step
1/1 _____ 0s 493ms/step

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-474.97900390625..1.0].

LIME Explanation for VGG - Class 1



unfreeze all layers

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np

# Load VGG16 without top layers
```

```

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = True # No freezing, fine-tune the entire model

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=True) # Training=True to fine-
tune
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator (without augmentation for the training
data)
train_datagen = ImageDataGenerator()

# No augmentation for the test data, just use it as it is
test_datagen = ImageDataGenerator()

# Apply the generator to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Fine-tune the model with the original images (no augmentation)
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

```

```

# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)

print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)

print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)

```

Epoch 1/10

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```
self._warn_if_super_not_called()
```

```
35/35 _____ 187s 5s/step - accuracy: 0.1772 - loss: 1.8138 - val_accuracy: 0.1864 - val_loss: 1.6114
```

Epoch 2/10

```
35/35 _____ 186s 5s/step - accuracy: 0.1962 - loss: 1.6094 - val_accuracy: 0.2079 - val_loss: 1.6094
```

Epoch 3/10

```
35/35 _____ 186s 5s/step - accuracy: 0.2062 - loss: 1.6095 - val_accuracy: 0.2079 - val_loss: 1.6094
```

Epoch 4/10

```
35/35 _____ 185s 5s/step - accuracy: 0.1854 - loss: 1.6100 - val_accuracy: 0.2079 - val_loss: 1.6095
```

Epoch 5/10

```
35/35 _____ 186s 5s/step - accuracy: 0.2025 - loss: 1.6095 - val_accuracy: 0.1864 - val_loss: 1.6096
```

Epoch 6/10

```
35/35 _____ 186s 5s/step - accuracy: 0.1993 - loss: 1.6095 - val_accuracy: 0.1864 - val_loss: 1.6096
```

Epoch 7/10

```
35/35 _____ 186s 5s/step - accuracy: 0.1967 - loss: 1.6095 - val_accuracy: 0.1864 - val_loss: 1.6096
```

Epoch 8/10

```
35/35 _____ 186s 5s/step - accuracy: 0.2044 - loss: 1.6094 - val_accuracy: 0.1864 - val_loss: 1.6096
```

Epoch 9/10

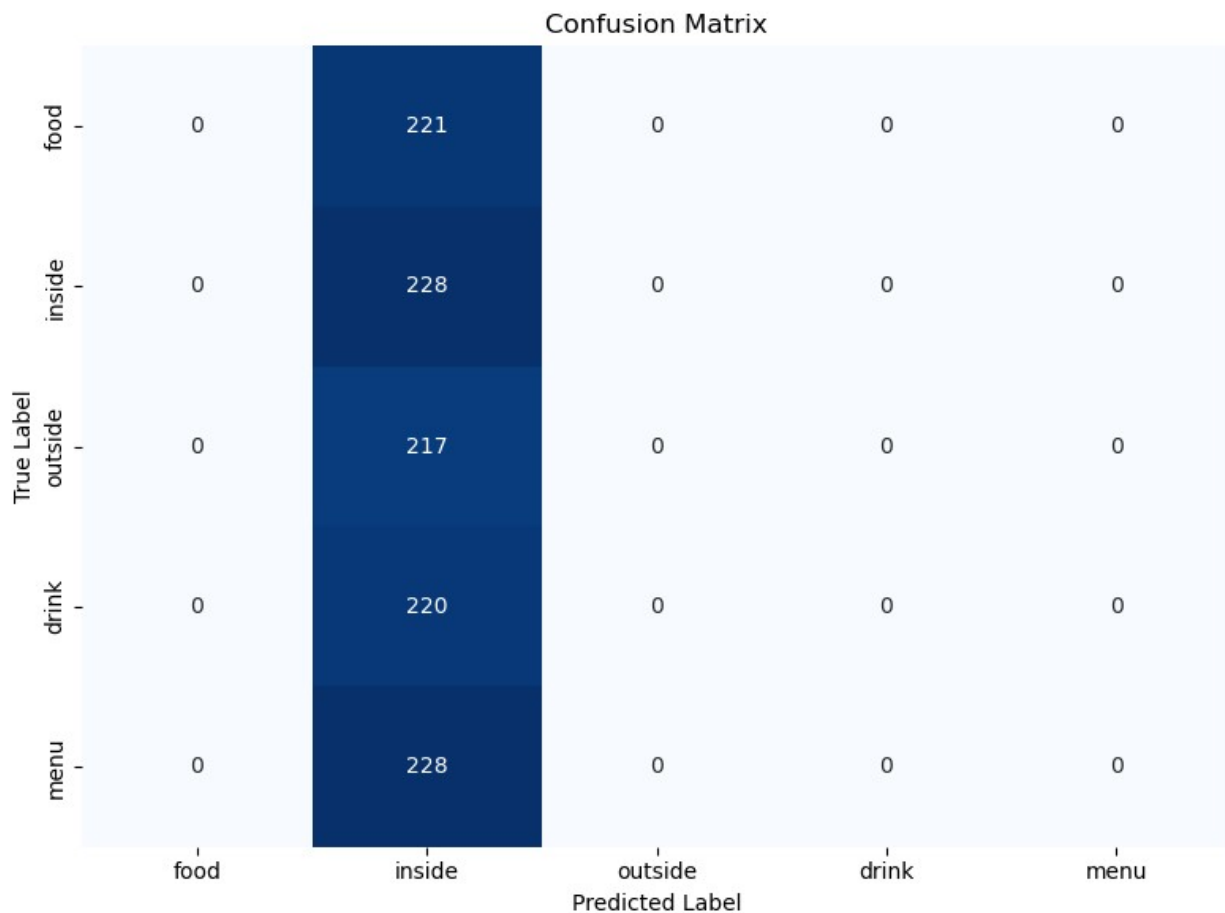
```
35/35 _____ 186s 5s/step - accuracy: 0.2055 - loss:
```

```

1.6096 - val_accuracy: 0.1864 - val_loss: 1.6096
Epoch 10/10
35/35 _____ 186s 5s/step - accuracy: 0.2036 - loss:
1.6092 - val_accuracy: 0.1864 - val_loss: 1.6097
Training Time: 1860.55 seconds
35/35 _____ 47s 1s/step
9/9 _____ 12s 1s/step
Confusion Matrix for train data
Confusion Matrix:

```

	food	inside	outside	drink	menu
food	0	221	0	0	0
inside	0	228	0	0	0
outside	0	217	0	0	0
drink	0	220	0	0	0
menu	0	228	0	0	0



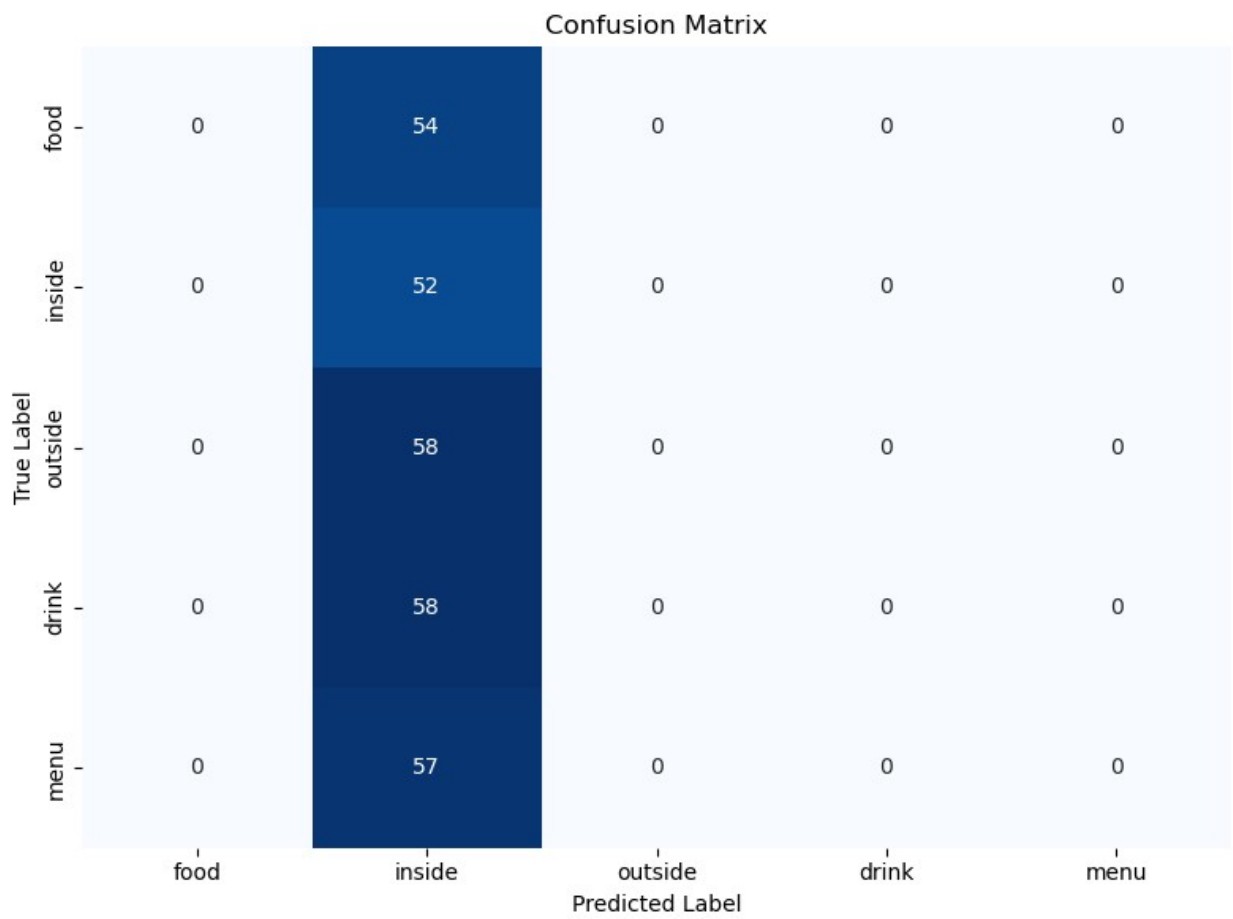
```

Confusion Matrix for test data
Confusion Matrix:

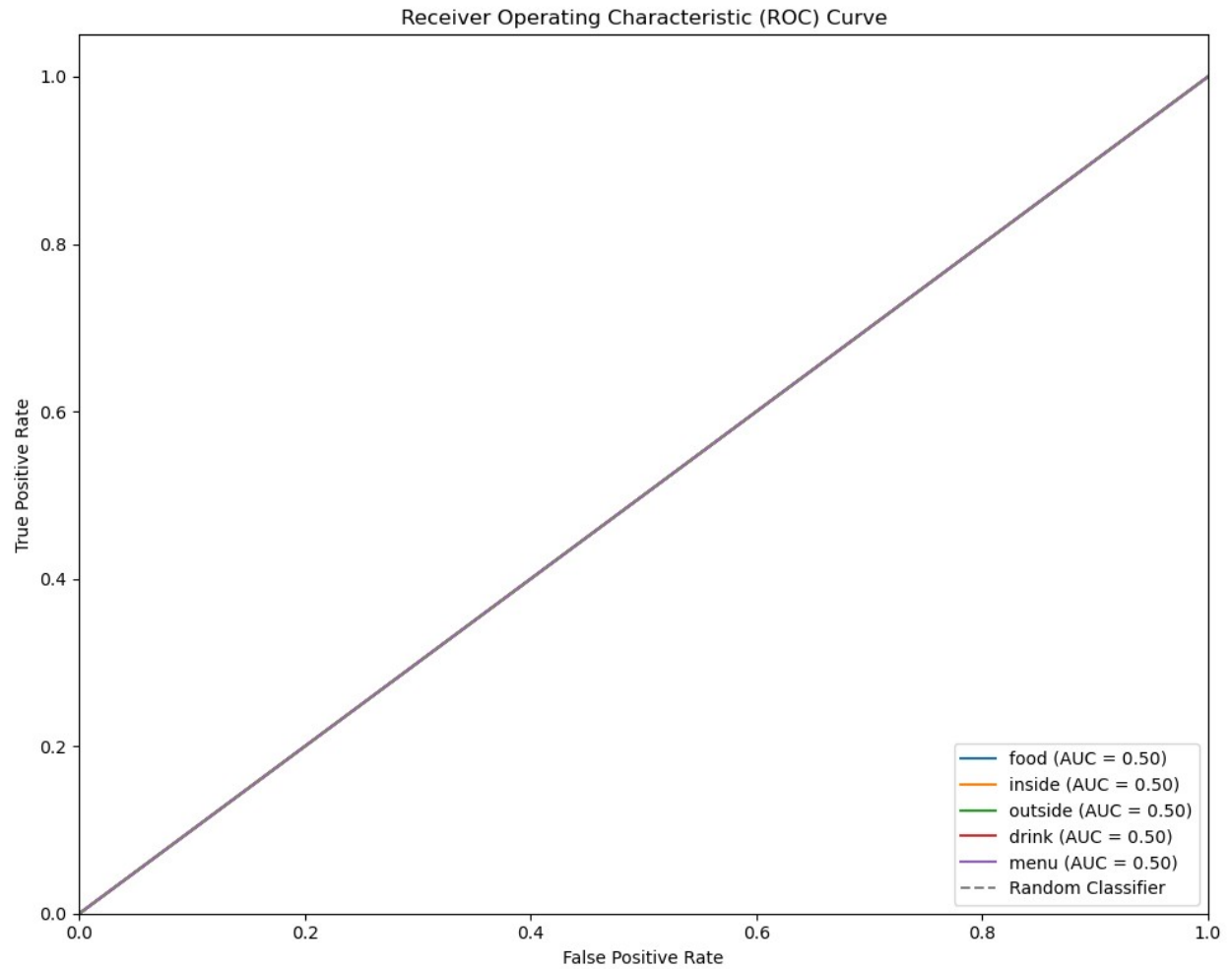
```

	food	inside	outside	drink	menu
food	0	54	0	0	0
inside	0	52	0	0	0

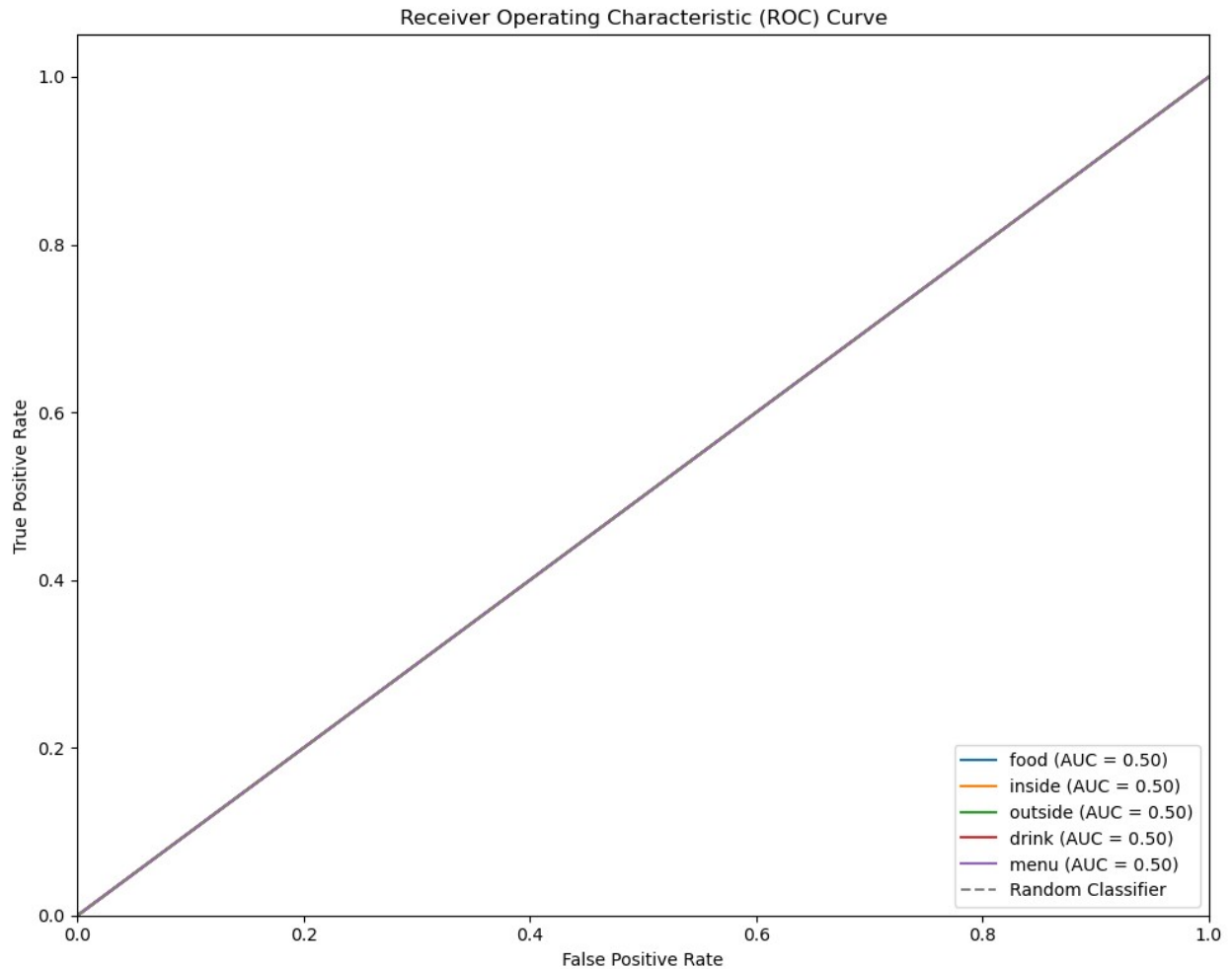
outside	0	58	0	0	0
drink	0	58	0	0	0
menu	0	57	0	0	0



R0C for train data



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.50
AUC for class drink: 0.50
AUC for class menu: 0.50
ROC for test data
```



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.50
AUC for class drink: 0.50
AUC for class menu: 0.50
```

lime

```
explain_with_lime(model, x_test[0], model_type="vgg")
{"model_id": "a7ed0a5ef49f4c2682792c3ad39e3fe1", "version_major": 2, "version_minor": 0}

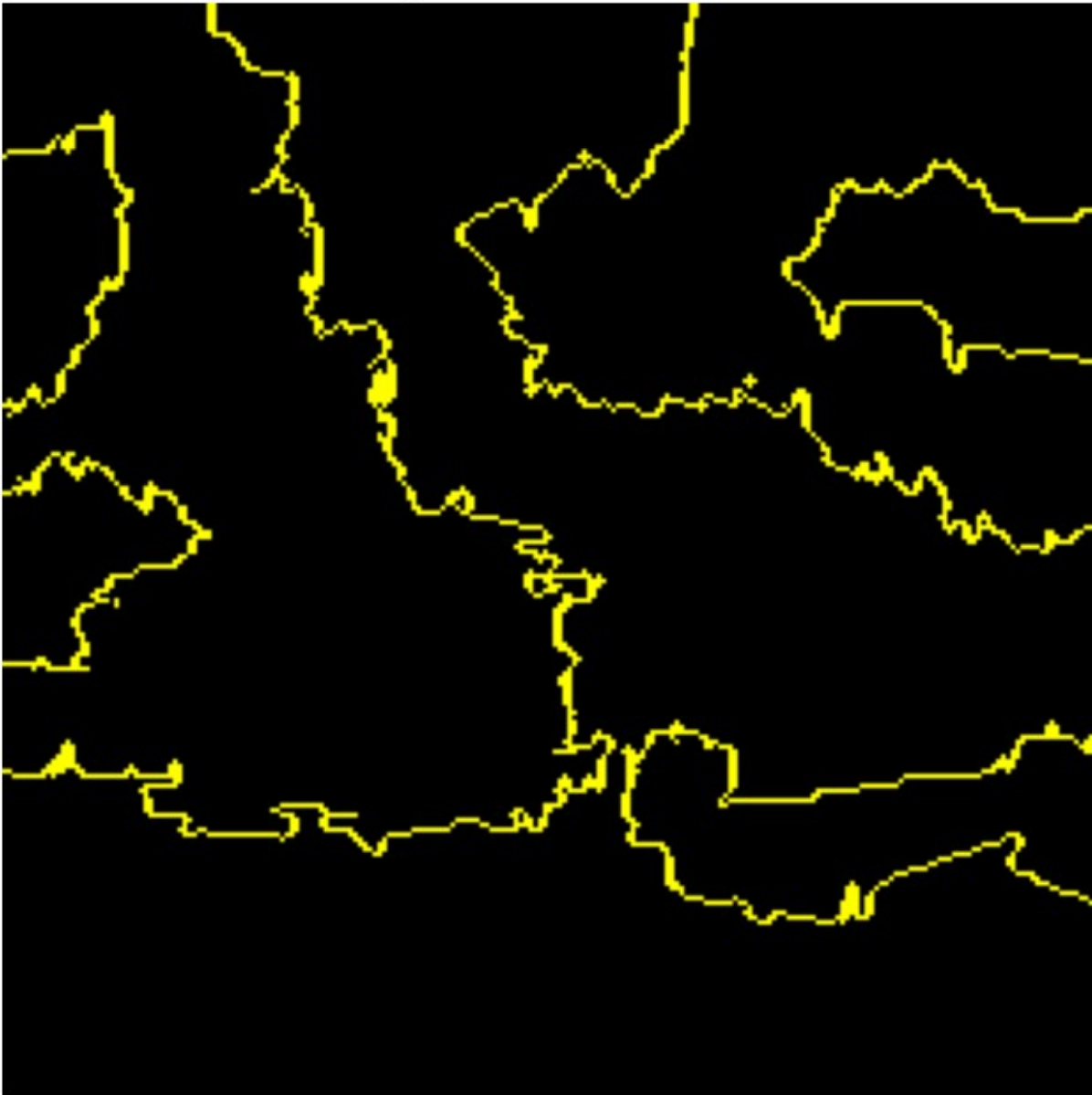
1/1 _____ 1s 530ms/step
1/1 _____ 1s 535ms/step
1/1 _____ 0s 477ms/step
1/1 _____ 0s 459ms/step
1/1 _____ 1s 541ms/step
1/1 _____ 1s 509ms/step
```

1/1	_____	1s	511ms/step
1/1	_____	0s	445ms/step
1/1	_____	0s	485ms/step
1/1	_____	1s	501ms/step
1/1	_____	1s	551ms/step
1/1	_____	1s	510ms/step
1/1	_____	0s	446ms/step
1/1	_____	0s	489ms/step
1/1	_____	0s	485ms/step
1/1	_____	0s	493ms/step
1/1	_____	1s	511ms/step
1/1	_____	1s	544ms/step
1/1	_____	1s	541ms/step
1/1	_____	1s	511ms/step
1/1	_____	1s	532ms/step
1/1	_____	0s	461ms/step
1/1	_____	1s	517ms/step
1/1	_____	0s	451ms/step
1/1	_____	1s	540ms/step
1/1	_____	0s	477ms/step
1/1	_____	0s	450ms/step
1/1	_____	1s	505ms/step
1/1	_____	1s	509ms/step
1/1	_____	1s	549ms/step
1/1	_____	0s	445ms/step
1/1	_____	0s	445ms/step
1/1	_____	1s	526ms/step
1/1	_____	1s	524ms/step
1/1	_____	0s	484ms/step
1/1	_____	1s	540ms/step
1/1	_____	1s	522ms/step
1/1	_____	0s	461ms/step
1/1	_____	0s	462ms/step
1/1	_____	1s	532ms/step
1/1	_____	0s	445ms/step
1/1	_____	0s	454ms/step
1/1	_____	0s	454ms/step
1/1	_____	0s	499ms/step
1/1	_____	0s	500ms/step
1/1	_____	0s	465ms/step
1/1	_____	1s	526ms/step
1/1	_____	1s	537ms/step
1/1	_____	1s	502ms/step
1/1	_____	1s	552ms/step
1/1	_____	1s	524ms/step
1/1	_____	0s	446ms/step
1/1	_____	1s	535ms/step
1/1	_____	0s	442ms/step
1/1	_____	0s	450ms/step

```
1/1 _____ 1s 548ms/step
1/1 _____ 0s 453ms/step
1/1 _____ 1s 518ms/step
1/1 _____ 0s 446ms/step
1/1 _____ 0s 436ms/step
1/1 _____ 1s 543ms/step
1/1 _____ 0s 476ms/step
1/1 _____ 1s 534ms/step
1/1 _____ 0s 426ms/step
1/1 _____ 0s 441ms/step
1/1 _____ 0s 462ms/step
1/1 _____ 0s 435ms/step
1/1 _____ 1s 518ms/step
1/1 _____ 0s 469ms/step
1/1 _____ 0s 492ms/step
1/1 _____ 0s 459ms/step
1/1 _____ 0s 462ms/step
1/1 _____ 0s 460ms/step
1/1 _____ 1s 501ms/step
1/1 _____ 0s 452ms/step
1/1 _____ 0s 487ms/step
1/1 _____ 0s 486ms/step
1/1 _____ 1s 519ms/step
1/1 _____ 1s 517ms/step
1/1 _____ 1s 541ms/step
1/1 _____ 1s 550ms/step
1/1 _____ 1s 516ms/step
1/1 _____ 1s 551ms/step
1/1 _____ 1s 510ms/step
1/1 _____ 1s 519ms/step
1/1 _____ 1s 506ms/step
1/1 _____ 1s 525ms/step
1/1 _____ 1s 533ms/step
1/1 _____ 1s 517ms/step
1/1 _____ 1s 507ms/step
1/1 _____ 0s 453ms/step
1/1 _____ 1s 552ms/step
1/1 _____ 1s 542ms/step
1/1 _____ 0s 493ms/step
1/1 _____ 0s 445ms/step
1/1 _____ 1s 522ms/step
1/1 _____ 1s 545ms/step
1/1 _____ 1s 505ms/step
1/1 _____ 1s 534ms/step
1/1 _____ 1s 517ms/step
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-598.6589965820312..1.0].

LIME Explanation for VGG - Class 1



unfreeze all layers and image augmentation

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
```

```
# Load VGG16 without top layers (include_top=False) for fine-tuning
```

```

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# No freezing of any layers (all layers are trainable)
base_model.trainable = True

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=True) # Training=True to ensure
all layers are trainable
x = GlobalAveragePooling2D()(x) # Pooling the feature map output from
the base model
x = Dense(256, activation='relu')(x) # Fully connected layer
x = Dropout(0.5)(x) # Dropout to prevent overfitting
x = Dense(128, activation='relu')(x) # Another fully connected layer
x = Dropout(0.3)(x) # Dropout
final_output = Dense(5, activation='softmax')(x) # Output layer with
softmax for multi-class classification

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define the ImageDataGenerator with augmentation for the training
data
train_datagen = ImageDataGenerator(
    rotation_range=40, # Random rotation between 0 and 40 degrees
    width_shift_range=0.2, # Horizontal shift
    height_shift_range=0.2, # Vertical shift
    shear_range=0.2, # Shearing transformations
    zoom_range=0.2, # Random zoom
    horizontal_flip=True, # Random horizontal flip
    fill_mode='nearest' # Filling mode for newly created pixels
)

# No augmentation for the test data, just use it as it is
test_datagen = ImageDataGenerator()

# Apply the augmentation to your training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)

start_time = time.time()
# Fine-tune the model with augmented images
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()

```

```

# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")

# Predict the labels for the training set
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)

# Predict the labels for the test set
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)

# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)

print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)

# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)

print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)

```

```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.

```

```

    self._warn_if_super_not_called()

```

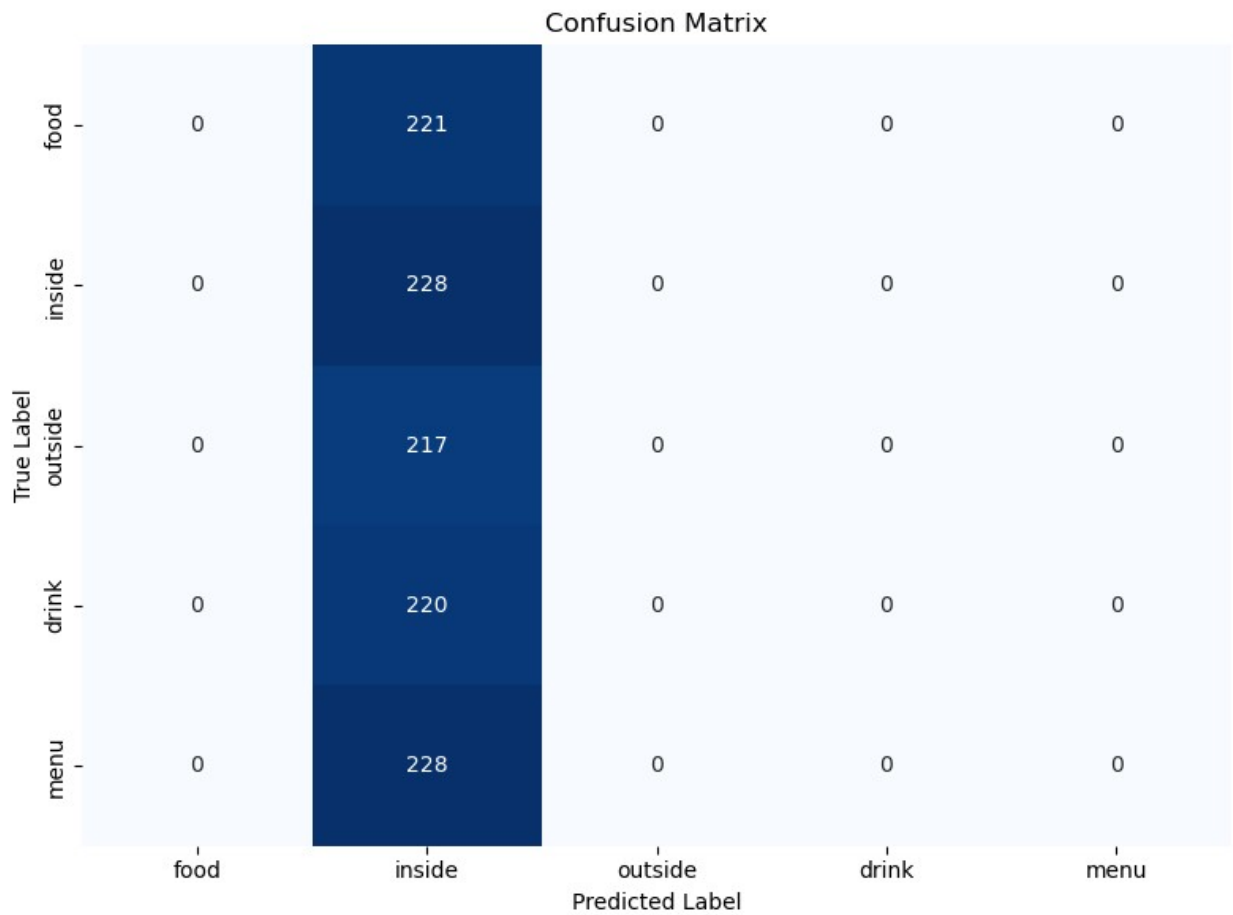
```

Epoch 1/10
35/35 ━━━━━━━━━━━ 194s 5s/step - accuracy: 0.1737 - loss:
1.8172 - val_accuracy: 0.2043 - val_loss: 1.6118
Epoch 2/10
35/35 ━━━━━━━━━━━ 187s 5s/step - accuracy: 0.1833 - loss:
1.6184 - val_accuracy: 0.1864 - val_loss: 1.6095
Epoch 3/10
35/35 ━━━━━━━━━━━ 187s 5s/step - accuracy: 0.2299 - loss:
1.6095 - val_accuracy: 0.1864 - val_loss: 1.6095
Epoch 4/10
35/35 ━━━━━━━━━━━ 188s 5s/step - accuracy: 0.2049 - loss:
1.6092 - val_accuracy: 0.1864 - val_loss: 1.6095
Epoch 5/10
35/35 ━━━━━━━━━━━ 186s 5s/step - accuracy: 0.1940 - loss:
1.6097 - val_accuracy: 0.2079 - val_loss: 1.6094

```



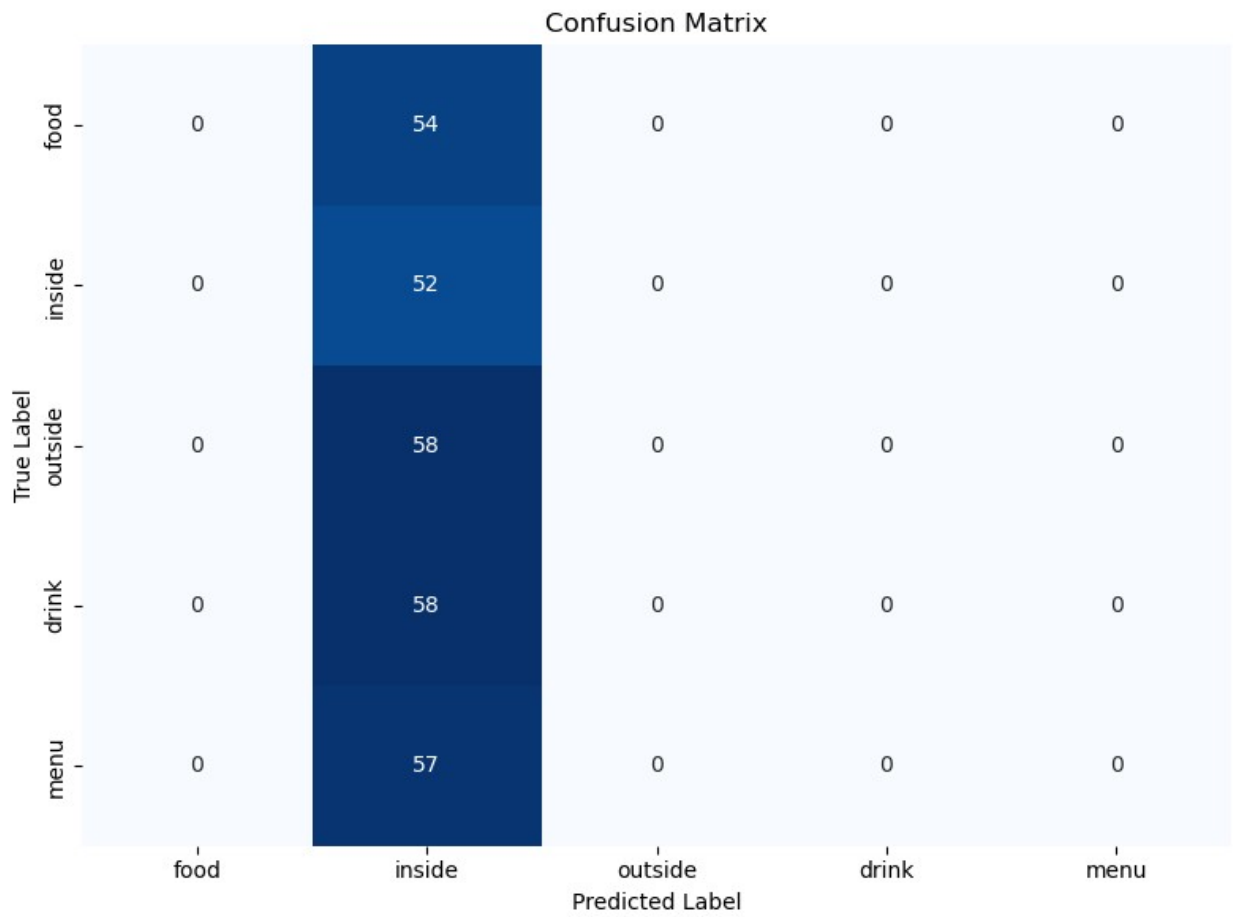
```
Epoch 6/10
35/35 _____ 186s 5s/step - accuracy: 0.1941 - loss:
1.6095 - val_accuracy: 0.2079 - val_loss: 1.6094
Epoch 7/10
35/35 _____ 186s 5s/step - accuracy: 0.2147 - loss:
1.6094 - val_accuracy: 0.2043 - val_loss: 1.6094
Epoch 8/10
35/35 _____ 188s 5s/step - accuracy: 0.1984 - loss:
1.6095 - val_accuracy: 0.2043 - val_loss: 1.6095
Epoch 9/10
35/35 _____ 188s 5s/step - accuracy: 0.1975 - loss:
1.6094 - val_accuracy: 0.1864 - val_loss: 1.6095
Epoch 10/10
35/35 _____ 187s 5s/step - accuracy: 0.2143 - loss:
1.6093 - val_accuracy: 0.1864 - val_loss: 1.6095
Training Time: 1877.27 seconds
35/35 _____ 47s 1s/step
9/9 _____ 12s 1s/step
Confusion Matrix for train data
Confusion Matrix:
      food  inside  outside  drink  menu
food      0      221      0      0      0
inside    0      228      0      0      0
outside   0      217      0      0      0
drink     0      220      0      0      0
menu      0      228      0      0      0
```



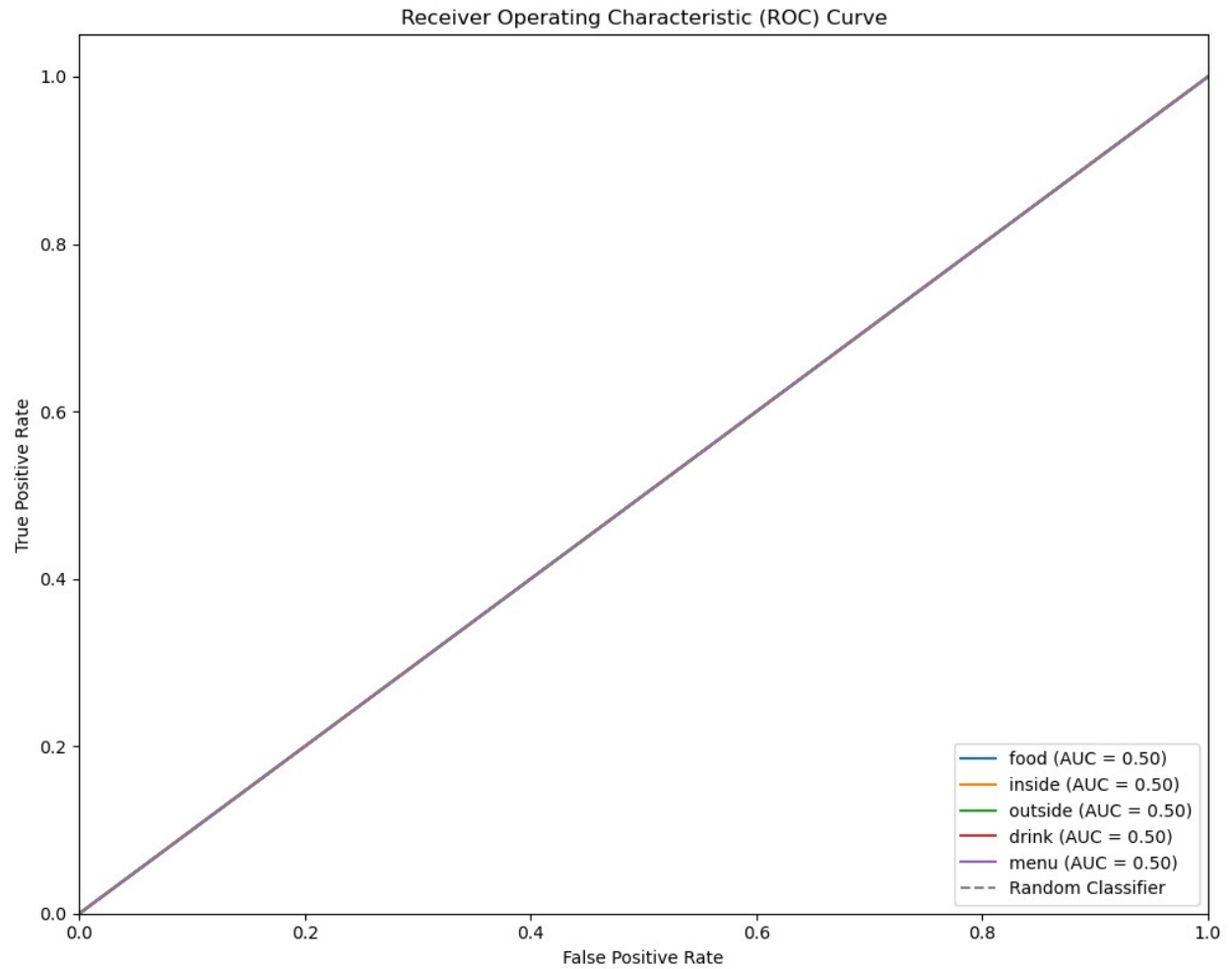
Confusion Matrix for test data

Confusion Matrix:

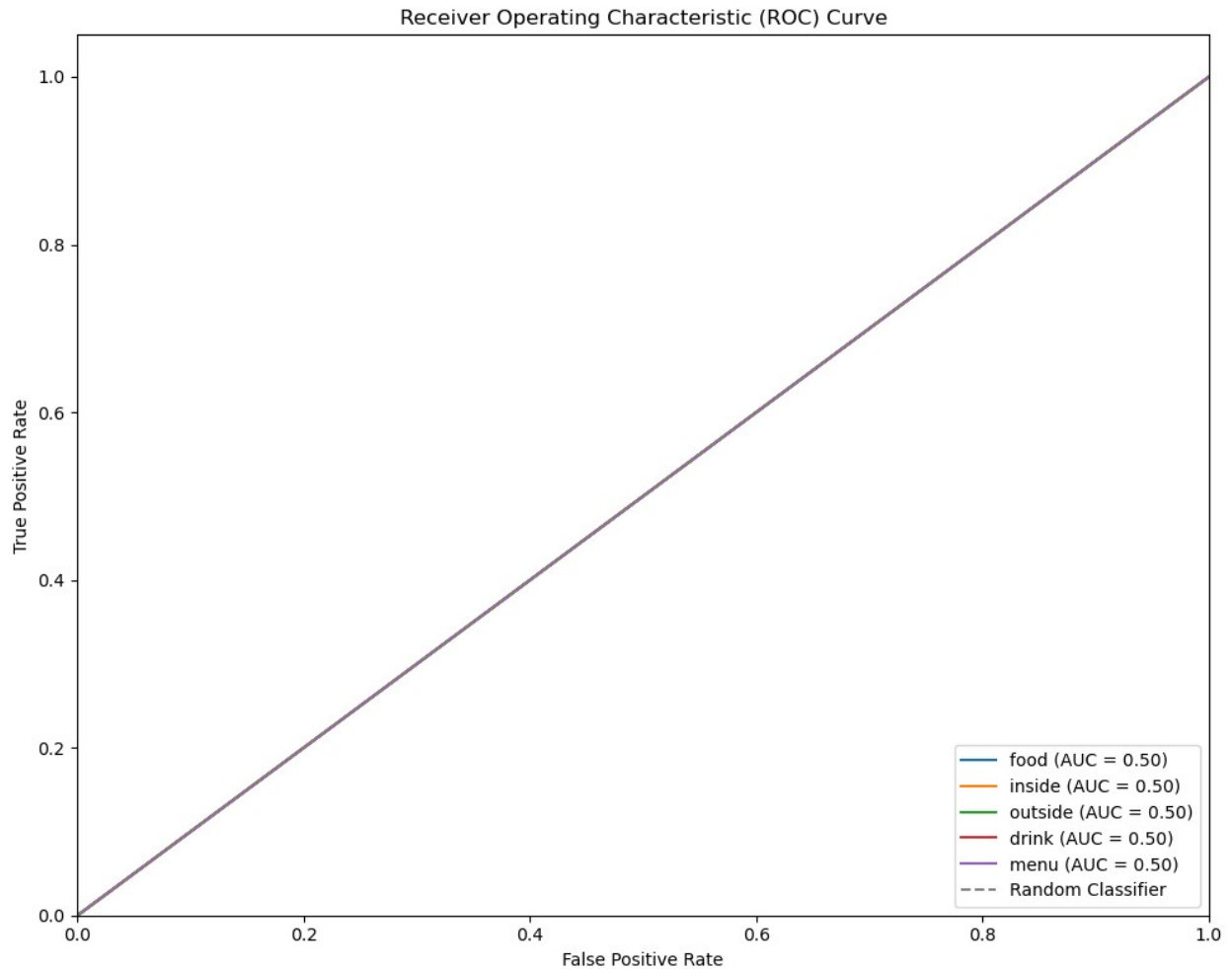
	food	inside	outside	drink	menu
food	0	54	0	0	0
inside	0	52	0	0	0
outside	0	58	0	0	0
drink	0	58	0	0	0
menu	0	57	0	0	0



ROC for train data



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.50
AUC for class drink: 0.50
AUC for class menu: 0.50
ROC for test data
```



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.50
AUC for class drink: 0.50
AUC for class menu: 0.50
```

```
y_test[0]
```

```
2
```

```
explain_with_lime(model, x_test[0], model_type="vgg")
```

```
{"model_id": "cdbc0f7e1b454ea0a05fd57522231542", "version_major": 2, "version_minor": 0}
```

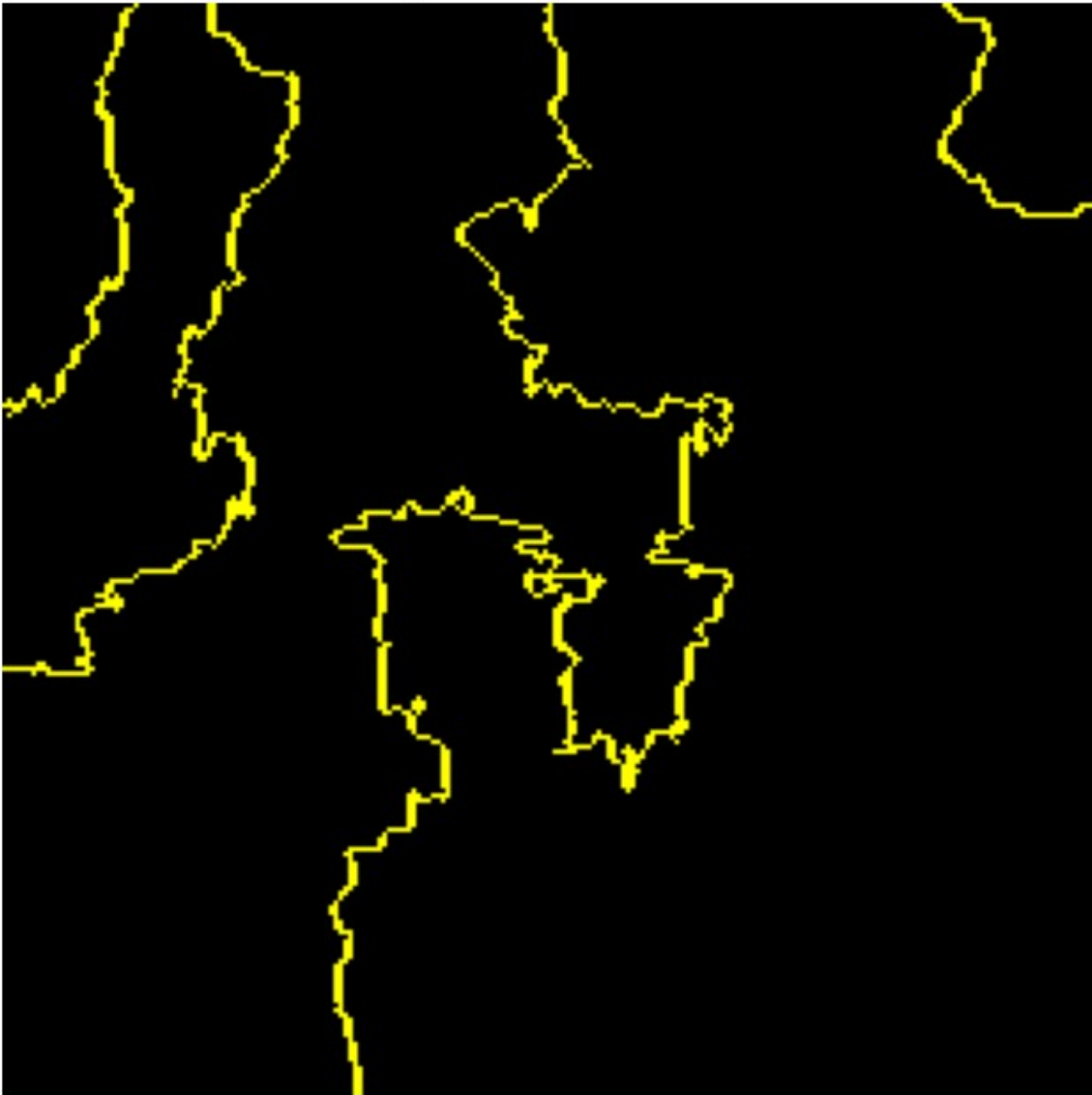
```
1/1 ██████████ 0s 460ms/step
1/1 ██████████ 0s 492ms/step
1/1 ██████████ 1s 554ms/step
1/1 ██████████ 1s 537ms/step
1/1 ██████████ 1s 522ms/step
```

1/1	_____	1s	558ms/step
1/1	_____	1s	532ms/step
1/1	_____	0s	457ms/step
1/1	_____	1s	544ms/step
1/1	_____	1s	540ms/step
1/1	_____	0s	454ms/step
1/1	_____	1s	538ms/step
1/1	_____	0s	460ms/step
1/1	_____	1s	554ms/step
1/1	_____	1s	544ms/step
1/1	_____	1s	547ms/step
1/1	_____	1s	531ms/step
1/1	_____	1s	516ms/step
1/1	_____	1s	528ms/step
1/1	_____	1s	532ms/step
1/1	_____	0s	485ms/step
1/1	_____	1s	520ms/step
1/1	_____	0s	459ms/step
1/1	_____	1s	529ms/step
1/1	_____	1s	513ms/step
1/1	_____	1s	548ms/step
1/1	_____	0s	461ms/step
1/1	_____	1s	553ms/step
1/1	_____	1s	536ms/step
1/1	_____	0s	489ms/step
1/1	_____	1s	538ms/step
1/1	_____	1s	527ms/step
1/1	_____	1s	545ms/step
1/1	_____	0s	452ms/step
1/1	_____	1s	555ms/step
1/1	_____	1s	534ms/step
1/1	_____	1s	532ms/step
1/1	_____	1s	514ms/step
1/1	_____	1s	518ms/step
1/1	_____	1s	513ms/step
1/1	_____	1s	525ms/step
1/1	_____	1s	518ms/step
1/1	_____	0s	467ms/step
1/1	_____	0s	457ms/step
1/1	_____	1s	509ms/step
1/1	_____	0s	476ms/step
1/1	_____	0s	469ms/step
1/1	_____	1s	520ms/step
1/1	_____	1s	543ms/step
1/1	_____	0s	497ms/step
1/1	_____	1s	519ms/step
1/1	_____	1s	509ms/step
1/1	_____	1s	552ms/step
1/1	_____	0s	496ms/step

```
1/1 _____ 1s 551ms/step
1/1 _____ 1s 565ms/step
1/1 _____ 1s 556ms/step
1/1 _____ 1s 518ms/step
1/1 _____ 1s 531ms/step
1/1 _____ 0s 473ms/step
1/1 _____ 1s 500ms/step
1/1 _____ 0s 453ms/step
1/1 _____ 1s 535ms/step
1/1 _____ 0s 452ms/step
1/1 _____ 0s 478ms/step
1/1 _____ 1s 519ms/step
1/1 _____ 0s 485ms/step
1/1 _____ 1s 551ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 1s 547ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 1s 527ms/step
1/1 _____ 1s 559ms/step
1/1 _____ 0s 481ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 1s 559ms/step
1/1 _____ 0s 470ms/step
1/1 _____ 1s 523ms/step
1/1 _____ 1s 508ms/step
1/1 _____ 0s 478ms/step
1/1 _____ 0s 485ms/step
1/1 _____ 0s 452ms/step
1/1 _____ 1s 549ms/step
1/1 _____ 0s 477ms/step
1/1 _____ 0s 450ms/step
1/1 _____ 0s 500ms/step
1/1 _____ 1s 539ms/step
1/1 _____ 1s 517ms/step
1/1 _____ 1s 530ms/step
1/1 _____ 1s 549ms/step
1/1 _____ 0s 494ms/step
1/1 _____ 0s 499ms/step
1/1 _____ 0s 489ms/step
1/1 _____ 1s 525ms/step
1/1 _____ 1s 514ms/step
1/1 _____ 1s 525ms/step
1/1 _____ 0s 477ms/step
1/1 _____ 1s 537ms/step
1/1 _____ 0s 452ms/step
1/1 _____ 0s 475ms/step
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-722.3389892578125..1.0].

LIME Explanation for VGG - Class 1



vss with unfreeze all layers and gaussian blur

```
!pip install opencv-python
```

```
Requirement already satisfied: opencv-python in c:\users\gaurav\anaconda3\lib\site-packages (4.11.0.86)
```

```
Requirement already satisfied: numpy>=1.21.2 in c:\users\gaurav\anaconda3\lib\site-packages (from opencv-python) (1.26.4)
```

```
import tensorflow as tf
import numpy as np
import cv2
```



```

from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Function to apply Gaussian Blur
def apply_gaussian_blur(image):
    """
    Applies Gaussian blur to an image.

    Parameters:
    - image: Input image in NumPy array format.

    Returns:
    - Blurred image.
    """
    image = np.array(image) # Convert to NumPy array (if not already)
    blurred_image = cv2.GaussianBlur(image, (5, 5), 0) # Kernel size
    (5,5), standard deviation 0
    return blurred_image

# Load VGG16 without top layers
base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
base_model.trainable = True # Fine-tune the entire model

# Build the custom model
image_input = Input(shape=(224, 224, 3))
x = base_model(image_input, training=True) # Training=True to fine-
tune
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
final_output = Dense(5, activation='softmax')(x)

model = Model(inputs=image_input, outputs=final_output)

# Compile the model
model.compile(optimizer=Adam(learning_rate=0.0001),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Define ImageDataGenerator with Gaussian Blur
train_datagen =
ImageDataGenerator(preprocessing_function=apply_gaussian_blur)
test_datagen =
ImageDataGenerator(preprocessing_function=apply_gaussian_blur)

```

```
# Apply the generator to training and testing data
train_generator = train_datagen.flow(x_train, y_train, batch_size=32)
validation_generator = test_datagen.flow(x_test, y_test,
batch_size=32)
```

```
start_time = time.time()
# Fine-tune the model
model.fit(train_generator, epochs=10,
validation_data=validation_generator)
end_time = time.time()
```

```
# Calculate training time
training_time = end_time - start_time
print(f"Training Time: {training_time:.2f} seconds")
```

```
# Predict labels for train and test data
y_train_pred = model.predict(x_train)
y_train_pred_labels = np.argmax(y_train_pred, axis=1)
```

```
y_test_pred = model.predict(x_test)
y_test_pred_labels = np.argmax(y_test_pred, axis=1)
```

```
# Confusion Matrix
print("Confusion Matrix for train data")
plot_confusion_matrix(y_train, y_train_pred_labels)
```

```
print("Confusion Matrix for test data")
plot_confusion_matrix(y_test, y_test_pred_labels)
```

```
# ROC Curve
print("ROC for train data")
plot_roc_curve(y_train, y_train_pred)
```

```
print("ROC for test data")
plot_roc_curve(y_test, y_test_pred)
```

Epoch 1/10

```
C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
```

```
self._warn_if_super_not_called()
```

```
35/35 ————— 191s 5s/step - accuracy: 0.1930 - loss:
1.7539 - val_accuracy: 0.2079 - val_loss: 1.6093
```

Epoch 2/10

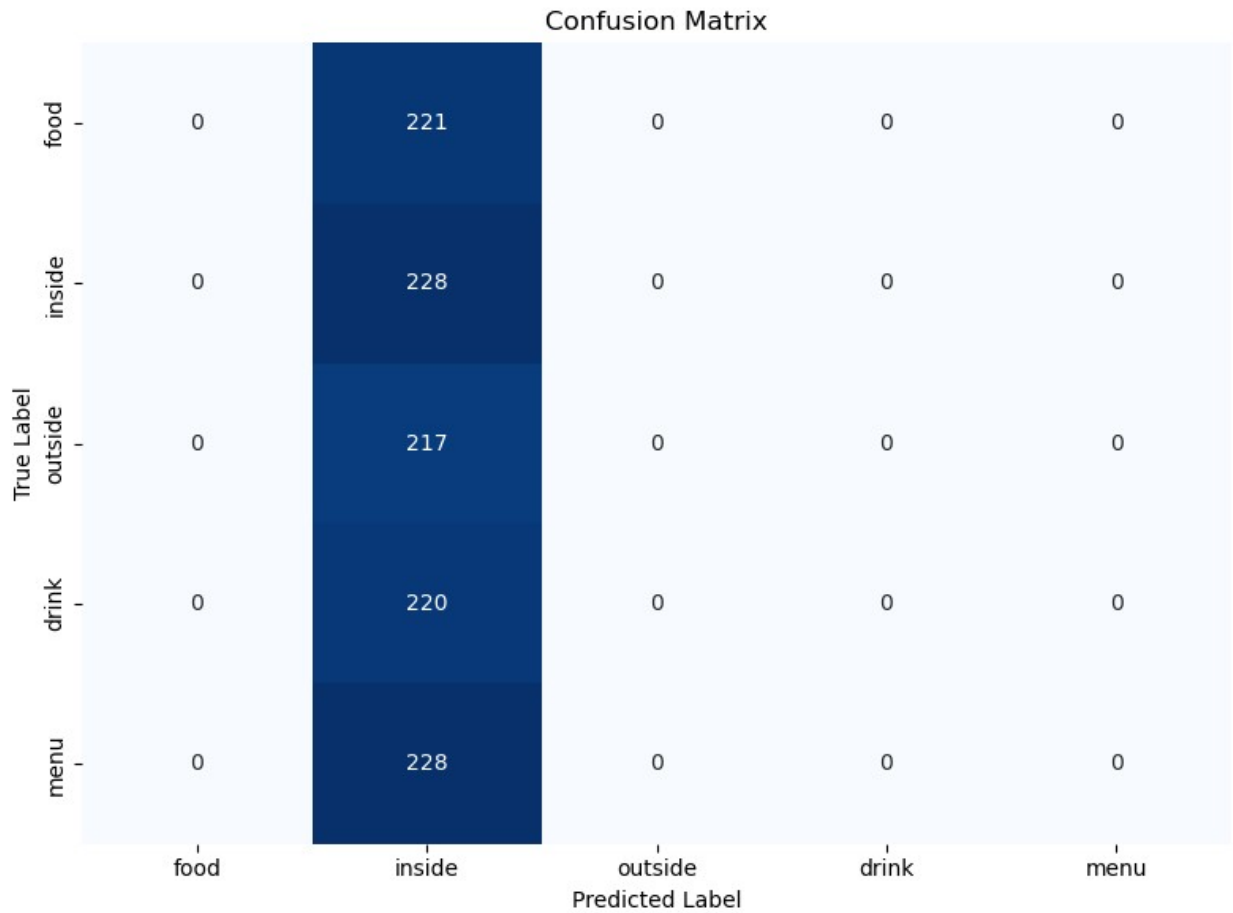
```
35/35 ————— 187s 5s/step - accuracy: 0.2200 - loss:
```

```

1.6095 - val_accuracy: 0.2079 - val_loss: 1.6093
Epoch 3/10
35/35 _____ 187s 5s/step - accuracy: 0.1966 - loss:
1.6094 - val_accuracy: 0.1900 - val_loss: 1.6095
Epoch 4/10
35/35 _____ 186s 5s/step - accuracy: 0.2007 - loss:
1.6099 - val_accuracy: 0.1864 - val_loss: 1.6095
Epoch 5/10
35/35 _____ 185s 5s/step - accuracy: 0.1958 - loss:
1.6094 - val_accuracy: 0.1864 - val_loss: 1.6095
Epoch 6/10
35/35 _____ 185s 5s/step - accuracy: 0.1940 - loss:
1.6094 - val_accuracy: 0.1864 - val_loss: 1.6095
Epoch 7/10
35/35 _____ 185s 5s/step - accuracy: 0.1869 - loss:
1.6096 - val_accuracy: 0.2043 - val_loss: 1.6095
Epoch 8/10
35/35 _____ 186s 5s/step - accuracy: 0.1932 - loss:
1.6094 - val_accuracy: 0.1864 - val_loss: 1.6096
Epoch 9/10
35/35 _____ 186s 5s/step - accuracy: 0.1995 - loss:
1.6093 - val_accuracy: 0.1864 - val_loss: 1.6096
Epoch 10/10
35/35 _____ 186s 5s/step - accuracy: 0.1855 - loss:
1.6097 - val_accuracy: 0.1864 - val_loss: 1.6096
Training Time: 1864.97 seconds
35/35 _____ 47s 1s/step
9/9 _____ 12s 1s/step
Confusion Matrix for train data
Confusion Matrix:

```

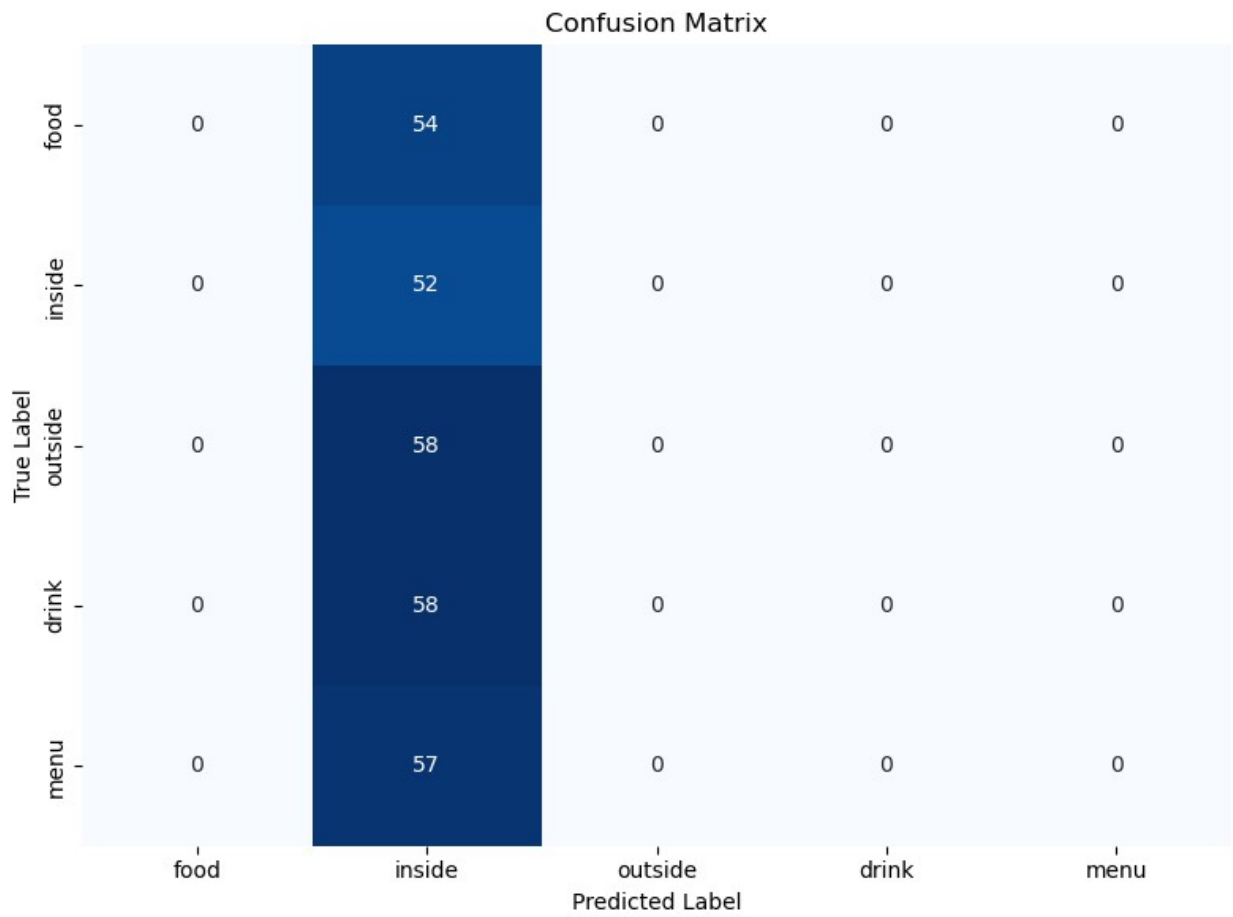
	food	inside	outside	drink	menu
food	0	221	0	0	0
inside	0	228	0	0	0
outside	0	217	0	0	0
drink	0	220	0	0	0
menu	0	228	0	0	0



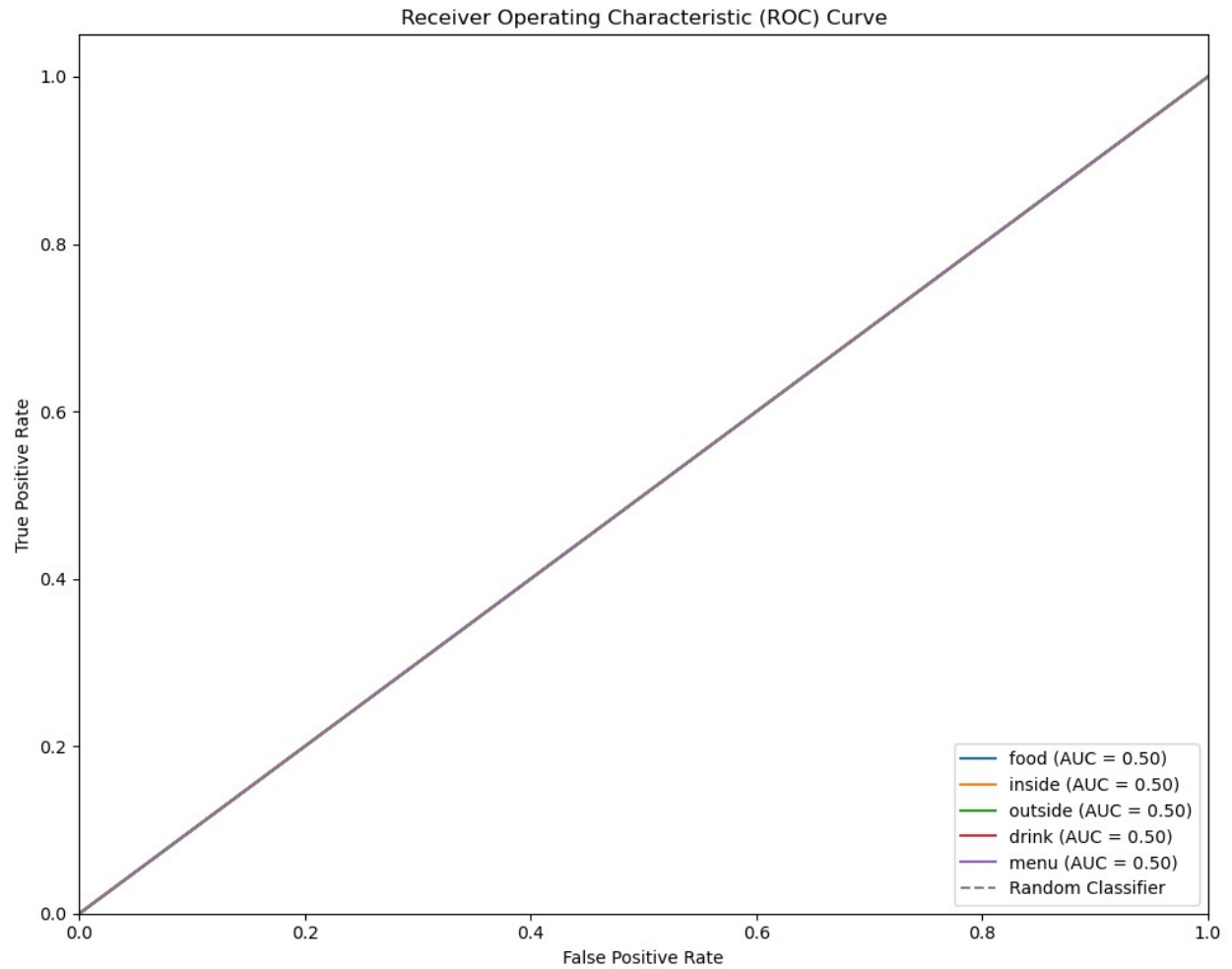
Confusion Matrix for test data

Confusion Matrix:

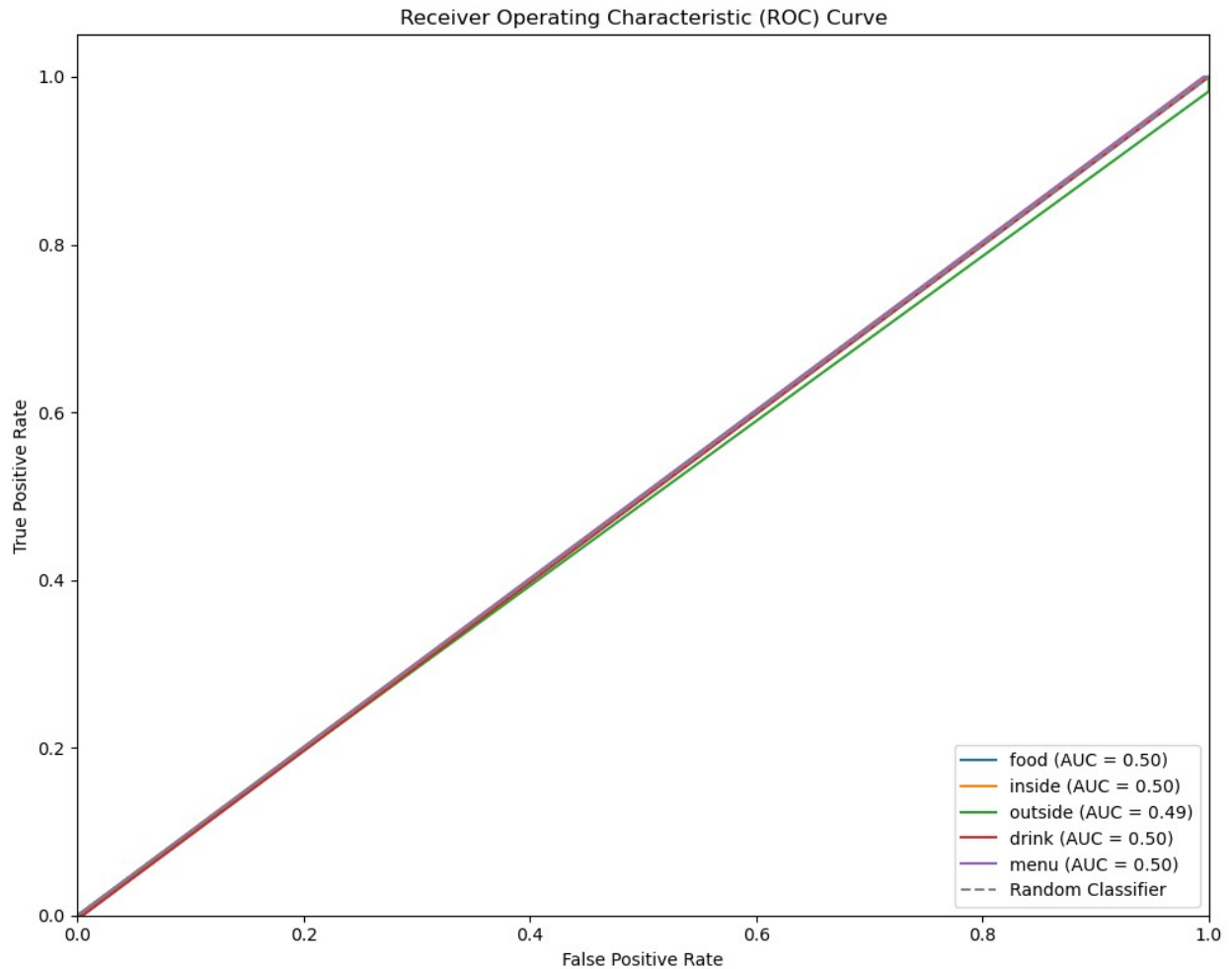
	food	inside	outside	drink	menu
food	0	54	0	0	0
inside	0	52	0	0	0
outside	0	58	0	0	0
drink	0	58	0	0	0
menu	0	57	0	0	0



ROC for train data



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.50
AUC for class drink: 0.50
AUC for class menu: 0.50
ROC for test data
```



```
AUC for class food: 0.50
AUC for class inside: 0.50
AUC for class outside: 0.49
AUC for class drink: 0.50
AUC for class menu: 0.50
```

lime

```
explain_with_lime(model, x_test[0], model_type="vgg")
{"model_id":"fbeed46ea20d41eea051a62e9a586072","version_major":2,"version_minor":0}
```

```
1/1 _____ 0s 495ms/step
1/1 _____ 1s 541ms/step
1/1 _____ 1s 523ms/step
1/1 _____ 1s 544ms/step
1/1 _____ 0s 465ms/step
1/1 _____ 1s 558ms/step
```

1/1	_____	1s	525ms/step
1/1	_____	0s	452ms/step
1/1	_____	1s	548ms/step
1/1	_____	0s	481ms/step
1/1	_____	0s	499ms/step
1/1	_____	0s	467ms/step
1/1	_____	1s	545ms/step
1/1	_____	0s	454ms/step
1/1	_____	0s	483ms/step
1/1	_____	0s	464ms/step
1/1	_____	1s	526ms/step
1/1	_____	0s	456ms/step
1/1	_____	1s	547ms/step
1/1	_____	0s	464ms/step
1/1	_____	0s	474ms/step
1/1	_____	1s	523ms/step
1/1	_____	1s	513ms/step
1/1	_____	0s	469ms/step
1/1	_____	1s	550ms/step
1/1	_____	1s	528ms/step
1/1	_____	1s	559ms/step
1/1	_____	0s	463ms/step
1/1	_____	1s	511ms/step
1/1	_____	1s	546ms/step
1/1	_____	0s	497ms/step
1/1	_____	0s	460ms/step
1/1	_____	1s	546ms/step
1/1	_____	1s	539ms/step
1/1	_____	1s	541ms/step
1/1	_____	0s	458ms/step
1/1	_____	0s	496ms/step
1/1	_____	1s	543ms/step
1/1	_____	1s	513ms/step
1/1	_____	0s	464ms/step
1/1	_____	0s	484ms/step
1/1	_____	0s	462ms/step
1/1	_____	0s	477ms/step
1/1	_____	1s	542ms/step
1/1	_____	0s	479ms/step
1/1	_____	0s	496ms/step
1/1	_____	0s	474ms/step
1/1	_____	1s	566ms/step
1/1	_____	0s	458ms/step
1/1	_____	1s	548ms/step
1/1	_____	0s	456ms/step
1/1	_____	0s	456ms/step
1/1	_____	1s	526ms/step
1/1	_____	0s	499ms/step
1/1	_____	1s	510ms/step


```
1/1 _____ 0s 494ms/step
1/1 _____ 1s 538ms/step
1/1 _____ 0s 493ms/step
1/1 _____ 1s 539ms/step
1/1 _____ 1s 532ms/step
1/1 _____ 1s 504ms/step
1/1 _____ 1s 583ms/step
1/1 _____ 1s 549ms/step
1/1 _____ 1s 508ms/step
1/1 _____ 0s 455ms/step
1/1 _____ 0s 460ms/step
1/1 _____ 1s 530ms/step
1/1 _____ 0s 449ms/step
1/1 _____ 0s 496ms/step
1/1 _____ 1s 528ms/step
1/1 _____ 0s 472ms/step
1/1 _____ 0s 474ms/step
1/1 _____ 1s 538ms/step
1/1 _____ 1s 533ms/step
1/1 _____ 1s 541ms/step
1/1 _____ 1s 537ms/step
1/1 _____ 1s 555ms/step
1/1 _____ 1s 531ms/step
1/1 _____ 1s 544ms/step
1/1 _____ 1s 548ms/step
1/1 _____ 0s 461ms/step
1/1 _____ 1s 509ms/step
1/1 _____ 0s 455ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 1s 501ms/step
1/1 _____ 0s 495ms/step
1/1 _____ 1s 536ms/step
1/1 _____ 1s 508ms/step
1/1 _____ 1s 545ms/step
1/1 _____ 1s 543ms/step
1/1 _____ 1s 546ms/step
1/1 _____ 0s 465ms/step
1/1 _____ 1s 553ms/step
1/1 _____ 1s 529ms/step
1/1 _____ 0s 483ms/step
1/1 _____ 1s 527ms/step
1/1 _____ 0s 454ms/step
1/1 _____ 0s 493ms/step
1/1 _____ 1s 519ms/step
1/1 _____ 1s 506ms/step
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-846.0189819335938..1.0].

LIME Explanation for VGG - Class 1

